Directum RX

HR Pro 2.7. Прикладная конфигурация личного кабинета. Версия Directum RX 25.2

© Компания Directum, 2025

Содержание

Введение ... 4

Предметная область решения ... 5

Утилита ess ... 11

Прикладные решения ... 11

new – создание конфигурации для нового решения ... 12

install – установка прикладного решения ... 12

update – обновление прикладного решения ... 13

activate/deactivate – активация/отключение прикладного решения ... 13

sln – вывод списка прикладных решений ... 13

uninstall – удаление прикладного решения ... 14

clear – удаление прикладных решений и объектов ... 14

dbversion – получение версии схемы данных ... 15

normalize – нормализация прикладного решения ... 15

upgrade – модернизация прикладного решения ... 16

Прикладная конфигурация ... 16

list – вывод списка всех прикладных объектов ... 16

export – экспорт конфигурации прикладного объекта ... 17

import – импорт конфигурации прикладного объекта ... 18

verify – проверка корректности прикладной конфигурации ... 18

Целевые системы ... 19

connect – подключение целевой системы ... 19

disconnect – отключение целевой системы ... 20

sys – вывод списка точек подключения целевых систем ... 20

Работа с файлами ... 20

Конфигурация решения ... 22

authorization – права доступа к объекту ... 22

facilityGroup – группа услуг ... 24

view – представление группы услуг на устройствах ... 25

facility – услуга ... 29

widget – виджет ... 61

dataSourceUrl – адрес источника данных ... 62

Тип виджета itemList ... 63

Тип виджета workItemList ... 65

Тип виджета facilityShortcuts ... 66

Тип виджета templateWidget ... 69

Тип виджета embedded ... 70

Свободно конфигурируемый виджет ... 71

tile – плитка быстрого действия ... 76

function – прикладная функция ... 78

localizedStringValue – строка локализации ... 81

usageAgreement – пользовательское соглашение ... 82

notificationTemplate – шаблон оповещения ... 83

objectCard – карточка объекта ... 84

views – представления карточки объекта ... 85

buttons – кнопки в карточке объекта ... 87

projection – проекция ... 87

properties – свойства проекции ... 88

sources – источники проекции ... 88

extension – расширение объекта ... 90
page – прикладная страница ... 91
component – сторонний компонент ... 92
valueConverter – прикладной конвертер ... 95
script – прикладной сценарий ... 99
role – роль ... 102
Иконки решения ... 103
Прикладные функции платформы личного кабинета ... 105
Обработчики событий ... 105
В офертах ... 106
В услугах ... 107
В карточках объектов ... 113
В прикладных страницах ... 115
В виджетах ... 115
Пространство имен ESS ... 115
actions – действия ... 116
activities – функции для работы с индикатором активности ... 119
cloneObject – клонирует объект ... 120
closeElement – закрывает элемент ... 120
constants – системные константы ... 120
dialogs – диалоги ... 121
downloadDocument – скачивает документ с сервера по его описанию ... 129
downloadFile – скачивает по ссылке файл с сервера ... 130
enums – объект, который содержит перечисления или дочерние объекты с перечислениями ... 130
formatString – форматирует строку с описателями типа {0} ... 140
getColor – получает цвета из палитры по токену ... 140
hints – функции для работы с панелью подсказок на странице ... 140
L – возвращает строку локализации ... 141
methods – методы ... 142
network – функции для работы с сетевыми запросами ... 144
overlay – функции для работы с оверлеем ... 145
parseDate – функции для получения даты из строки ... 146
parseFloat – расширенная функция выделения числа с плавающей точкой ... 146
preview – функции по работе с предпросмотром ... 146
replaceSubstrings – заменяет все вхождения указанной подстроки на новую подстроку ... 147
scripts – прикладные сценарии ... 147
signing – функции подписания ... 148
simplifyDate – возвращает локализованную дату в виде строки ... 151
toasts – функции для работы с toast-уведомлениями ... 151
updateElement – обновляет элемент ... 153
widgets – функции по работе с виджетами ... 153
Пространство имен FormFunctions ... 153
Пространство имен CardFunctions ... 162
Используемые типы ... 162
Лямбда-функции, доступные в mustache-шаблонах ... 183
Mustache-шаблон ... 185
Стилизация элементов в Mustache-шаблонах ... 185
Рекомендации по оформлению текста ... 187
Физическая структура базы данных ... 190

Введение

СОВЕТ. Это и другие руководства по продукту HR Pro с удобным переходом по ссылкам можно посмотреть на сайте сообщества Directum Club.

Решение «HR Pro. Личный кабинет» – это платформа, которая предоставляет сотрудникам компании легкий доступ к кадровым услугам и документам. Состав услуг и то, в каком виде они будут доступны, определяется прикладной конфигурацией решения.

В руководстве описана прикладная конфигурация решения «HR Pro. Личный кабинет» и рекомендации по ее модификации.

Для работы с прикладной конфигурацией используются:

- утилита ess – основной инструмент для управления конфигурацией;
- файлы конфигурации решений, структура которых задается в узле solution;
- стандартные иконки, доступные для использования в решении;
- функции для выполнения прикладной логики решения;
- прикладные CSS-классы.

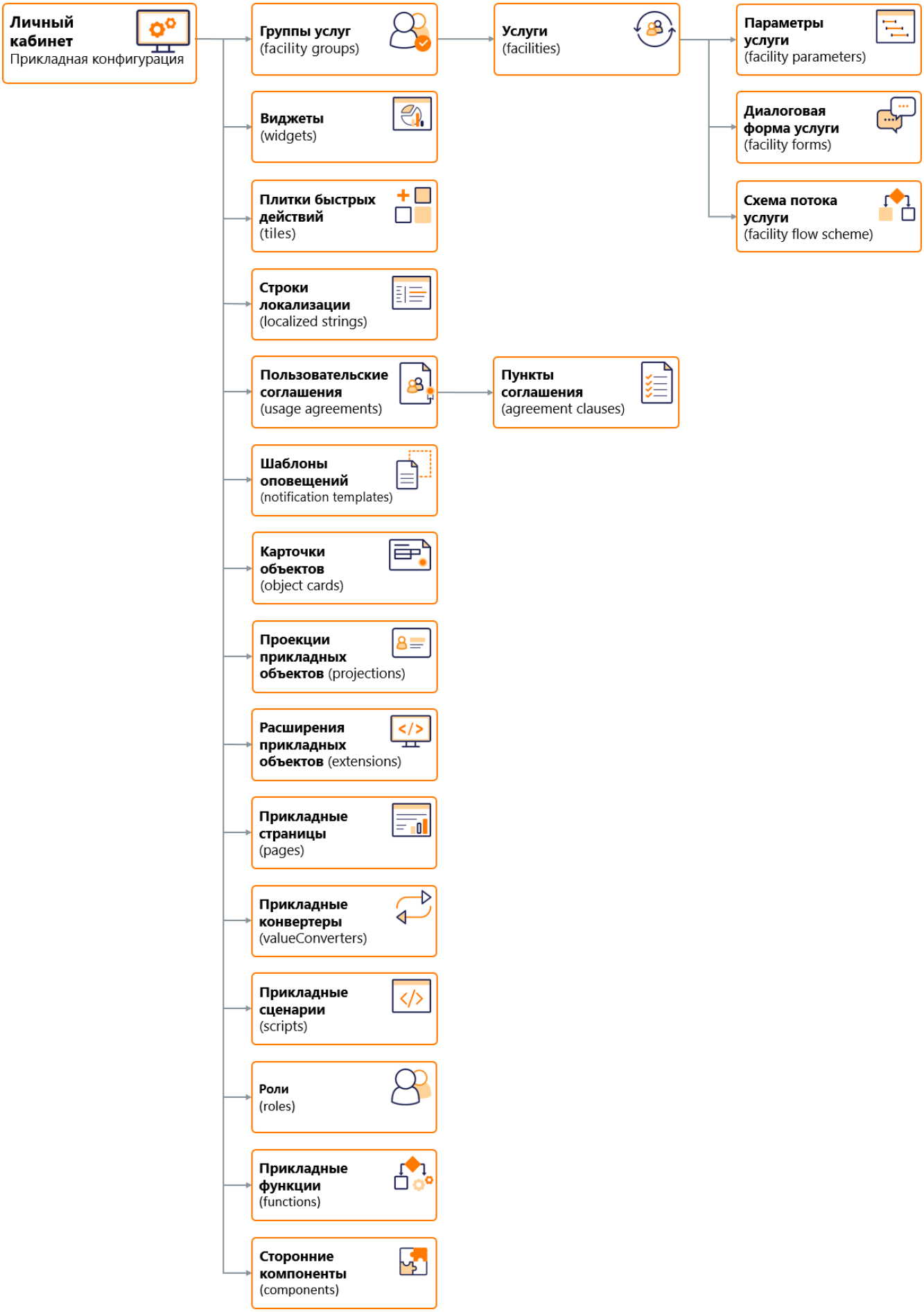
Перед началом работы с прикладной конфигурацией рекомендуется ознакомиться:

- с предметной областью решения «HR Pro. Личный кабинет»;
- с физической структурой базы данных.

Предметная область решения

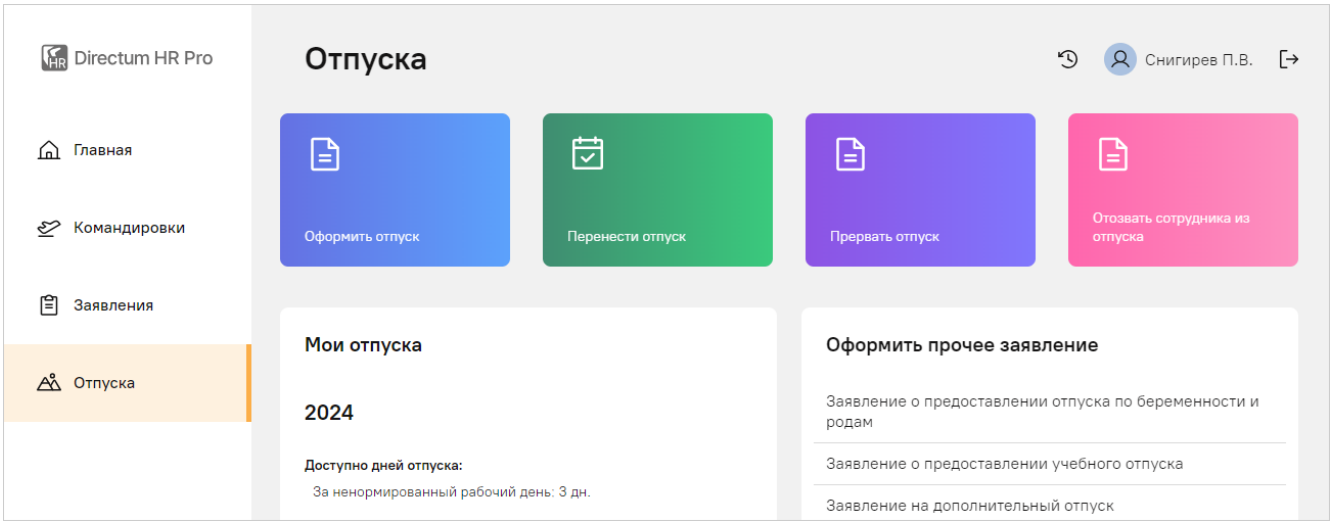
Прикладная конфигурация формирует состав предметной области решения. Предметная область определяет, какие возможности по запросу информации будут у пользователя личного кабинета, а также какие услуги будут ему доступны.

Состав и иерархия объектов предметной области решения приведены на схеме:



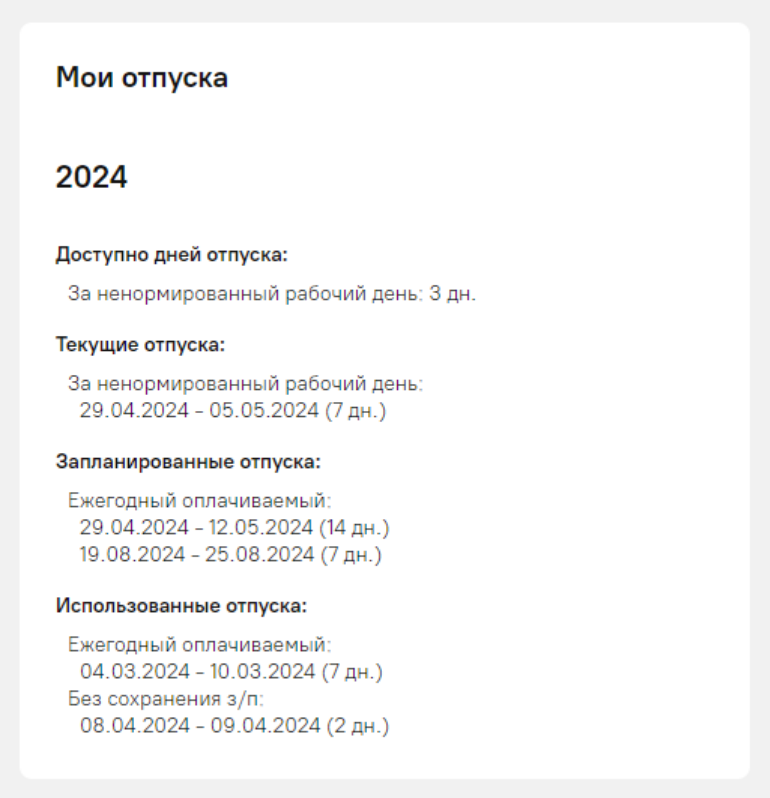
Разделы личного кабинета

Группы услуг (facilityGroup). Используются для логической группировки предоставляемых пользователю услуг. Также определяют точки доступа пользователя к интерфейсу личного кабинета, формируя разделы на панели навигации сайта. Например, группа услуг **Отпуска**:

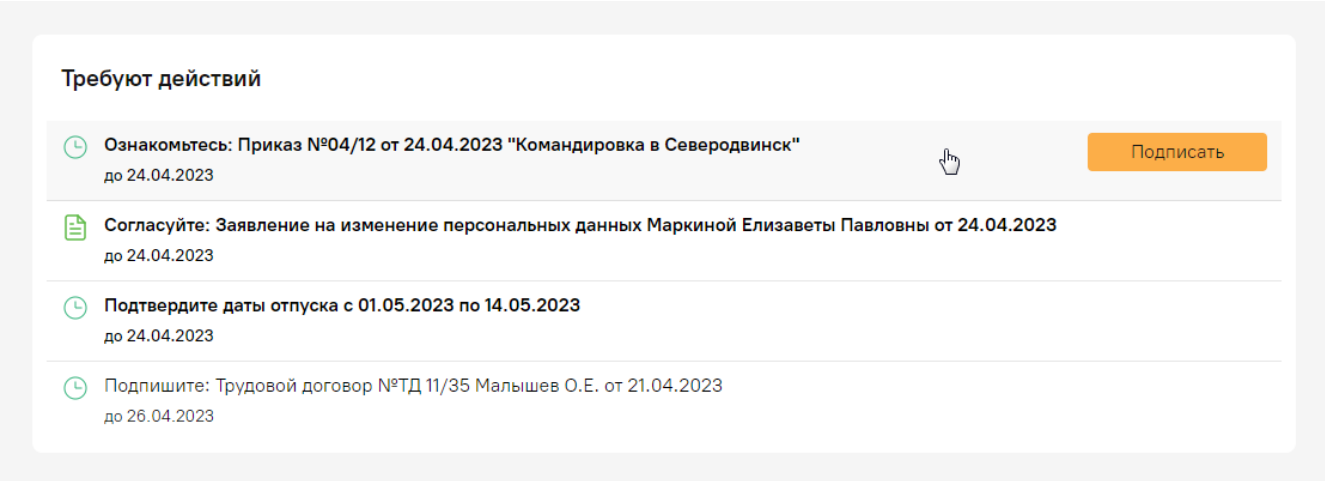


Виджеты (widget). Дочерние компоненты групп услуг, часть доменной структуры решения. Виджет содержит представление определенной информации, либо иные компоненты, с помощью которых пользователь взаимодействует с личным кабинетом. Например:

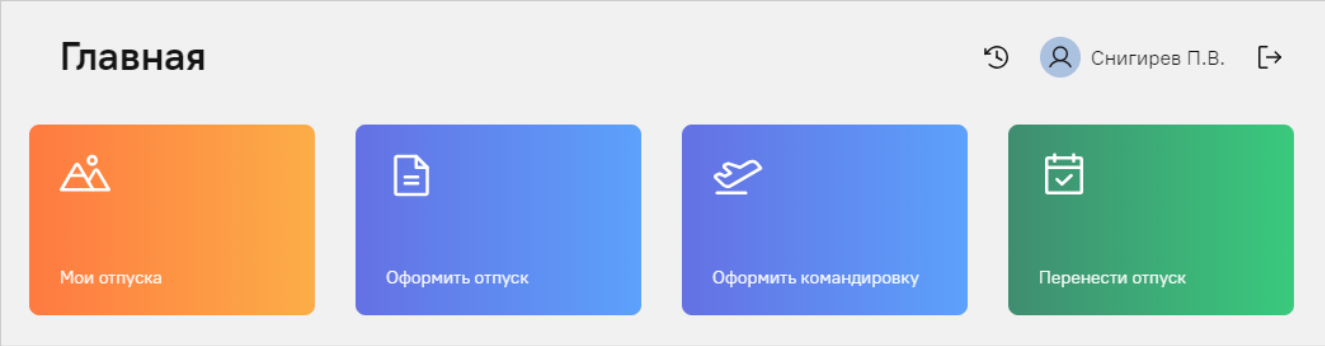
- виджет «Мои отпуска» содержит информацию об отпусках текущего пользователя:



- виджет «Требуют действий» содержит список задач, с которыми пользователю необходимо выполнить определенные действия:



Плитки быстрых действий (tile). Представляет некоторую краткую информацию пользователю и позволяет выполнить действие или переход к другому прикладному объекту.



Услуги (facility). Некоторая работа, которую личный кабинет выполняет для пользователя. Услуга запускается пользователем в личном кабинете и состоит из нескольких шагов, которые формируют бизнес-процесс услуги. Во время выполнения услуги отдельные ее шаги могут выполняться как внутри самого личного кабинета, так и в различных подключаемых внешних системах (целевые системы). Это определяется конфигурацией схемы потока услуги. Для параметризированного запуска услуги в конфигурации можно определить набор параметров, которые будут запрошены у пользователя в диалоговой форме при запуске услуги. Например, «Оформить отпуск» или «Оформить перенос отпуска».

Параметры услуги (facilityParameter). С каждой услугой связан набор параметров, которые определяют опции ее выполнения. Параметры задаются в конфигурации услуги. Каждый параметр определяется типом, например строка или число, и значением, например максимальная длина или точность.

Диалоговая форма услуги (facilityFormItem). Часть значений параметров услуги может быть запрошена у пользователя при ее запуске. Для этого используется форма услуги, которая содержит набор элементов формы. Каждый элемент формы определяет визуальное представление параметра в диалоге запуска услуги. Также он может быть привязан к параметру услуги.

Схема потока услуги (flowSteps). Определяет процесс выполнения услуги. Схема состоит из отдельных шагов и связей между ними. При запуске услуги по схеме создается экземпляр услуги, который в дальнейшем обрабатывается движком услуг, последовательно проходя по всем шагам.

Инструменты взаимодействия с пользователем

Строки локализации (localizedStringValue). Используются для перевода интерфейса на различные языки. Строка локализации определяется кодом, по которому можно получить одно из нескольких значений для заданного языка. Для локализации значений отдельных свойств других доменных объектов используются ссылки на строки локализации по их коду.

Пример: в заголовке группы услуг «Документы» может быть указано значение **@.FACILITYGROUPS_TITLE_DOCUMENTS**, что представляет собой ссылку на строку локализации с кодом **FACILITYGROUPS_TITLE_DOCUMENTS** в текущем прикладном решении.

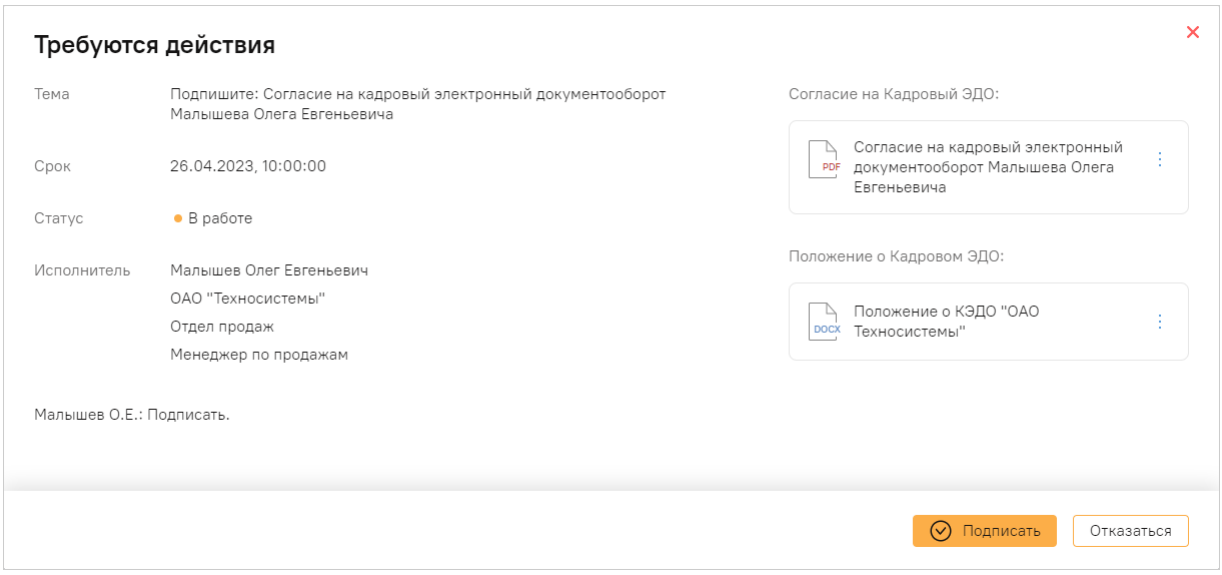
Пользовательские соглашения (usageAgreement). Форма договора между пользователем личного кабинета и организацией-владельцем ресурса. Состоит из текста соглашения и отдельных его пунктов.

Соглашение отображается в личном кабинете при регистрации пользователя и при обновлении/создании нового соглашения. Пользователь может принять соглашение или отказаться от его принятия. Если соглашение обязательно к принятию, пользователь допускается к работе в личном кабинете только после того, как примет соглашение.

Пункты соглашения (agreementClause). Отдельные опции соглашения. При принятии соглашения пользователь может принять или отказаться от принятия отдельных его пунктов.

Шаблоны оповещений (notificationTemplate). Шаблон используется для формирования оповещения пользователю в ответ на событие внутри личного кабинета либо при получении сообщения из внешней системы. Шаблон содержит схему преобразования данных события или сообщения в модель оповещения, а также параметры привязки шаблона к конкретному событию или сообщению.

Карточки объектов (objectCard). Используются для настройки отображения карточек объектов, которые открываются из виджетов. Для карточек добавляется возможность настроить заголовок, тело, вложения и обработчики кнопок.



Прикладные объекты

Проекции прикладных объектов (projection). Используются для настройки источника получения данных для виджетов.

Расширения прикладных объектов (extension). Используются для расширения прикладных объектов в установленных прикладных решениях.

Прикладные страницы (page). Используются для настройки отображения страницы прикладного решения.

Прикладные конвертеры (valueConverter). Используются для настройки конвертации значений прикладных объектов в проекциях.

Прикладные сценарии (script). Используются для выполнения пользовательских сценариев.

Роли пользователей (role). Используются для ограничения доступа пользователей к прикладным объектам решения.

Прикладные функции (functions). Используются для выполнения запросов к удаленным сервисам, запуска сценариев и выполнения функций целевых систем.

Сторонние компоненты (components). Используются для настройки подключаемых сторонних компонентов.

Утилита ess

Cli-утилита **ess** – это основной инструмент для работы с прикладной конфигурацией личного кабинета. Утилита используется для работы с:

- прикладными решениями;
- прикладной конфигурацией;
- целевыми системами;
- файлами.

Утилита входит в комплект поставки решения «HR Pro. Личный кабинет» и находится в папке .\Tools\Ess-Cli-win-x64.zip.

Чтобы настроить утилиту на работу с конкретной базой данных, в конфигурационном файле утилиты appsettings.json в параметре **Database** добавьте строку подключения к базе данных.

Пример настройки:

```
{
// Строки подключения
  "ConnectionStrings": {
    // Строка подключения к базе данных (ProviderName=;Data Source=;Initial
Catalog=;Integrated Security=False;User ID=;Password=;)
    "Database": "Data Source=mssql\\sql2019;Initial Catalog=ess;Integrated Security=False;User
ID=sa;Password=11111",
  }
}
```

Чтобы получить справку по командам утилиты, используйте параметр **[-?|-h|--help]**:

```
ess --help
```

Чтобы получить справку по конкретной команде, используйте параметр **[-?|-h|--help]** вместе с именем команды:

```
ess <ИмяКоманды> --help
```

где **ИмяКоманды** – название команды, по которой запрашивается справка.

Пример:

```
ess install --help
```

Прикладные решения

Для работы с прикладными решениями утилита **ess** предоставляет следующие команды:

new	Создание конфигурации для нового прикладного решения
install	Установка прикладного решения
update	Обновление прикладного решения
activate/deactivate	Активация/отключение установленного решения
sln	Вывод списка прикладных решений
uninstall	Удаление прикладного решения
clear	Удаление всех прикладных решений и объектов из базы данных
dbversion	Получение версии схемы данных

new	Создание конфигурации для нового прикладного решения
normalize	Нормализация прикладного решения
upgrade	Модернизация прикладного решения

new – создание конфигурации для нового решения

Команда **new** используется для создания пустой конфигурации нового прикладного решения.

Синтаксис:

```
ess new <ИмяРешения> [-o|--Output <ИмяФайлаКонфигурацииРешения>]
```

где:

- **ИмяРешения** – имя нового решения;
- **ИмяФайлаКонфигурацииРешения** – имя XML-файла конфигурации. Если имя файла не задано, то оно будет сформировано как <ИмяРешения>.xml.

install – установка прикладного решения

Команда **install** используется для установки решения в базу данных личного кабинета. Для этого требуется конфигурация решения в виде XML-файла.

Синтаксис:

```
ess install <ПутьДоФайлаКонфигурацииРешения> [-a|--activate]  [-q|--quiet]
```

где:

- **ПутьДоФайлаКонфигурацииРешения** – путь до XML-файла конфигурации;
- **-a|--activate** – флаг активации решения. Если указан, то решение активируется;
- **-q|--quiet** – флаг установки в неинтерактивном режиме. Если указан, то запрос активации решения не отправляется.

При установке решения:

1. Происходит проверка, было ли решение установлено раньше. Если оно уже установлено, возвращается соответствующая ошибка.
2. В базу данных в таблицу **Solutions** добавляется информация о решении, при этом в статусе решения устанавливается значение **Deploying**, что исключает решение из активных, с которыми могут работать пользователи.
3. В базу данных последовательно записываются все прикладные объекты, входящие в дерево решения. Так как объекты привязаны к решению, а решение неактивно, на работу пользователей добавление объектов не повлияет.
4. В базу данных добавляются строки локализации, имя группы услуг устанавливается равным имени решения.
5. После того как все прикладные объекты конфигурации будут импортированы из XML-файла конфигурации решения в базу данных личного кабинета:
   - если при установке был указан флаг **--activate**, то у решения устанавливается статус **Active** и решение становится активным. Пользователи могут работать с решением;

- если флаги **activate** и **quiet** не были указаны, то администратор получает запрос на активацию решения.

update – обновление прикладного решения

Команда **update** используется для обновления прикладного решения в базе данных личного кабинета.

Синтаксис:

```
ess update <ИмяФайлаКонфигурацииРешения>  [-a|--activate]  [-q|--quiet]
```

где:

- **ИмяФайлаКонфигурацииРешения** – название конфигурационного XML-файла решения;
- **-a|--activate** – флаг активации решения. Если указан, то решение активируется;
- **-q|--quiet** – флаг установки в неинтерактивном режиме. Если указан, то запрос активации решения не отправляется.

Если флаги **activate** и **quite** не указаны, то администратор получает запрос на активацию решения.

Команда выполняется в транзакции, поэтому, если возникли ошибки, изменения откатываются.

activate/deactivate – активация/отключение прикладного решения

Команда **activate** используется для активации решения.

Синтаксис:

```
ess activate <ИмяРешения>
```

где **ИмяРешения** – имя решения.

Выполнение команды приводит к смене состояния решения на **Active**, решение становится доступно пользователям.

Команда **deactivate** используется для отключения решения.

Синтаксис:

```
ess deactivate <ИмяРешения>
```

где **ИмяРешения** – имя решения.

Выполнение команды приводит к смене состояния решения на **Deactivated**, решение становится недоступно пользователям.

sln – вывод списка прикладных решений

Команда **sln** используется для вывода списка всех прикладных решений, установленных в базе данных личного кабинета.

Синтаксис:

```
ess sln
```

В результате отображается информация по всем прикладным решениям: идентификатор, имя, заголовок, версия и статус.

uninstall – удаление прикладного решения

Команда **uninstall** используется для удаления прикладного решения из базы данных личного кабинета.

Синтаксис:

```
ess uninstall <ИмяРешения> [-q|--quiet]
```

где:

- **ИмяРешения** – имя решения;
- **-q|--quiet** – флаг удаления в неинтерактивном режиме. Если указан, то запрос на удаление решения не отправляется.

При удалении решения:

1. Если флаг **--quiet** не был указан, то администратор получает запрос на удаление решения.
2. Проверяется наличие установленного решения в базе данных. Если решение не установлено, то возвращается соответствующая ошибка.
3. Решение в базе данных переводится в состояние **Deactivated**.
4. Строка с решением удаляется из таблицы **Solutions**, при этом срабатывает каскадное удаление по всем остальным таблицам.

clear – удаление прикладных решений и объектов

Команда **clear** используется для удаления всех прикладных решений и прикладных объектов в базе данных личного кабинета.

Синтаксис:

```
ess clear
```

В качестве подтверждения команда запрашивает имя базы данных. Имя должно совпадать с тем, что указано в файле appsettings.json утилиты **ess**.

В результате выполнения команды удаляются объекты:

- Converters – конвертеры;
- Facilities – услуги;
- FacilityFlows – потоки услуг;
- FacilityGroups – группы услуг;
- Tiles – плитки быстрых действий;
- Widgets – виджеты;
- LocalizedStrings – строки локализации;

- ObjectCards – карточки прикладных объектов;
- Pages – прикладные страницы;
- Components – сторонние компоненты;
- Projections – проекции прикладных объектов;
- Solutions – прикладные решения;
- UsageAgreements – пользовательские соглашения;
- Scripts – прикладные сценарии;
- Functions – прикладные функции.

Команда выполняется в транзакции, поэтому, если возникли ошибки, изменения откатываются.

dbversion – получение версии схемы данных

Команда **dbversion** используется для получения версии схемы базы данных личного кабинета.

Синтаксис:

```
ess dbversion
```

В результате отображается информация по версии схемы базы данных личного кабинета.

normalize – нормализация прикладного решения

Команда **normalize** используется для нормализации конфигурационного XML-файла решения.

Синтаксис:

```
ess normalize <ИмяФайлаКонфигурацииРешения> [-q|--quiet]
```

где:

- **ИмяФайлаКонфигурацииРешения** – название конфигурационного XML-файла решения;
- **-q|--quiet** – флаг нормализации в неинтерактивном режиме. Если указан, то запрос подтверждения нормализации решения не появляется.

В результате нормализации:

- пустые необязательные узлы и атрибуты, а также комментарии удаляются;
- текстовые узлы заключаются в синтаксис CDATA;
- узлы приводятся к стилю camelCase.

Нормализация не проверяет XML-файлы, подключенные через узел **include**.

Изменения сохраняются в исходном конфигурационном XML-файле решения.

upgrade – модернизация прикладного решения

Команда **upgrade** используется для модернизации конфигурационного XML-файла прикладного решения на актуальную версию. Она:

- обновляет конфигурацию решения до версии утилиты;
- удаляет узлы, которые больше не поддерживаются;
- переименовывает узлы, если в новой версии у них изменились названия;
- меняет расположение узлов при необходимости.

В результате файл конфигурации решения, а также все файлы, подключенные к нему через узел **include**, приводятся к актуальному формату.

Изменения сохраняются в исходных конфигурационных XML-файлах решения.

Если в конфигурации отсутствует узел **dataSchemeVersion**, то версия принимается как **1.0**.

Синтаксис:

```
ess upgrade <ИмяФайлаКонфигурацииРешения> [-q|--quiet] [-i|--imitation]
```

где:

- **ИмяФайлаКонфигурацииРешения** – название конфигурационного XML-файла решения;
- **-q|--quiet** – флаг модернизации в неинтерактивном режиме. При указании запрос подтверждения для модернизации конфигурационного файла решения не появляется;
- **-i|--imitation** – флаг имитации модернизации. Используется, если необходимо ознакомиться с планируемыми изменениями без перезаписи конфигурационного файла. В этом случае все необходимые изменения записываются в консоль.

Прикладная конфигурация

Для управления прикладной конфигурацией решений утилита **ess** предоставляет команды:

list	Вывод списка всех прикладных объектов
export	Экспорт конфигурации прикладного объекта
import	Импорт конфигурации прикладного объекта
verify	Проверка корректности прикладной конфигурации

list – вывод списка всех прикладных объектов

Команда **list** используется для просмотра всех прикладных объектов в базе данных.

Синтаксис:

```
ess list <ТипОбъекта> [-s|--solution <ИмяРешения>]
```

где:

- **ТипОбъекта** – обязательный параметр. В качестве типа объекта могут выступать:
  - Facilities – услуги;

    - FacilityGroups – группы услуг;
    - Tiles – плитки быстрых действий;
    - Widgets – виджеты;
    - NotificationTemplates – шаблоны оповещений;
    - ObjectCards – карточки прикладных объектов;
    - Pages – прикладные страницы;
    - Components – сторонние компоненты;
    - Projections – проекции прикладных объектов;
    - UsageAgreements – пользовательские соглашения;
    - ValueConverters – прикладные конвертеры;
    - Scripts – прикладные сценарии;
    - Functions – прикладные функции.
- **ИмяРешения** – имя решения. Необязательный параметр, задается для ограничения выборки.

Команда отображает на экране все объекты указанного типа в формате таблицы со следующими столбцами:

- **Id** – идентификатор объекта;
- **Name** – имя объекта;
- **ParentId** – идентификатор родительского объекта;
- **Solution** – имя решения.

export – экспорт конфигурации прикладного объекта

Команда **export** используется для экспорта конфигурации одного прикладного объекта, например, если его необходимо поменять.

Синтаксис:

```
ess export <ТипОбъекта> [-i|--id <ИдентификаторОбъекта>] [-s|--solution
<ИмяРешения>] [-l|--language <Язык>] [-o|--output <ИмяФайлаКонфигурации>]
```

где:

- **ТипОбъекта** – в качестве типа объекта могут выступать следующие:
    - Facilities – услуги;
    - FacilityGroups – группы услуг;
    - Tiles – плитки быстрых действий;
    - Widgets – виджеты;
    - NotificationTemplates – шаблоны оповещений;
    - ObjectCards – карточки прикладных объектов;
    - Pages – прикладные страницы;
    - Components – сторонние компоненты;
    - Projections – проекции прикладных объектов;
    - Strings – строки локализации;

  - UsageAgreements – пользовательские соглашения;
  - ValueConverters – прикладные конвертеры;
  - Scripts – прикладные сценарии;
  - Functions – прикладные функции.
- **ИдентификаторОбъекта** – идентификатор объекта в базе данных. Если не был указан идентификатор объекта, то экспортируются все объекты указанного типа;
- **ИмяРешения** – имя решения;
- **Язык** – указывается язык выгрузки строк локализации;
- **ИмяФайлаКонфигурации** – имя файла. Конфигурация объекта экспортируется в XML-файл с указанным именем. Если имя файла не задано, то оно будет сформировано как <ТипОбъекта>.<ИдентификаторОбъекта>.xml.

ПРИМЕЧАНИЕ. При экспорте объекта экспортируются также все вложенные подобъекты. Например, при экспорте группы услуг экспортируются также все услуги, входящие в группу.

import – импорт конфигурации прикладного объекта

Команда **import** используется для импорта измененной конфигурации.

Синтаксис:

```
ess import <ИмяФайлаКонфигурации> [-p|--parent
<ИдентификаторРодительскогоОбъекта>] [-l|--language <Язык>]
```

где:

- **ИмяФайлаКонфигурации** –имя файла;
- **ИдентификаторРодительскогоОбъекта** – идентификатор родительского объекта из базы данных;
- **Язык** – указывается язык загрузки строк локализации.

Если в конфигурацию необходимо добавить новый объект, то его идентификатор задается значением **NEWID**.

ВАЖНО. Использовать команду import рекомендуется только в базах разработки. В продуктивной системе рекомендуется внести изменения в конфигурацию прикладного решения, а затем удалить и установить его целиком.

verify – проверка корректности прикладной конфигурации

Команда **verify** используется для проверки корректности XML-файла с прикладной конфигурацией решения.

Синтаксис:

```
ess verify <ИмяФайлаКонфигурации>
```

где **ИмяФайлаКонфигурации** –имя файла.

Команда проверяет корректность XML-файла с прикладной конфигурацией решения и выводит список ошибок.

Целевые системы

Для добавления, удаления и перенастройки точек подключения к целевым системам (внешним подключаемым системам, например Directum RX) утилита **ess** предоставляет следующие команды:

connect	Подключение целевой системы
disconnect	Отключение целевой системы
sys	Вывод списка точек подключения целевых систем

connect – подключение целевой системы

Команда **connect** используется для подключения целевой системы к личному кабинету.

Синтаксис:

```
ess connect "<ПутьДоФайлаКонфигурации>.json" [-p|--property
<Название>=<Значение>] [-q|--quiet] Пример: ess connect
```

где:

- **ИмяФайлаКонфигурации** – имя файла конфигурации;
- **-p|--property** – свойства конфигурации. Указываются в формате <Название>=<Значение>. В параметре **ServerVersion** указывается номер версии целевой системы. Пример вызова команды с заполнением свойств **UserIdentity** и **Endpoint**:

  ```
  ess connect "C:\HRPro. Платформа личного
  кабинета\Solutions\Ess\TargetSystems\EssBaseEndpoint.json" -p
  UserIdentity="DirectumRX" -p
  Configuration:AppServerConnection:Endpoint="https://drx.contoso.com/integrat
  ion/odata" -p Configuration:ServerVersion="4.9"
  ```

- **-q|--quiet** – флаг установки в неинтерактивном режиме. Если указан, то запрос активации решения не появляется.

Команда добавляет соответствующую запись в таблицу **TargetSystems** или обновляет существующую запись.

Пример файла конфигурации:

```
{
  "Name": "EssBase",
  "Title": "Directum RX Ess base integration service",
  "AdapterType": "Directum.SelfServiceOffice.Infrastructure.RxIntegrationClient.RxSystem,
Directum.SelfServiceOffice.Infrastructure.RxIntegrationClient, Version=1.0.0.0,
Culture=neutral, PublicKeyToken=427ba5252f628cb0",
  "UserIdentity": "",
  "Configuration": {
    "AppServerConnection": {
      "Endpoint": "",
    },
    "ServerVersion": ""
  }
}
```

где:

- **Name** – уникальное имя точки подключения целевой системы. Будет использоваться в XML-файле конфигурации;
- **Title** – заголовок точки подключения целевой системы;
- **AdapterType** – полное .NET имя типа адаптера. Пример для адаптера Directum RX:

  Directum.SelfServiceOffice.Infrastructure.RxIntegrationClient.RxSystem, Directum.SelfServiceOffice.Infrastructure.RxIntegrationClient, Version=1.0.0.0, Culture=neutral, PublicKeyToken=427ba5252f628cb0
- **UserIdentity** – используемое системой удостоверение пользователя, предоставляемое сервисом идентификации;
- **Configuration** – конфигурация адаптера.

ВАЖНО. После подключения новой системы перезапустите сервер приложения личного кабинета.

disconnect – отключение целевой системы

Команда **disconnect** используется для отключения целевой системы от личного кабинета.

Синтаксис:

```
ess disconnect <ИмяТочкиПодключенияСистемы>
```

где **ИмяТочкиПодключенияСистемы** – уникальное имя точки подключения целевой системы.

Команда удаляет соответствующую запись из таблицы **TargetSystems**.

sys – вывод списка точек подключения целевых систем

Команда **sys** используется для вывода списка всех точек подключения целевых систем.

Синтаксис:

```
ess sys
```

В результате отображается информация по всем точкам подключения целевых систем: идентификатор, имя, заголовок и используемое системой удостоверение пользователя.

Работа с файлами

Для работы с файлами утилита **ess** предоставляет следующие команды:

bundle – создание бандла из файлов.

Синтаксис:

```
ess bundle <ПутьКФайлам> <ИмяФайлаРезультата> <ТипФайлаРезультата>
```

где:

- **ПутьКФайлам** – в качестве пути может использоваться:
  - относительный путь. Пример: **.\icons**;
  - абсолютный путь. Пример: **D:\icons**;
  - маска. Пример: **D:\icons*.svg** или **D:\icons\icon-?.svg**.
- **ИмяФайлаРезультата** – имя файла готового бандла;
- **ТипФайлаРезультата** – тип файла результата. Возможные варианты: JSON – в этом случае бандл собирается из файлов с расширением SVG в JSON-файл, представляющий из себя словарь, где ключ – имя файла иконки без расширения, а значение – содержимое файла с иконкой.

Пример вызова команды с созданием бандла из иконок в папке icons:

```
ess bundle D:\icons\ icons.json json
```

Пример вызова команды с созданием бандла из одного файла icon.svg:

```
ess bundle D:\icon.svg icon.json json
```

Конфигурация решения

Корневым узлом конфигурации прикладного решения является **solution**. Он содержит обязательный атрибут **name**, в котором указывается имя прикладного решения.

Дочерними для **solution** являются узлы:

authorization	Содержит список прав доступа к прикладному объекту. Подробнее см. в разделе «Узел authorization»
title	Заголовок прикладного решения
description	Описание прикладного решения
[Обязательный] version	Определяет мажорную и минорную версию конфигурации. Например **1.2**
startPage	Наименование стартовой страницы. Список прикладных страниц перечислен в узле pages
dataSchemeVersion	Версия схемы базы данных. Указывается совместимая для решений версия схемы базы данных личного кабинета
facilityGroups	Содержит конфигурацию групп услуг в виде набора узлов facilityGroup
tiles	Содержит конфигурацию плиток быстрых действий в виде набора узлов tile
localizedStringValues	Содержит значения строк локализации в виде набора узлов localizedStringValue
usageAgreements	Содержит настройки пользовательских соглашений в виде набора узлов usageAgreement
notificationTemplates	Содержит шаблоны оповещений в виде набора узлов notificationTemplate
objectCards	Содержит конфигурацию карточек объектов в виде набора узлов objectCard
projections	Содержит конфигурацию проекций прикладных объектов в виде набора узлов projection
extensions	Содержит конфигурацию расширений для прикладных объектов в виде набора узлов extension
pages	Содержит конфигурацию прикладных страниц в виде набора узлов page
components	Содержит данные об используемых сторонних компонентах в виде набора узлов component
valueConverters	Содержит конфигурацию прикладных конвертеров в виде набора узлов valueConverter
scripts	Содержит конфигурацию прикладных сценариев в виде набора узлов script
roles	Содержит конфигурацию ролей в виде набора узлов role
widgets	Содержит список виджетов группы услуг в виде набора узлов widget
functions	Содержит список прикладных функций в виде набора узлов function

authorization – права доступа к объекту

Узел **authorization** предназначен для настройки прав доступа к прикладному объекту.

Структура узла:

```
<authorization>
  <allow roles="{Роли, которым разрешен доступ}"/>
  <deny roles="{Роли, которым запрещен доступ}"/>
</authorization>
```

allow – разрешающие права. В обязательном атрибуте **roles** указываются роли, которым разрешен доступ к объекту. Список ролей может быть пустым.

deny – запрещающие права. В обязательном атрибуте **roles** указываются роли, которым запрещен доступ к объекту. Список ролей может быть пустым.

Примеры настройки^

Вариант доступа	Настройка

Доступ разрешен для всех ролей	Узел **authorization** не добавляется или указывается в следующем виде:
	`<authorization>`
	`  <allow roles="*"/>`
	`</authorization>`

Доступ запрещен для всех ролей	Узел **authorization** указывается в следующем виде:
	`<authorization>`
	`  <deny roles="*"/>`
	`</authorization>`

Доступ разрешен, если есть одна из указанных ролей (OR)	Роли указываются одним из способов:
	• в одном узле **allow** через запятую. Пример:
	`<authorization>`
	`  <allow roles="Developer, Administrator"/>`
	`</authorization>`
	• в отдельных узлах **allow**. Пример:
	`<authorization>`
	`  <allow roles="Developer"/>`
	`  <allow roles="Administrator"/>`
	`</authorization>`

Доступ разрешен, если есть все указанные роли (AND)	Все роли указываются в одном узле **allow** через **+**. Пример:
	`<authorization>`
	`  <allow roles="Developer+Administrator"/>`
	`</authorization>`

Доступ разрешен для одной роли и запрещен для другой	Пример:
	`<authorization>`
	`  <allow roles="Developer"/>`
	`  <deny roles="Administrator"/>`
	`</authorization>`

Применение прав доступа

При определении итоговых прав доступа используются правила:

1. Если прикладное решение не активировано, доступ к его объектам запрещен.
2. Если для корневого объекта не заданы права, то по умолчанию для него добавляется узел **allow**, который разрешает доступ к объекту для всех пользователей:

```
<allow roles="*"/>
```

3. Дочерний объект наследует права родительского объекта в дополнение к своим:
   - все объекты наследуют права доступа, заданные в узле **solution**. Если права в этом узле не заданы, то доступ к объектам наследуется для всех пользователей;
   - услуги наследуют права доступа от узла **solution** и от групп услуг, в которые входят;
   - приоритет запрещающих прав выше, чем разрешающих.

   Примеры совместного использования разрешающих и запрещающих прав:

Вариант доступа	Настройка

Доступ разрешен только для указанной роли.
У других ролей нет доступа к объекту
	`<authorization>`
	`  <allow roles="Admin"/>`
	`</authorization>`

Доступ запрещен для указанной роли.
У других ролей нет доступа к объекту
	`<authorization>`
	`  <deny roles="Admin"/>`
	`</authorization>`

Доступ запрещен для указанной роли.
У других ролей есть доступ к объекту
	`<authorization>`
	`  <allow roles="*"/>`
	`  <deny roles="Admin"/>`
	`</authorization>`

facilityGroup – группа услуг

Узел **facilityGroup** предназначен для настройки группы услуг, а также всех услуг, которые в нее входят. Атрибуты узла:

- **id**. Идентификатор группы;
- [Обязательный] **name**. Имя группы.

Структура узла:

```
<facilityGroup id="{ИД группы}" name="Имя группы">
  <authorization>
    {Права доступа к группе}
  </authorization>
  <icon>{Иконка}</icon>
  <title>{Отображаемое имя}</title>
  <description>{Отображаемое описание}</description>
  <priority>{Приоритет}</priority>
  <isVisible>{Признак отображения}</isVisible>
  <collectAllNotifications>{Отображение всех уведомлений}</collectAllNotifications>
  <referenceUrl>{Адрес веб-справки по группе услуг}</referenceUrl>
  <views>
    {Представления}
  </views>
  <facilities>
    {Услуги}
  </facilities>
</facilityGroup>
```

authorization – настройки прав доступа к группе услуг. Подробнее см. раздел «authorization – права доступа к объекту».

icon – имя стандартной иконки для группы услуг, отображаемой в интерфейсе. Можно указать только стандартные иконки решения.

title – отображаемый заголовок группы. В качестве значения указывается текст или ссылка на строку локализации.

description – отображаемое описание группы. В качестве значения указывается текст или ссылка на строку локализации.

[Обязательный] **priority** – приоритет группы. Определяет порядок отображения группы на панели навигации: группа с приоритетом **0** отображается первой, **1** – второй и т.д.

[Обязательный] **isVisible** – признак отображения группы на панели навигации.

[Обязательный] **collectAllNotifications** – признак отображения уведомлений из других групп.

referenceUrl – адрес веб-справки по группе услуг. Можно указать полный путь или путь относительно адреса справки, указанного в настройках сайта личного кабинета. Подробнее см. в руководстве администратора HR Pro, раздел «Настройка платформы личного кабинета».

views – представления группы на различных устройствах.

facilities – услуги группы.

Пример настройки

Настройка для группы услуг «Документы», которая отображается в главном меню личного кабинета:

```
<facilityGroup id="9a661f41-ce53-4247-9f18-e22edc1967a3" name="Documents">
  <authorization>
    <allow roles="User" />
  </authorization>
  <icon>documents</icon>
  <title>@.FACILITYGROUPS_TITLE_DOCUMENTS</title>
  <description>Текст описания услуги</description>
  <priority>0</priority>
  <isVisible>true</isVisible>
  <collectAllNotifications>false</collectAllNotifications>
  <referenceUrl>https://webhelp.contoso.ru/facilityGroup.html</referenceUrl>
  <views />
  <facilities />
</facilityGroup>
```

ПРИМЕЧАНИЕ. В примере в группу не добавлены представления и услуги. Примеры заполнения этих узлов см. в соответствующих разделах.

view – представление группы услуг на устройствах

Узел **view** предназначен для настройки представления группы на различных устройствах.

Структура узла:

```
<views>
  <view name="{Имя представления}" device="{Тип представления}" container="{Тип контейнера}">
    <tileslots>
      <tileslot tileName="{Имя плитки быстрого действия}" size="{Размер плитки}"/>
    </tileslots>
    <bounds>
      <bound widgetName="{Имя виджета}" {Атрибуты для типа контейнера или виджета}/>
    </bounds>
  </view>
</views>
```

view – представление:

- **name**. Имя представления. Указывается для расширений в прикладном решении;
- [обязательный] **device**. Тип представления, предназначенный для определенных устройств. Возможные значения:
  - **Mobile** – мобильное представление;
  - **Web** – веб-представление;
- [обязательный] **container**. Тип контейнера. Возможные значения:
  - **StaticGrid** – фиксированная сетка с шириной 29 столбцов (координаты с 1 по 30). При использовании этого типа контейнера в дочерних узлах **bound** указываются параметры для размещения виджета или плиток быстрого действия в сетке (**left**, **top**, **right**, **bottom**);
  - **StackPanel** – расположение в столбец (стек). Виджеты или плитки быстрых действий отображаются в заданном порядке, один за другим. Если в контейнер этого типа добавлен виджет типа **embedded**, то для него в параметре **aspectRatio** можно указать соотношение сторон содержимого;
  - **FlexBox** – гибкое представление. Для контейнера такого типа можно определять направление расположения виджетов и их компоновку.

tileslots – информация о размещении плиток быстрых действий. Содержит узлы **tileslot** с атрибутами:

- [обязательный] **tileName**. Имя плитки;
- **size**. Размер плитки. Возможные значения: **Normal** – обычный, **Wide** – широкий;

[Обязательный] **bounds** – границы виджетов. Используется только для типов контейнеров **StaticGrid** и **StackPanel**. Содержит узлы **bound** с атрибутами:

- для всех типов контейнера:

  [обязательный] **widgetName**. Имя виджета;
- для типа контейнера **StaticGrid**:

  left. Положение левого края прямоугольника по горизонтали;

  top. Положение верхнего края прямоугольника по вертикали;

  right. Положение правого края прямоугольника по горизонтали;

  bottom. Положение нижнего края прямоугольника по вертикали;
- для типа контейнера **StackPanel** и типа виджета **embedded**:

  aspectRatio. Соотношение сторон содержимого виджета.

  Дополнительный дочерний узел **orientation** определяет порядок расположения элементов в контейнере **StackPanel**. Возможные значения:
  - **Vertical** – вертикальный;
  - **Horizontal** – горизонтальный.

Фиксированная сетка StaticGrid и расположение в столбец StackPanel

Пример настройки представлений для виджетов **contacts**, **department**, **personalData**, **embeddedVideo** и плиток быстрых действий **StopThePlanet** и **WorkitemTile**:

- в веб-представлении – по фиксированной сетке **StaticGrid**. Для виджетов указаны координаты их границ. Для плиток координаты в примере не задаются;

- в представлении для мобильных устройств – в столбец **StackPanel**.

```
<views>
  <view name="ExampleWeb" device="Web" container="StaticGrid">
    <tileslots>
      <tileslot tileName="StopThePlanet" size="Normal"/>
      <tileslot tileName="WorkitemTile" size="Wide"/>
    </tileslots>
    <bounds>
      <bound widgetName="contacts" left="1" top="10" right="15" bottom="18" />
      <bound widgetName="department" left="1" top="19" right="15" bottom="27" />
      <bound widgetName="personalData" left="16" top="1" right="30" bottom="23" />
      <bound widgetName="embeddedVideo" left="1" top="28" right="30" bottom="46" />
    </bounds>
  </view>
  <view name="ExampleMobile" device="Mobile" container="StackPanel">
    <orientation>Vertical</orientation>
    <tileslots>
      <tileslot tileName="StopThePlanet" size="Normal"/>
      <tileslot tileName="WorkitemTile" size="Normal"/>
    </tileslots>
    <bounds>
      <bound widgetName="contacts" />
      <bound widgetName="department" />
      <bound widgetName="personalData" />
      <bound widgetName="embeddedVideo" aspectRatio="1.778" />
    </bounds>
  </view>
</views>
```

Гибкое представление FlexBox

Для типа контейнера **FlexBox** используются дополнительные дочерние узлы:

[Обязательный] **direction** – направление расположения элементов в контейнере. Возможные значения: **row** – в строку, **column** – в столбец;

[Обязательный] **items** – описание элементов контейнера. Содержит дочерние узлы **item** с обязательным атрибутом **type** – тип элемента в контейнере. Возможные значения атрибута:

- **bound** – элемент с заданными границами виджета. В этом случае узел содержит дочерний узел **bound** с атрибутами:
  - [Обязательный] **widgetName**. Имя виджета;
  - **minWidth**. Минимальная ширина. Значение по умолчанию **auto** – ширина вычисляется автоматически в зависимости от содержимого виджета. Ширину можно указать в пикселях, например 100px, или процентах, например 50%;
  - **minHeight**. Минимальная высота. Значение по умолчанию **auto** – высота вычисляется автоматически в зависимости от содержимого виджета. Высоту можно указать в пикселях, например 100px, или процентах, например 50%.
- **flexbox** – вложенный контейнер типа **FlexBox**. Уровней вложенности может быть несколько.

ПРИМЕЧАНИЕ. На компьютере при разрешении экрана меньше 1280px виджеты в контейнере **FlexBox** отображаются в одном столбце, как в контейнере **StackPanel**. На смартфоне виджеты в контейнере **FlexBox** всегда отображаются в одном столбце.

Пример настройки

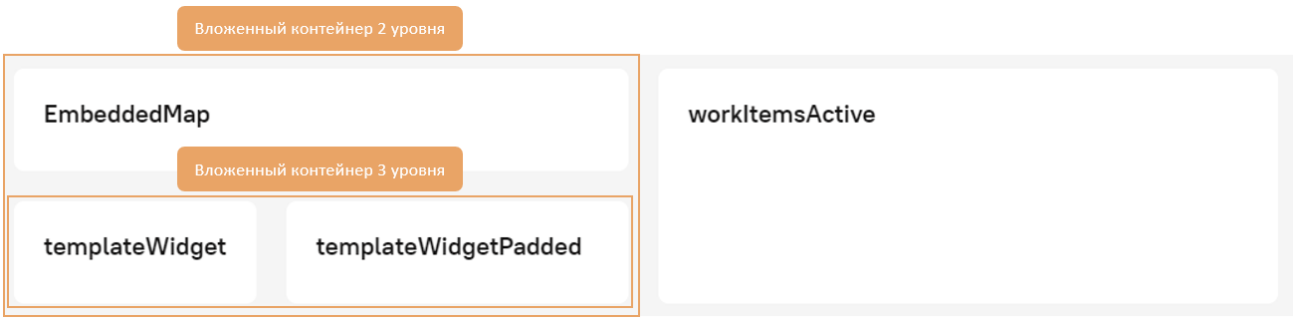
Виджеты **EmbeddedMap**, **templateWidget**, **templateWidgetPadded** и **workItemsActive** расположены в контейнере **FlexBox** в строку. Строка содержит:

- вложенный контейнер с гибким представлением, где в столбец располагаются элементы:
  - в первой строке столбца – виджет **EmbeddedMap** с минимальной шириной 400px и минимальной высотой 200px;
  - вторая строка столбца содержит контейнер с гибким представлением, где виджеты располагаются в строку. Строка содержит виджеты **templateWidget** и **templateWidgetPadded** с минимальной шириной 400px.
- виджет **workItemsActive**.

Конфигурация:

```
<views>
  <view name="ExampleFlex" device="Web" container="FlexBox">
    <direction>row</direction>
    <items>
      <item type="flexbox">
        <direction>column</direction>
        <items>
          <item type="bound">
            <bound widgetName="EmbeddedMap" minWidth="400px" minHeight="200px"/>
          </item>
          <item type="flexbox">
            <direction>row</direction>
            <items>
              <item type="bound">
                <bound widgetName="templateWidget" />
              </item>
              <item type="bound">
                <bound widgetName="templateWidgetPadded" minWidth="400px"/>
              </item>
            </items>
          </item>
        </items>
      </item>
      <item type="bound">
        <bound widgetName="workItemsActive" />
      </item>
    </items>
    <tileslots/>
  </view>
</views>
```

Представление в личном кабинете:



facility – услуга

Узел facility предназначен для настройки услуги. Атрибуты узла:

- **id**. Идентификатор услуги;
- [Обязательный] **name**. Уникальное имя услуги.

Структура узла:

```
<facility id="{ИД услуги}" name="{Имя услуги}">
  <authorization>
    {Права доступа к услуге}
  </authorization>
  <title>{Отображаемое имя}</title>
  <description>{Описание}</description>
  <facilityParameters>
    {Параметры услуги}
  </facilityParameters>
  <facilityFormItems>
    {Элементы формы услуги}
  </facilityFormItems>
  <facilityFlowScheme firstStepIndex="{Индекс первого шага}" lastStepIndex="{Индекс последнего
шага}">
    <flowSteps>
      {Шаги услуги}
    </flowSteps>
  </facilityFlowScheme>
  <openingEventHandler>
    <Обработчик события «Начало открытия формы услуги»>
  </openingEventHandler>
  <openedEventHandler>
    <Обработчик события «Окончание открытия формы услуги»>
  </openedEventHandler>
  <beforeValidationEventHandler>
    <Обработчик события «Начало валидации формы услуги»>
  </beforeValidationEventHandler>
  <afterValidationEventHandler>
    <Обработчик события «Окончание валидации формы услуги»>
  </afterValidationEventHandler>
  <beforeSubmitEventHandler>
    <Обработчик события «Начало отправки формы»>
  </beforeSubmitEventHandler>
  <afterSubmitEventHandler>
    <Обработчик события «Окончание отправки формы»>
  </afterSubmitEventHandler>
  <closingEventHandler>
    <Обработчик события «Начало закрытия формы услуги»>
  </closingEventHandler>
  <closedEventHandler>
    <Обработчик события «Окончание закрытия формы услуги»>
  </closedEventHandler>
  <contractForm>{Признак уменьшения высоты формы под ее содержимое}</contractForm>
  <wideForm>{Признак отображения расширенной формы}</wideForm>
  <successMessage>{Сообщение об успешном запуске услуги}</successMessage>
  <referenceUrl>{Адрес веб-справки по услуге}</referenceUrl>
</facility>
```

authorization – настройки прав доступа к услуге. Подробнее см. раздел «authorization – права доступа к объекту».

title – отображаемое имя услуги. В качестве значения указывается текст или ссылка на строку локализации.

description – отображаемое описание услуги. В качестве значения указывается текст или ссылка на строку локализации.

facilityParameters – параметры услуги. Задаются в дочерних узлах **facilityParameter**.

facilityFormItems – элементы формы услуги. Задаются в дочерних узлах **facilityFormItem**.

[Обязательный] **facilityFlowScheme** – схема услуги. Атрибуты узла:

- **firstStepIndex**. Индекс первого шага услуги;
- **lastStepIndex**. Последний шаг услуги.

Схема задается в дочернем узле **flowSteps**.

openingEventHandler – обработчик события «Начало открытия формы услуги».

openedEventHandler – обработчик события «Окончание открытия формы услуги».

beforeValidationEventHandler – обработчик события «Начало валидации формы услуги».

afterValidationEventHandler – обработчик события «Окончание валидации формы услуги».

beforeSubmitEventHandler – обработчик события «Начало отправки формы услуги».

afterSubmitEventHandler – обработчик события «Окончание отправки формы услуги».

closingEventHandler – обработчик события «Начало открытия формы услуги».

closedEventHandler – обработчик события «Окончание открытия формы услуги».

contractForm – признак уменьшения высоты формы под ее содержимое. Применяется только к формам услуг, в которых не используются шаги. Возможные значения: **true**, **false**.

wideForm – признак отображения расширенной формы. Возможные значения: **true**, **false**. Формы, в которых есть с таблицы, всегда отображаются с увеличенной шириной.

referenceUrl – адрес веб-справки по группе услуг. Можно указать полный путь или путь относительно адреса справки, указанного в настройках сайта личного кабинета. Подробнее см. в руководстве администратора HR Pro, раздел «Настройка платформы личного кабинета».

Пример настройки для услуги, с помощью которой оформляется отпуск без сохранения заработной платы:

```
<facility id="91353a35-a29e-4e48-b4a3-480820baf12d" name="GetDayOff">
  <authorization>
    <allow roles="User" />
  </authorization>
  <title>Оформить отгул</title>
  <description></description>
  <facilityParameters />
  <facilityFormItems />
  <facilityFlowScheme firstStepIndex="0" lastStepIndex="-1">
    <flowSteps />
  </facilityFlowScheme>
  <openingEventHandler><![CDATA[console.log("opening event"); resolve();]]
></openingEventHandler>
  <openedEventHandler><![CDATA[console.log("opened event");]]></openedEventHandler>
  <beforeValidationEventHandler><![CDATA[console.log("before validation event");]]
></beforeValidationEventHandler>
  <afterValidationEventHandler><![CDATA[console.log("after validation event");]]
></afterValidationEventHandler>
  <beforeSubmitEventHandler><![CDATA[console.log("before submit event");]]
></beforeSubmitEventHandler>
```

```
  <afterSubmitEventHandler><![CDATA[console.log("after submit event");]]
></afterSubmitEventHandler>
  <closingEventHandler><![CDATA[console.log("closing event"); resolve();]]
></closingEventHandler>
  <closedEventHandler><![CDATA[console.log("closed event");]]></closedEventHandler>
  <successMessage></successMessage>
  <referenceUrl>https://webhelp.contoso.ru/facility.html</referenceUrl>
</facility>
```

ПРИМЕЧАНИЕ. В примере для услуги не добавлены параметры, элементы формы, шаги и обработчики событий. Примеры заполнения этих узлов см. в соответствующих разделах.

flowSteps – схема услуги

Узел **flowSteps** предназначен для настройки схемы услуги. Может содержать дочерние узлы:

Узел	Описание
flowStep	Содержит информацию по шагу потока услуги
actionStep	Запускает выполнение клиентского действия
convertStep	Конвертирует значения параметра потока услуги
scriptStep	Выполняет прикладной сценарий
property	Указывает связь свойства шага с параметром услуги. Узел используется в шагах **flowStep**, **actionStep**, **convertStep**
decisionStep	Содержит информацию о шаге решения

flowStep – шаг услуги

Узел **flowStep** предназначен для настройки шага услуги. Атрибуты узла:

- [Обязательный] **index**. Индекс шага;
- **name**. Наименование;
- **title**. Описание;
- [Обязательный] **type**. Тип шага.

Структура узла:

```
<flowStep index="{Индекс шага}" name=""{Имя шага} title="{Отображаемое имя шага}" type="{Тип шага}">
  <property name="" binding="" type="">
    {Свойство шага}
  </property>
  ...
</flowStep>
```

property – свойства узла. Набор доступных свойств зависит от типа шага.

Пример настройки

```
<flowStep index="" name="" title="" type="">
  <property name="" binding="" type=””></property>
</flowStep>
```

Далее перечислены доступные типы шагов и их свойства:

ChangeFacilityParameterStep – изменение параметра услуги

ChangeWorkItemStateStep – изменение состояния рабочего элемента

DocumentGenerationStep – генерация документа по шаблону

PackFacilityParameterStep – упаковка параметра услуги

StartWorkProcessStep – запуск рабочего процесса в целевой системе

StopWorkProcessStep – прекращение рабочего процесса в целевой системе

WaitWorkItemStep – ожидание изменения состояния рабочего элемента

WaitWorkProcessStep – ожидание изменения состояния рабочего процесса

CreateAttachmentGroupStep – создание группы вложений

DebugDumpStep – логирование текущих значений параметров услуги

ImportDocumentGroupStep – импорт группы документов в систему

ImportDocumentStep – импорт документа в систему

NotificationStep – отправка оповещения

ExecuteFunctionStep – вызов прикладной функции

AddDocumentSignatureStep – добавление подписи к документу

DownloadDocumentStep – выгрузка документа в локальную папку

DownloadDocumentSignStep – выгрузка подписи документа

JoinDocumentsStep – соединение нескольких документов

У каждого типа есть свой обязательный набор свойств. Дополнительно для всех типов задаются обязательные свойства:

NextStepIndex – индекс следующего шага, на который перейдет в случае успешного выполнения потока услуги.

Actor – идентификатор действующего лица в целевой системе. Необязательное свойство шага, для шагов работы с целевой системой. По умолчанию используется текущий идентификатор действующего лица, который соответствует значению **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.ActorId)**. Значение действующего лица должно быть в списке действующих лиц пользователя, список соответствует значению **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.ActorIds)**.

ChangeFacilityParameterStep – изменение параметра услуги

В результате шага в параметр услуги, указанный в свойстве **ParameterName**, запишется значение, указанное в свойстве **ParameterValue**.

Свойства:

ParameterName – имя параметра услуги, значение которого будет изменено.

ParameterValue – новое значение параметра услуги.

Пример:

```
<flowSteps>
  <flowStep index="0" name="ChangeFacilityParameter" title="Изменение параметра услуги"
type="ChangeFacilityParameterStep">
    <property name="ParameterName">Category</property>
    <property name="ParameterValue">6</property>
    <property name="NextStepIndex">-1</property>
  </flowStep>
</flowSteps>
```

ChangeWorkItemStateStep – изменение состояния рабочего элемента

В результате шага в рабочий элемент процесса, идентификатор которого указан в свойстве **WorkItemId**, добавляются вложения, указанные в свойстве **Attachments**. Рабочий элемент процесса переводится в состояние, указанное в свойстве **State**, с причиной, указанной в свойстве **StateChangeReason**.

Свойства:

TargetSystem – имя целевой системы.

WorkItemName – имя рабочего элемента процесса, для Directum RX – задания.

WorkItemId – идентификатор рабочего элемента процесса в целевой системе.

State – состояние. Для выполнения задания значение всегда **Complete**.

StateChangeReason – причина изменения состояния: результат выполнения задания, который передается в свойство **Result**.

Attachments – массив объектов с информацией о вложениях. Тип **Attachment[]**.

Пример:

```
<flowSteps>
  <flowStep index="0" name="ChangeWorkItemState" title="Выполнить задание"
type="ChangeWorkItemStateStep">
    <property name="WorkItemName" binding="ItemName" />
    <property name="TargetSystem">DirectumRX</property>
    <property name="NextStepIndex">-1</property>
    <property name="WorkItemId" binding="ItemId" />
    <property name="State">Complete</property>
    <property name="StateChangeReason">Refuse</property>
    <property name="ActiveText" binding="Reason" />
  </flowStep>
</flowSteps>
```

DocumentGenerationStep – генерация документа по шаблону

В результате шага в параметр услуги, указанный с режимом связи **PropertyToParameter**, записывается информация о документе, загруженном в сервис хранения BLOB-объектов. Тип объекта, который сохраняется как значение в параметре, – **StoredDocument**.

Свойства:

TargetSystem – имя целевой системы.

DocumentId – идентификатор документа в целевой системе.

DocumentGroup – имя группы документа, для Directum RX – имя типа.

Необязательные свойства:

DocumentFormat – формат сгенерированного документа. Возможные значения: **doc97**, **docx**, **pdfa**. При варианте pdfa происходит конвертация документа. Если значение не указано, то формат берется от документа из целевой системы, используется свойство **DocumentId**.

DocumentFields – поля для заполнения в шаблоне. Тип **KeyValue[]**, где ключ – имя параметра услуги, которое соответствует имени поля в шаблоне документа.

DocumentName – новое имя для сгенерированного документа.

```
<flowSteps>
  <flowStep index="0" name="ApplicationGeneration" title="Сгенерировать заявление"
type="DocumentGenerationStep">
    <property name="NextStepIndex">1</property>
    <property name="TargetSystem">EssBase</property>
    <property name="DocumentId" binding="Document" type="Int64" />
    <property name="DocumentGroup">IElectronicDocuments</property>
    <property name="DocumentFormat">docx</property>
    <property name="DocumentFields" binding="Field_*" type="KeyValue[]" />
    <property name="GenerationValue" binding="StoredDocumentParam,Mode=PropertyToParameter"
type="StoredDocument"/>
    <property name="DocumentName" binding="Name" />
  </flowStep>
  <flowStep index="1" name="ApplicationGenerationPdfA" title="Сгенерировать заявление в
формате PdfA" type="DocumentGenerationStep">
    <property name="NextStepIndex">-1</property>
    <property name="TargetSystem">EssBase</property>
    <property name="DocumentId" binding="Document" type="Int64" />
    <property name="DocumentGroup">IElectronicDocuments</property>
    <property name="DocumentFormat">pdfa</property>
    <property name="DocumentFields" binding="Field_*" type="KeyValue[]" />
    <property name="GenerationValue" binding="StoredDocumentParam,Mode=PropertyToParameter"
type="StoredDocument"/>
    <property name="DocumentName" binding="Name" />
  </flowStep>
</flowSteps>
```

PackFacilityParameterStep – упаковка параметра услуги

В результате шага в параметр услуги, указанный с режимом связи **PropertyToParameter**, записывается значение параметра услуги, указанного в свойстве **UnpackedValue** и преобразованного с помощью метода, указанного в свойстве **Method**.

Свойства:

Method – метод упаковки параметра. Возможное значение – **JSON**. Если будет передано другое значение, то параметр приведется к строке.

UnpackedValue – значение до упаковки.

Пример:

```
<flowSteps>
  <flowStep index="0" title="Упаковать целое значение в JSON"
type="PackFacilityParameterStep">
    <property name="NextStepIndex">1</property>
```

```
    <property name="Method">JSON</property>
    <property name="UnpackedValue" binding="IntValue" type="Int32">12345</property>
    <property name="PackedValue" binding="JsonValue,Mode=PropertyToParameter"></property>
  </flowStep>
  <flowStep index="1" title="Упаковать массив дат в JSON" type="PackFacilityParameterStep">
    <property name="NextStepIndex">-1</property>
    <property name="Method">JSON</property>
    <property name="UnpackedValue" binding="DateTimeArray" type="DateTimeOffset[]"></property>
    <property name="PackedValue" binding="JsonValue,Mode=PropertyToParameter"></property>
  </flowStep>
</flowSteps>
```

StartWorkProcessStep – запуск рабочего процесса в целевой системе

В результате шага идентификатор созданной и стартованной задачи записывается в параметр услуги, указанный с режимом связи **PropertyToParameter**.

Свойства:

ProcessName – имя рабочего процесса, для Directum RX – задачи.

TargetSystem – имя целевой системы.

Attachments – массив объектов с информацией о вложениях. Тип **Attachment[]**.

Пример:

```
<flowSteps>
  <flowStep index="0" name="StartVacationShiftTask" title="Запустить задачу на перенос
отпуска" type="StartWorkProcessStep">
    <property name="ProcessName">ISimpleTasks</property>
    <property name="TargetSystem">EssBase</property>
    <property name="NextStepIndex">1</property>
    <property name="Author">{'Id':$(CurrentUser.DirectumRX.ActorId)}</property>
    <property name="Subject">Тема задачи</property>
    <property name="ActiveText">Текст задачи</property>
    <property name="RouteSteps">{'Id':$(CurrentUser.DirectumRX.ActorId)}</property>
    <property name="Id" binding="ProcessId,Mode=PropertyToParameter" type="Int64" />
  </flowStep>
</flowSteps>
```

StopWorkProcessStep – прекращение рабочего процесса в целевой системе

В результате шага рабочий процесс, идентификатор которого указан в свойстве **ProcessId**, прекращается.

Свойства:

ProcessName – имя рабочего процесса, для Directum RX – задачи.

TargetSystem – имя целевой системы.

ProcessId – идентификатор рабочего процесса в целевой системе.

Пример:

```
<flowSteps>
```

```
  <flowStep index="1" name="StopWorkProcess" title="Прекращение задачи"
type="StopWorkProcessStep">
    <property name="ProcessName">ISimpleTasks</property>
    <property name="TargetSystem">EssBase</property>
    <property name="NextStepIndex">-1</property>
    <property name="ProcessId" binding="VacationId"/>
  </flowStep>
</flowSteps>
```

WaitWorkItemStep – ожидание изменения состояния рабочего элемента

Шаг с указанной периодичностью проверяет значение свойства рабочего элемента процесса. Пока значение свойства равно текущему, шаг продолжает вызываться.

Свойства:

TargetSystem – имя целевой системы.

NextExecuteTime – время до последующего выполнения в минутах.

CheckingProcessProperty – наименование свойства, которое проверяется у рабочего элемента процесса.

WaitWhileValue – исходное значение свойства.

WorkItemName – имя рабочего элемента процесса, для Directum RX – задания.

WorkItemId – идентификатор рабочего элемента процесса в целевой системе.

WaitWorkProcessStep – ожидание изменения состояния рабочего процесса

Шаг с указанной периодичностью проверяет значение свойства рабочего процесса. Пока значение свойства равно текущему, шаг продолжает вызываться.

Свойства:

ProcessName – имя рабочего процесса, для Directum RX – задачи.

TargetSystem – имя целевой системы.

NextExecuteTime – время последующего выполнения в минутах.

CheckingProcessProperty – наименование свойства, которое проверяется у рабочего процесса.

WaitWhileValue – исходное значение свойства.

ProcessId – идентификатор рабочего процесса в целевой системе.

Пример:

```
<flowSteps>
  <flowStep index="0" name="WaitWorkItem" title="Ожидание смены состояния задачи"
type="WaitWorkProcessStep">
    <property name="ProcessName">ISimpleTasks</property>
    <property name="TargetSystem">EssBase</property>
    <property name="NextStepIndex">-1</property>
```

```
    <property name="ProcessId" binding="Id"/>
    <property name="CheckingProcessProperty">Status</property>
    <property name="WaitWhileValue">InProcess</property>
    <property name="NextExecuteTime">2</property>
    <property name="Title" binding="ProcessSubject,Mode=PropertyToParameter" />
    <property name="Status" binding="ProcessStatus,Mode=PropertyToParameter" />
  </flowStep>
</flowSteps>
```

CreateAttachmentGroupStep – создание группы вложений

В результате шага в параметр услуги, указанный с режимом связи **PropertyToParameter**, записывается объект типа **Attachment[]** с информацией о группе вложений. Имя берется из свойства **GroupName**, вложения – из свойства **Documents**.

Свойства:

GroupName – наименование группы вложений.

Documents – массив ссылок на документы в целевой системе. Тип **DocumentLink[]**. Пример задания свойства со ссылкой на один документ в Directum RX: **[{"Id": 123, "Title": "Документ-вложение"}]**.

Пример:

```
<flowSteps>
  <flowStep index="0" title="Формирует группу вложений" type="CreateAttachmentGroupStep">
    <property name="NextStepIndex">-1</property>
    <property name="GroupName">TestAttachments</property>
    <property name="Documents" binding="AttachingDocuments" type="DocumentLink[]" />
    <property name="Result" binding="Attachments,Mode=PropertyToParameter"></property>
  </flowStep>
</flowSteps>
```

DebugDumpStep – логирование текущих значений параметров услуги

В результате в основной лог-файл сервера приложения записывается строка с информацией по указанным свойствам вида <Имя свойства шага>:<Значение свойства шага>. Подробнее о лог-файлах см. в руководстве администратора HR Pro, раздел «Логирование в HR Pro» (Linux, Windows).

Пример:

```
<flowSteps>
  <flowStep index="0" title="Залогировать значения параметров услуги" type="DebugDumpStep">
    <property name="NextStepIndex">-1</property>
    <property name="InitiatorId" binding="Initiator" />
  </flowStep>
</flowSteps>
```

ImportDocumentGroupStep – импорт группы документов в систему

Шаг создает документы в целевой системе. Карточка документа заполняется из свойств шага, тело документа загружается из хранилища. При указании **Signature** и **SignatureType** на документ добавляется внешняя подпись.

В результате шага в параметр услуги, указанный с режимом связи **PropertyToParameter**, записывается объект типа **DocumentLink[]** со ссылками на документы в целевой системе.

Свойства:

TargetSystem – имя целевой системы.

ImportingDocuments – массив объектов с информацией о документе из хранилища. Тип **StoredDocument[]**.

DocumentGroup – имя группы документа, для Directum RX – имя типа.

Пример:

```
<flowSteps>
  <flowStep index="1" name="ImportDocumentsRX" title="Импортировать документы в RX"
type="ImportDocumentGroupStep">
    <property name="NextStepIndex">-1</property>
    <property name="TargetSystem">EssBase</property>
    <property name="ImportingDocuments" binding="StoredDocument*" type="StoredDocument[]" />
    <property name="DocumentGroup">ISimpleDocuments</property>
    <property name="Author">{'Id':$(CurrentUser.DirectumRX.ActorId)}</property>
    <property name="ImportedDocuments"
binding="ImportedDocumentLinks,Mode=PropertyToParameter"/>
  </flowStep>
</flowSteps>
```

ImportDocumentStep – импорт документа в систему

Шаг создает документ в целевой системе. Карточка документа заполняется из свойств шага, тело документа загружается из хранилища. При указании **Signature** и **SignatureType** на документ добавляется внешняя подпись. При указании **DocumentId** новый документ не создается, а тело импортируется в новую версию документа с идентификатором **DocumentId**. Если документ с идентификатором **DocumentId** не существует или на него недостаточно прав, то свойство **DocumentId** игнорируется и создается новый документ. Для создания новой версии документа необходимы права на изменение или полные права на него.

В результате шага в параметр услуги, указанный с режимом связи **PropertyToParameter**, записывается объект типа **DocumentLink** со ссылкой на документ в целевой системе.

Свойства:

TargetSystem – имя целевой системы.

ImportingDocument – объект с информацией о документе из хранилища. Тип **StoredDocument**.

DocumentGroup – имя группы документа, для Directum RX – имя типа.

Пример:

```
<flowSteps>
  <flowStep index="1" name="ImportDocumentRX" title="Импортировать документ в RX"
type="ImportDocumentStep">
    <property name="NextStepIndex">-1</property>
    <property name="TargetSystem">EssBase</property>
    <property name="ImportingDocument" binding="StoredDocumentParam" type="StoredDocument" />
    <property name="DocumentGroup">ISimpleDocuments</property>
    <property name="Author">{'Id':$(CurrentUser.DirectumRX.ActorId)}</property>
    <property name="Name" binding="DocumentName" />
```

```
    <property name="ImportedDocument"
binding="ImportedDocumentLink,Mode=PropertyToParameter"/>
    <property name="DocumentId" binding="DocIdForNewVersion" type="Int64" />
  </flowStep>
</flowSteps>
```

NotificationStep – отправка оповещения

Шаг отправляет оповещение через сервис сообщений.

Свойства:

Receiver – получатель. Укажите один из вариантов:

- идентификатор сотрудника в Directum RX;
- макрос **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.EmployeeId)**.

Title – заголовок.

Content – содержимое.

IsImportant – признак важности.

ExpirationTime – время истечения срока годности.

Actions – вызываемые действия. Тип **Action[]**.

Properties – свойства. Задаются в виде пар ключ-значение.

Attachments – массив объектов с информацией о вложениях. Тип **Attachment[]**.

Пример:

```
<flowSteps>
  <flowStep index="0" title="Сформировать уведомление" type="NotificationStep">
    <property name="NextStepIndex">-1</property>
    <property name="Receiver">$(CurrentUser.DirectumRX.ActorId)</property>
    <property name="Title" binding="DateBegin,DateEnd,Format=Ваш отпуск с {0} по {1}
согласован"></property>
    <property name="Content" binding="UserContent,Format={0}"></property>
    <property name="IsImportant">true</property>
    <property name="FacilityGroupId">74F0BBE7-9A6A-4729-8BADC92BCDA69675</property>
    <property name="Actions"
type="Action[]">[{"name":"RUN_FACILITY","title":"Выполнить","parameters":{"id":"690C445C-70E9-
4069-8BA0-5A6266E8FFB4","TaskId":"{Id}","dialogTitle":"{Name}"}}]</property>
    <property name="Properties">{"PROP1":"Value1","PROP2":"Value2"}</property>
  </flowStep>
  <flowStep index="1" title="Сформировать уведомление с вложением" type="NotificationStep">
    <property name="NextStepIndex">-1</property>
    <property name="Receiver">$(CurrentUser.DirectumRX.ActorId)</property>
    <property name="Title" binding="DateBegin,DateEnd,Format=Ваш отпуск с {0} по {1}
согласован"></property>
    <property name="Content" binding="UserContent,Format={0}"></property>
    <property name="IsImportant">true</property>
    <property name="FacilityGroupId">74F0BBE7-9A6A-4729-8BADC92BCDA69675</property>
    <property name="Actions"
type="Action[]">[{"name":"RUN_FACILITY","title":"Выполнить","parameters":{"id":"690C445C-70E9-
4069-8BA0-5A6266E8FFB4","TaskId":"{Id}","dialogTitle":"{Name}"}}]</property>
    <property name="Properties">{"PROP1":"Value1","PROP2":"Value2"}</property>
    <property name="Attachments"
type="Attachment[]">[{"title":"Вложение1","url":"EssBase:/ElectronicDocument/97"},
{"title":"Вложение2","system":"DirectumRX","name":"ElectronicDocument","id":"100"}]</property>
  </flowStep>
```

```
</flowSteps>
```

ExecuteFunctionStep – вызов прикладной функции

В результате шага в параметр услуги, указанный с режимом связи **PropertyToParameter**, записывается результат выполнения прикладной функции, имя которой указано в параметре **FunctionName**.

Свойства:

TargetSystem – имя целевой системы.

FunctionName – имя прикладной функции.

ExecutionMethod – метод вызова функции. Возможные значения: **GET**, **POST**. Если свойство не задано, то метод вызова будет **GET**.

Все свойства шага, имеющие привязку **forward** (**Mode=ParameterToProperty**), передаются как параметры для вызываемой прикладной функции.

Пример:

```
<flowSteps>
  <flowStep index="0" name="ExecuteFunction" title="Запуск прикладной функции
GetEmployeeByUniqueId" type="ExecuteFunctionStep">
    <property name="NextStepIndex">-1</property>
    <property name="TargetSystem">EssBase</property>
    <property name="FunctionName">EssPlatform.GetEmployeeByUniqueId</property>
    <property name="ExecutionMethod">GET</property>
    <property name="uniqueId">Local@a29793b9-955a-4111-a5c4-3afd4b8d8558</property>
    <property name="Result" binding="FunctionResult,Mode=PropertyToParameter"/>
  </flowStep>
  <flowStep index="1" name="ExecuteFunction" title="Запуск прикладной функции
GetEmployeePosition" type="ExecuteFunctionStep">
    <property name="NextStepIndex">-1</property>
    <property name="TargetSystem">EssBase48</property>
    <property name="FunctionName">EssPlatform.GetEmployeePosition</property>
    <property name="employeeId" type="Int32">70</property>
    <property name="declensionCase">Genitive</property>
    <property name="Result" binding="FunctionResult,Mode=PropertyToParameter"/>
  </flowStep>
</flowSteps>
```

AddDocumentSignatureStep – добавление подписи к документу

В результате в целевой системе к документу добавляется подпись указанного типа.

Свойства:

TargetSystem – имя целевой системы.

DocumentGroup – имя группы документа, для Directum RX – имя типа.

DocumentId – идентификатор документа в целевой системе.

SignatureType – тип подписи.

Signature – подпись документа в формате **Base64**.

Пример:

```
<flowSteps>
  <flowStep index="0" title="Добавляет подпись к документу" type="AddDocumentSignatureStep">
    <property name="NextStepIndex">-1</property>
    <property name="TargetSystem">EssBase</property>
    <property name="DocumentId" binding="DocId" type="Int32" />
    <property name="DocumentGroup">ISimpleDocuments</property>
    <property name="SignatureType">Approval</property>
    <property name="Signature" binding="DocSignature" type="String" />
  </flowStep>
</flowSteps>
```

DownloadDocumentStep – выгрузка документа в локальную папку

Шаг выгружает документ из целевой системы в локальную папку на сервере.

В результате шага в параметр услуги, указанный с режимом связи **PropertyToParameter**, записывается объект типа **String** с путем к телу документа, выгруженного из целевой системы.

Свойства:

TargetSystem – имя целевой системы.

DocumentGroup – имя группы документа, для Directum RX – имя типа.

DocumentId – идентификатор документа в целевой системе.

Пример:

```
<flowSteps>
  <flowStep index="0" title="Выгружает документ" type="DownloadDocumentStep">
    <property name="NextStepIndex">-1</property>
    <property name="TargetSystem">EssBase</property>
    <property name="DocumentId" binding="DocId" type="Int32" />
    <property name="DocumentGroup">ISimpleDocuments</property>
    <property name="Result" binding="DocFileName,Mode=PropertyToParameter" />
  </flowStep>
</flowSteps>
```

DownloadDocumentSignStep – выгрузка подписи документа

Шаг выгружает подписи документа из целевой системы.

В результате в параметр услуги, указанный с режимом связи **PropertyToParameter**, записывается объект типа **String** с подписью документа, выгруженной из целевой системы.

Свойства:

TargetSystem – имя целевой системы.

DocumentGroup – имя группы документа, для Directum RX – имя типа.

DocumentId – идентификатор документа в целевой системе.

JoinDocumentsStep – соединение нескольких документов

В результате шага в параметр услуги, указанный с режимом связи **PropertyToParameter**, записывается объект типа **StoredDocument** с информацией об итоговом документе из сервиса хранения BLOB-объектов. Имя определяется свойством **DocumentName**, формат – свойством **TargetFormat**. Документы передаются из свойства **JoiningDocuments**.

Ограничения:

- все исходные документы должны иметь одинаковое расширение;
- поддерживаемые расширения исходных документов: PDF, PNG, JPG, JPEG, TIFF, TIF, BMP.

Свойства:

DocumentName – имя итогового документа.

TargetFormat – целевой формат документа. Возможные значения: **pdf**, **tiff**.

JoiningDocuments – массив объектов с информацией о документе из хранилища. Тип **StoredDocument[]**.

Пример:

```
<flowSteps>
  <flowStep index="0" name="JoinDocuments" title="Склеивание документов"
type="JoinDocumentsStep">
    <property name="NextStepIndex">-1</property>
    <property name="TargetSystem">EssBase</property>
    <property name="DocumentName" binding="Name" />
    <property name="TargetFormat">pdf</property>
    <property name="JoiningDocuments" binding="Documents" type="StoredDocument[]" />
    <property name="GenerationValue" binding="StoredDocumentParam,Mode=PropertyToParameter"
type="StoredDocument"/>
  </flowStep>
</flowSteps>
```

actionStep – выполнение клиентского действия

Шаг, запускающий выполнение клиентского действия. Особенностью данного шага является то, что при его выполнении на стороне сервера выполнение потока услуги приостанавливается в состоянии **Awaiting**. В этот момент контекст выполнения возвращается клиентскому приложению (вызывающей стороне), где требуется выполнить указанное прикладное действие.

Свойство **ActionName** – имя прикладного сценария, который необходимо выполнить на клиенте.

Прикладной сценарий указывается в формате **<Имя решения>.<Имя сценария>**.

Сценарий возвращает промис. Подробнее см. на сайте developer.mozilla.org, статья Promise. При успешном выполнении промиса выполняются шаги движка услуг. При отмене промиса отображается ошибка. Чтобы задать свой текст ошибки, при отмене промиса необходимо передать объект со свойством **message**.

Пример использования прикладного сценария с именем **createSignature** из прикладного решения **Applications**:

```
<property name="ActionName">Applications.createSignature</property>
```

Все свойства шага, имеющие привязку **forward** (Mode=ParameterToProperty), передаются как параметры для вызываемого действия.

Свойство шага, имеющее привязку **backward** (Mode=PropertyToParameter), используется для сохранения некоторого значения, которое может вернуть прикладное действие.

Пример настройки клиентского шага для получения подписи на документ

Получение подписи на документ происходит через прикладной сценарий с именем **createSignature** из прикладного решения **Applications**. Шаг для документа из сервиса хранилищ (параметр **StoredDocumentParam**) формирует подпись и в параметре **StoredDocumentParam** заполняет свойства **Signature** (сформированная подпись) и **SignatureType** (тип подписи из свойства шага):

```
<actionStep index="0" name="CreateSignatureAction" title="Добавление подписи к документу из сервиса хранилищ">
  <property name="NextStepIndex">1</property>
  <property name="ActionName">Applications.createSignature</property>
  <property name="SigningDocument" binding="StoredDocumentParam" type="StoredDocument" />
  <property name="SignatureType">Approval</property>
  <property name="Result" binding="StoredDocumentParam,Mode=PropertyToParameter" />
</actionStep>
```

Функция сценария **cancelationCallback**, вызывается при его отмене действия для снятия блокировки формы услуги. В этом случае, в сценарий **Applications.createSignature** поступает параметр в виде объекта:

```
{
  SigningDocument,
  SignatureType,
  cancelationCallback,
}
```

Пример сценария, оформленного отдельным файлом:

```
class createSignature {
  execute(params) {
    const document = JSON.parse(params.SigningDocument);
    const serviceUrl = '/api/functions/GetEmployeeCertificates';
    return new Promise((resolve, reject) => {
      ESS.network.sendRequest(serviceUrl)
        .then(certs => ESS.signing.createSignature({
          certificates: certs,
          document: {
            binaryDataId: document.BinaryDataId,
            name: document.Name,
          },
          cancelationCallback: params.cancelationCallback,
        }))
        .then(signature => {
          resolve(JSON.stringify({
            ...document,
            Signature: signature,
            SignatureType: params.SignatureType,
          }));
```

```
        })
        .catch(reject);
    });
  }
}
```

convertStep – конвертация значения параметра

Шаг конвертирования значения параметра потока услуги. Атрибуты:

- [Обязательный] **convert** – вариант конвертации значения параметра. Возможные значения:

  ToInt – приведение к типу **Int64**;

  ToBoolean – приведение к типу **Boolean**;

  ToBase64 – приведение к строке в формате **Base64**;

  ToChar – приведение к типу **Char**;

  ToDateTime – приведение к типу **DateTime**;

  ToSingle – приведение к типу **Single**;

  ToNominative – склонение в именительный падеж;

  ToGenitive – склонение в родительный падеж;

  ToDative – склонение в дательный падеж;

  ToAccusative – склонение в винительный падеж;

  ToInstrumental – склонение в творительный падеж;

  ToPrepositional – склонение в предложный падеж;

- **defaultValue** – значение по умолчанию. Свойства:

  Value – конвертируемое значение;

  Result – результат конвертации или значение по умолчанию, если конвертируемое значение было пустым.

В результате в параметр услуги, указанный с режимом связи **PropertyToParameter**, запишется сконвертированное значение, указанное в свойстве **Value**, согласно варианту конвертации, указанному в атрибуте **convert**. Если свойство **Value** не было заполнено, в параметр запишется значение по умолчанию, указанное в атрибуте **defaultValue**.

Пример конвертации

Конвертация значения параметра услуги **Field_ФИО сотрудника (р.п.)** типа **PersonName** в родительный падеж. Результат конвертации записывается в этот же параметр.

```
<convertStep index="0" convert="ToGenitive" defaultValue="">
  <property name="NextStepIndex">1</property>
  <property name="Value " binding="Field_ФИО сотрудника (р.п.)"  type="PersonName" />
  <property name="Result" binding="Field_ФИО сотрудника (р.п.),Mode=PropertyToParameter" />
</convertStep>
```

Помимо обязательных свойств в шаги могут передаваться дополнительные свойства.

scriptStep – выполнение прикладного сценария

Шаг выполнения прикладного сценария. Обязательный атрибут **script** – имя выполняемого прикладного сценария.

В качестве параметров в функцию сценария передаются все дополнительные свойства, указанные в конфигурации шага с режимом связи **ParameterToProperty**. Параметры передаются в том порядке, в котором перечислены в конфигурации. В результате выполнения сценария в параметр услуги, указанный с режимом связи **PropertyToParameter**, записывается результат, возвращаемый функцией сценария. Пример:

```
<scriptStep index="0" script="SumIt">
  <property name="NextStepIndex">-1</property>
  <property name="value1" binding="V1" type="Double"/>
  <property name="value2" binding="V2" type="Double"/>
  <property name="value3" binding="V3" type="Double"/>
  <property name="result" binding="Output,Mode=PropertyToParameter" />
</scriptStep>
```

При выполнении этого шага будет рассчитано среднее значение по значениям параметров **V1**, **V2**, **V3**, а результат сохранен в параметре **Output**.

property – свойство услуги

Используется в шагах **flowStep**, **actionStep**, **convertStep**, **scriptStep**.

Структура задания свойств:

```
<property name="GenerationValue" binding="BinaryDataId,Mode=PropertyToParameter"
type="String"/>
```

Атрибуты свойства:

- **name** – наименование свойства шага;
- **binding** – описание связи свойства шага с параметром услуги. Указывается связь с параметром/параметрами услуги и режим связи;
- **type** – тип параметра.

Возможные варианты связи свойства шага с параметром услуги

Значение свойства шага заполняется	Пример
Значением параметра услуги	`<property name="Content" binding="UserContent" />`
Значением параметра услуги по указанному шаблону	`<property name="Content" binding="UserContent,Format=Текст параметра {0}" />`
Значениями из нескольких параметров	`<property name="Attachments" binding="AttachGroup1,AttachGroup2" type="Attachment[]" />`
Значениями из нескольких параметров по указанному шаблону. Поддерживается переменная **#TODAY** – текущая дата	`<property name="Title" binding="DateBegin,DateEnd,Format=Ваш отпуск с {0} по {1} согласован" />` Пример с переменной **#TODAY**:

Значение свойства шага заполняется	Пример
	`<property name="Title" binding="DocumentName,#TODAY,Format= {0} от {1}" />`
Значениями из нескольких параметров по маске	`<property name="ImportingDocuments" binding="StoredDocument\*" type="StoredDocument[]" />`
Значениями из параметров по указанному шаблону Имена параметров задаются в виде: {{#ИмяПараметра}}	`<property name="Actions" type="Action[]">[{"name": "StartProcess", "parameters": { "system": "{{#System}}" , "process": "{{#ProcessName}}"}}, {"name": "DoSomething", "parameters": { "P3": "V3" , "P4": "V4"}}]</property>`
Значением свойства параметра услуги Свойство параметра услуги задается в виде: ИмяПараметра.ИмяСвойства	`<property name="documentId" type="Int32" binding="ImportedDocumentLink.Id" />`

Возможные значения режима связи

ParameterToProperty – значение по умолчанию, копируется в свойства из параметра услуги;

PropertyToParameter – значение из свойства шага копируется в параметр услуги;

BothWay – в обе стороны.

Возможные значения типа параметра

Datetime – дата и время. В качестве значения может быть задан макрос:

- **#NOW** или **$(Environment.Now)**;
- **#TODAY** или **$(Environment.Today)**.

Double – число с плавающей точкой.

Int32 – целое 32 битное число. В качестве значения может быть задан макрос:

- **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.EmployeeId)** – идентификатор сотрудника в целевой системе;
- **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.PersonId)** – идентификатор персоны в целевой системе;
- **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.ActorId)** – идентификатор действующего лица в целевой системе.

Int – целое 64 битное число. В качестве значения может быть задан макрос аналогичный **Int32**.

String – строка (по умолчанию). В качестве значения может быть задан макрос:

- **#CURRENT_USER_FULLNAME** или **$(CurrentUser.FullName)** – полное имя пользователя;
- **#CURRENT_USER_SHORTNAME** или **$(CurrentUser.ShortName)** – краткое имя пользователя;
- **#CURRENT_USER_NAME** или **$(CurrentUser.Name)** – имя пользователя;
- **#CURRENT_FACILITY_ID** – идентификатор текущей услуги;

- **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.EmployeeId)** – идентификатор сотрудника в целевой системе;
- **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.EmployeeIds)** – идентификаторы сотрудников в целевой системе, перечисляются через запятую;
- **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.ActorIds)** – идентификаторы действующих лиц в целевой системе, перечисляются через запятую;
- **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.PersonId)** – идентификатор персоны в целевой системе;
- **$(CurrentUser.<Имя удостоверения пользователя в целевой системе>.ActorId)** – идентификатор действующего лица в целевой системе.

String[] – массив строк.

DatetimeInterval – интервал дат.

Double[] – массив чисел с плавающей точкой.

Int32[] – массив целых 32 битных чисел.

Int[] – массив целых 64 битных чисел.

DatetimeOffSet[] – массив дат.

Action – прикладное действие. Свойства:

- **Name** – имя действия;
- **Title** – заголовок;
- **Parameters** – параметры. Параметры задаются в виде пар ключ–значение.

Action[] – массив прикладных действий.

StoredDocument – документ в хранилище. Свойства:

- **Name** – наименование документа;
- **BinaryDataId** – идентификатор документа в хранилище;
- **Signature** – подпись документа;
- **SignatureType** – тип подписи документа.

StoredDocument[] – массив документов в хранилище.

DocumentLink – ссылка на документ целевой системы. Свойства:

- **Id** – идентификатор документа;
- **Title** – заголовок документа.

DocumentLink[] – массив ссылок на документы целевой системы.

Attachment – вложение. Свойства:

- **Id** – идентификатор;
- **Type** – тип;
- **Name** – имя;
- **Group** – группа;
- **Title** – заголовок;
- **Modified** – дата изменения;

- **Extension** – расширение (только для документов);
- **Size** – размер в байтах (только для документов).

Attachment[] – массив вложений.

KeyValue[] – словарь (ключ – имя параметра, значение – значение параметра).

PersonName – имя персоны. Свойства:

- **FullName** – полное имя;
- **ShortName** – краткое имя;
- **GivenName** – имя собственное;
- **Surname** – фамилия;
- **Patronym** – отчество.

Boolean – логическое значение.

Примеры задания значения параметра с типом массива

```
<property name="StringListValue" type="String[]">Test String1,Test String2,Test
String3</property>
<property name="StringListValue" type="String[]">["Test String1,TestTest", "Test String2",
"Test String3"]</property>
```

Пример конфигурации услуги планирования отпуска на год

Услуга содержит шаг изменения состояния рабочего элемента. Внутри шага идет получение задания с типом **IVacationEmployeeSchedulingAssignments** из целевой системы Directum RX по идентификатору. Идентификатор задания (свойство **WorkItemId**) получается из параметра услуги **ItemId**. В задании заполняются свойства **VacationKindIds**, **VacationBeginDates**, **VacationEndDates**, **VacationDuration** значениями из соответствующих параметров услуги (имена указаны в атрибуте **binding**). После заполнения свойств задание выполняется. Для выполнения свойство **State** всегда должно иметь значение **Complete**.

```
<facilityFlowScheme firstStepIndex="0" lastStepIndex="-1">
  <flowSteps>
    <flowStep index="0" name="ChangeWorkItemState" title="Выполнить задание"
type="ChangeWorkItemStateStep">
      <property name="WorkItemName">IVacationEmployeeSchedulingAssignments</property>
      <property name="TargetSystem">DirectumRX</property>
      <property name="NextStepIndex">-1</property>
      <property name="WorkItemId" binding="ItemId" />
      <property name="VacationKindIds" binding="VacationKindId" type="Int32[]" />
      <property name="VacationBeginDates" binding="DateBegin" type="DateTimeOffSet[]" />
      <property name="VacationEndDates" binding="DateEnd" type="DateTimeOffSet[]" />
      <property name="VacationDuration" binding="VacationDuration" type="Int32[]" />
      <property name="State">Complete</property>
      <property name="StateChangeReason"></property>
    </flowStep>
  </flowSteps>
</facilityFlowScheme>
```

Пример услуги с несколькими шагами

Услуга состоит из двух шагов: вызова прикладной функции и старта задачи.

Шаг вызова прикладной функции через функцию **HRManagement.GetVacationKindIdFromVacationId** по параметру услуги **VacationId** получает идентификатор вида отпуска и записывает в параметр услуги **VacationKindId**.

Шаг старта задачи создает задачу типа **VacationShiftTask**, в задаче заполняет свойства **Author**, **StartedBy**, **Employee**, **Created**, **Modified**, **Vacation1**, **VacationKind1**, **DataBegin1**, **Duration1** и вызывает метод **Start** для старта задачи.

```
<facilityFlowScheme firstStepIndex="0" lastStepIndex="-1">
  <flowSteps>
    <flowStep index="0" name="GetVacationKindIdFromVacationId" type="ExecuteFunctionStep">
      <property name="NextStepIndex">1</property>
      <property name="ExecutionMethod">GET</property>
      <property name="TargetSystem">HRSolution</property>
      <property name="vacationId" binding="VacationId" />
      <property name="vacationKindId" binding="VacationKindId,Mode=PropertyToParameter"
type="Int32" />
      <property name="FunctionName">HRManagement.GetVacationKindIdFromVacationId</property>
    </flowStep>
    <flowStep index="1" name="RunVacationShiftTask" title="Запустить задачу по переносу
отпуска" type="StartWorkProcessStep">
      <property name="ProcessName">IVacationShiftTasks</property>
      <property name="TargetSystem">DirectumRX</property>
      <property name="NextStepIndex">-1</property>
      <property name="Author">{'Id':$(CurrentUser.DirectumRX.EmployeeId)}</property>
      <property name="StartedBy">{'Id':$(CurrentUser.DirectumRX.EmployeeId)}</property>
      <property name="Employee">{'Id':$(CurrentUser.DirectumRX.EmployeeId)}</property>
      <property name="Created" type="DateTimeOffSet">#NOW</property>
      <property name="Modified" type="DateTimeOffSet">#NOW</property>
      <property name="Vacation1" binding="VacationId,Format={{'Id':{0}}}" />
      <property name="VacationKind1" binding="VacationKindId,Format={{'Id':{0}}}" />
      <property name="DataBegin1" binding="BeginDate" type="DateTimeOffSet" />
      <property name="Duration1" binding="VacationDuration" type="String" />
    </flowStep>
  </flowSteps>
</facilityFlowScheme>
```

decisionStep – шаг решения

Узел содержит информацию по шагу решения.

```
<decisionStep index="" name="">
  <case condition="" nextStepIndex=""/>
</decisionStep>
```

Атрибуты узла:

- [Обязательный] **index** – индекс шага;
- **name** – наименование.

Внутри шага описываются варианты решений в виде узлов case, которые содержат набор условий. В зависимости от условий определяется, какой вариант выбрать и какой следующий шаг потока будет выполняться.

Узел case

Определяет вариант решения. Выполняет переход к указанному шагу, если условие верно.

Структура узла:

```
<case condition="WithAttachment = &amp;quot;true&amp;quot;" nextStepIndex="1" />
```

Атрибуты узла:

- **condition** – условие;
- [Обязательный] **nextStepIndex** – индекс следующего шага.

Типы поддерживаемых операций в условии:

<=	Меньше или равно
<	Меньше
>=	Больше или равно
>	Больше
!=	Не равно
!	Логическое отрицание
=	Равно
&	Логическая конъюнкция
|	Логическая дизъюнкция
in	Входит в множество
notin	Не входит в множество
contains	Содержит

Типы поддерживаемых операндов в условии:

null	Пустое значение
boolean	Логическое значение
integer	Целое значение
double	Значение с плавающей точкой
string	Строка
date	Дата
parameter	Значение параметра услуги
set	Множество
now	Текущее время
today	Текущая дата

facilityParameter – параметр услуги

Узел содержит информацию по различным типам параметров. Тип параметра определяется обязательным атрибутом **type**.

ВАЖНО. В виджете типа facilityShortcuts параметры услуги могут задаваться динамически из источника данных в формате JSON. Требуемый формат данных источника см. в разделе «Тип виджета facilityShortcuts».

Доступные типы параметров:

Тип параметра	Атрибуты
FacilityParameter – базовый класс параметра Содержит свойства в виде атрибутов	[Обязательный] **name** – имя параметра. **defaultValue** – значение по умолчанию. Не используется для типа параметра FacilityParameterFile. Для параметров типа FacilityParameterDateTime и FacilityParameterDateTimeInterval: • дата указывается в формате ISO 8601; • поддерживаются макросы **#NOW**, **$(Environment.Now)**, **#TODAY** и **$(Environment.Today)**. Если для параметра типа FacilityParameterDateTimeInterval указана дата или макрос из списка выше, соответствующее значение установится в начало и в конец интервала. [Обязательный] **isRequired** – признак обязательности заполнения. У видимых контролов формы для необязательных параметров добавляется подсказка **не обязательно**. [Обязательный] **isCollection** – признак того, что значение параметра является коллекцией. Для типа параметра FacilityParameterBoolean не поддерживается коллекция флажков (checkbox): при значении атрибута **isCollection=true** флажок не отображается. Для отображения в конфигурации необходимо указать значение **isCollection=false**

Тип параметра	Дополнительные свойства, которые задаются в дочерних узлах
FacilityParameterBoolean – параметр типа «Логическое значение»	
FacilityParameterDateTime – параметр типа «Дата и время»	**minValue** – минимальное значение. Указывается в формате ISO 8601. **maxValue** – максимальное значение. Указывается в формате ISO 8601
FacilityParameterDateTimeInterval – параметр типа «Интервал дат». Наследуется от **FacilityParameterDateTime**	**maxLength** – максимальная длительность интервала в днях
FacilityParameterNumber – параметр типа «Число»	**isInteger** – признак целочисленного значения. **precision** – точность. **minValue** – минимальное значение. **maxValue** – максимальное значение. **showThousandSeparator** – отображение разделителей. Значение по умолчанию **false** – число отображается без разделителей
FacilityParameterReferenceRecord – параметр типа «Запись справочника»	[Обязательный] **systemName** – имя целевой системы. [Обязательный] **referenceName** – имя справочника.

Тип параметра	Дополнительные свойства, которые задаются в дочерних узлах
	[Обязательный] **displayRequisite** – отображаемый реквизит. [Обязательный] **storeRequisite** – хранимый реквизит. **filter** – строка фильтра. **IsUserQuerySupported** – признак отображения поля ввода значения для фильтрации. **detailTemplate** – шаблон для дополнительной информации, отображаемой в списке. Поддерживается работа с mustache-шаблоном. В качестве данных передается объект соответствующей записи справочника. При разработке шаблона учитывайте особенности: • HTML-теги будут отображены в виде строки, а не встроены в разметку сайта; • вместо тега поддерживается стандартный перенос строк. **iconRequisite** – имя свойства справочника с иконкой для отображения в списке. Свойство должно содержать URL внешнего ресурса или имя стандартной иконки решения. **defaultIcon** – иконка для отображения в списке по умолчанию. Можно указать имя стандартной иконки решения или макрос **#INITIALS**. В этом случае иконка формируется из начальных букв первых двух слов в значении свойства, указанного в **displayRequisite**. Для фона используется цвет **Secondary30Color**, заданный в настройках сайта. Если не указать значение или указать некорректное имя иконки, то по умолчанию будет отображаться стандартная иконка **gallery**. Если ни **defaultIcon**, ни **iconRequisite** не указаны, то иконки отображаться не будут
FacilityParameterSelect – параметр типа «Выбор из нескольких значений»	[Обязательный] **values** – выбираемые значения. Содержит узлы **value**. Значения должны быть с типом «Число». [Обязательный] **captions** – список заголовков выбираемых значений. Содержит узлы **caption**. **details** – дополнительная информация для отображения в списке. Содержит узлы **detail** для каждого элемента списка. Поддерживается работа с mustache-шаблоном. Данные в этот узел не передаются. При разработке учитывайте особенности: • HTML-теги будут отображены в виде строки, а не встроены в разметку сайта; • вместо тега поддерживается стандартный перенос строк. **icons** – иконки для отображения в списке. Содержит узлы **icon** для каждого элемента списка. В узле **icon** можно указать URL внешнего ресурса или имя стандартной иконки решения. **defaultIcon** – иконка для отображения в списке по умолчанию. Можно указать имя стандартной иконки решения или макрос **#INITIALS**. В этом случае иконка формируется из начальных букв первых двух слов в значении свойства, указанного в **displayRequisite**. Для фона используется цвет **Secondary30Color**, заданный в настройках сайта.

Тип параметра	Дополнительные свойства, которые задаются в дочерних узлах
	Если не указать значение или указать некорректное имя иконки, то по умолчанию будет отображаться стандартная иконка **gallery**. Если ни **defaultIcon**, ни **iconRequisite** не указаны, то иконки отображаться не будут
FacilityParameterString – параметр типа «Строка»	**multiline** – признак отображения нескольких строк. **maxLength** – максимальная длина строки
FacilityParameterFile – параметр типа «Файл»	**maxSize** – максимальный размер файла в байтах. **fileTypes** – список допустимых расширений файлов через запятую. Например, **.doc,.docx,.pdf,.png,.jpg,.tiff,.jpeg**. Также расширения файлов можно задать с помощью типов **MIME**
FacilityParameterQuery – параметр типа «Выпадающий список» со значениями, получаемыми из внешнего источника	[Обязательный] **dataSource** – адрес источника данных. **dataSourceSingle** – адрес источника данных для получения одиночного элемента. ВАЖНО. Источник для свойств **dataSource** и **dataSourceSingle** должен возвращать JSON-массив строк, либо строку с сериализованным JSON-массивом объектов. [Обязательный] **dataSourceValueField** – имя поля в данных, которое сохраняется как значение параметра. [Обязательный] **dataSourceCaptionField** – имя поля в данных, которое отображается для выбора в интерфейсе. **detailTemplate** – шаблон для дополнительной информации, отображаемой в списке. Поддерживается работа с mustache-шаблоном. В качестве данных передается объект соответствующей записи справочника. При разработке шаблона учитывайте особенности: • HTML-теги будут отображены в виде строки, а не встроены в разметку сайта; • вместо тега поддерживается стандартный перенос строк. **dataSourceIconField** – имя свойства в источнике данных с иконкой для отображения в списке. Свойство должно содержать URL внешнего ресурса или имя стандартной иконки решения. **defaultIcon** – иконка для отображения в списке по умолчанию. Можно указать имя стандартной иконки решения или макрос **#INITIALS**. В этом случае иконка формируется из начальных букв первых двух слов в значении свойства, указанного в **displayRequisite**. Для фона используется цвет **Secondary30Color**, заданный в настройках сайта. Если не указать значение или указать некорректное имя иконки, то по умолчанию будет отображаться стандартная иконка **gallery**. Если ни **defaultIcon**, ни **iconRequisite** не указаны, то иконки отображаться не будут. В параметре с типом **FacilityParameterQuery** поддерживаются: • поиск и постраничная загрузка; • выбор инициатора

Поиск и постраничная загрузка

Пользовательский ввод

Если источник данных параметра с типом **FacilityParameterQuery** поддерживает поиск, то в этом параметре можно отобразить пользовательский ввод. Для этого используется макрос **{query}**, вместо которого подставляется введенное пользователем значение. Пример:

```
<dataSource><![CDATA[/api/references?systemName=EssBase&referenceName=ICities&requisites=Id%
2CName&queryRequisite=Name&query={query}]]></dataSource>
```

где значение **queryRequisite=Name** определяет, что поиск ведется по реквизиту **Name**, а введенное пользователем значение передается в параметр запроса **query**, который источник обрабатывает как значение для поиска.

Постраничная загрузка данных

Если источник данных поддерживает постраничную выгрузку, то в этом параметре можно включить постраничную загрузку данных. Для этого используется макрос **{top}**, вместо которого подставляется количество элементов, которое необходимо загрузить. Пример:

```
<dataSource><![CDATA[/api/references?systemName=EssBase&referenceName=ICities&requisites=Id%
2CName&takeCount={top}]]></dataSource>
```

где в параметре **takeCount** указывается макрос **{top}**, отвечающий за количество элементов, которое необходимо загрузить. При открытии списка загружается 25 элементов. Когда достигнут конец списка, загружается еще 25 элементов.

Значение по умолчанию

При постраничной загрузке данных для подгрузки значения по умолчанию необходимо указывать источник данных в свойстве dataSourceSingle. В нем используется макрос **{value}**, вместо него устанавливается значение, которое необходимо загрузить. Например:

```
<dataSourceSingle><![CDATA[/api/references?
systemName=EssBase&referenceName=ICities&requisites=Id%2CName&filter=Id%20eq%20{value}
&takeCount=1]]></dataSourceSingle>
```

Механизмом пропуска

При постраничной загрузке поддерживается работа с механизмом пропуска. Для этого используется макрос **{skip}**, вместо которого подставляется количество элементов, которое необходимо пропустить при загрузке. Пример:

```
<dataSource><![CDATA[/api/references?systemName=EssBase&referenceName=ICities&requisites=Id%
2CName&takeCount={top}&skipCount={skip}]]></dataSource>
```

где в параметре **skipCount** указывается макрос **{skip}**, отвечающий за количество элементов, которое необходимо пропустить. В приведенном примере **{top}** всегда равен **25**, а **{skip}** увеличивается на **25**.

Поиск и постраничная загрузка

Поддерживается одновременная работа с поиском и постраничной загрузкой:

```
<dataSource><![CDATA[/api/references?systemName=EssBase&referenceName=ICities&requisites=Id%
2CName&queryRequisite=Name&query={query}&takeCount={top}&skipCount={skip}]]></dataSource>
```

Выбор инициатора

Если в услуге требуется выбор инициатора, добавьте параметр типа **FacilityParameterQuery** с именем **Initiator**. Источником данных для параметра может быть прикладная функция или сервис. Информация возвращается в виде JSON в следующем виде:

positions – список должностей сотрудника. Каждая запись списка должна иметь свойства:

- **id** – идентификатор сотрудника для этой должности (int);
- **isPrimary** – признак основной должности (bool);
- **title** – название должности (string);
- **department** – название подразделения (string);
- **organization** – название организации (string).

Логика работы контрола выбора инициатора:

- если должностей нет (positions=[]), то контрол на форму не добавляется. Значение параметра устанавливается из свойства **defaultValue** параметра;
- если в списке должностей есть основная должность (isPrimary=true), то по умолчанию устанавливается значение, соответствующее идентификатору сотрудника для этой должности;
- если в списке нет основной должности, то проверяется наличие должности, чей идентификатор сотрудника совпадает со значением **defaultValue**. Если такая должность есть, то **defaultValue** устанавливается в качестве значения параметра.
- если ни одно из условий выше не выполняется, значение по умолчанию не устанавливается.

Пример конфигурации параметра выбора инициатора

```
<facilityParameter type="FacilityParameterQuery" name="Initiator"
defaultValue="$(CurrentUser.DirectumRX.EmployeeId)" isRequired="false" isCollection="false">
  <dataSource>https://mocki.io/v1/d92b0e1f-ca55-4a3c-8098-6efbb55e64d5</dataSource>
  <dataSourceValueField>id</dataSourceValueField>
  <dataSourceCaptionField>name</dataSourceCaptionField>
</facilityParameter>
```

Пример ответа сервиса, указанного в **dataSource**:

```
{
  "positions": [
    {
      "id": 120,
      "isPrimary": true,
      "title": "Начальник отдела информационного обеспечения",
      "department": "Отдел информационного обеспечения",
      "organization": "ООО Техносистемы"
    },
    {
```

```
      "id": 117,
      "isPrimary": false,
      "title": "Младший бухгалтер",
      "department": "Бухгалтерия",
      "organization": "ООО Техносистемы"
    }
  ]
}
```

Если для параметра **Initiator** не указан элемент формы, то он отображается на каждой вкладке.

Чтобы элемент формы с инициатором отображался только на определенной вкладке, добавьте отдельный узел facilityFormItem с типом FacilityFormParameter. В свойстве **parameterName** укажите имя параметра **Initiator**, а в свойстве **parentName** имя нужной вкладки. Пример:

```
<facilityFormItem type="FacilityFormParameter">
  <parameterName>Initiator</parameterName>
  <parentName>VacationInfo</parentName>
</facilityFormItem>
```

Конфигурация параметра типа «Справочник»

Параметр с именем **VacationKindId** типа «Запись справочника» является обязательным. Параметр не является коллекцией значений. Данные для параметра берутся из справочника **VacationKind** целевой системы Directum RX по фильтру **Paid eq false**. В качестве отображаемого значения реквизит **Name**, в качестве хранимого значения – реквизит **Id**.

```
<facilityParameter type="FacilityParameterReferenceRecord" name="VacationKindId"
defaultValue="" isRequired="true" isCollection="false">
  <systemName>DirectumRX</systemName>
  <referenceName>VacationKind</referenceName>
  <displayRequisite>Name</displayRequisite>
  <storeRequisite>Id</storeRequisite>
  <filter>Paid eq false</filter>
  <isUserQuerySupported>false</isUserQuerySupported>
</facilityParameter>
```

Конфигурация параметра типа «Выбор из нескольких значений»

Параметр с именем **Select** типа «Выбор из нескольких значений» не является обязательным и не является коллекцией значений. Данные для параметра передаются из узлов:

- **values** – хранимые значения;
- **captions** – отображаемые значения;
- **details** – дополнительная информация. Оформлена с разными вариантами экранирования.

Пример:

```
<facilityParameter type="FacilityParameterSelect" name="Select" defaultValue=""
isRequired="false" isCollection="false">
  <values>
    <value>1</value>
    <value>2</value>
    <value>3</value>
  </values>
  <captions>
```

```
    <caption>Caption1</caption>
    <caption>Caption2</caption>
    <caption>Caption3</caption>
  </captions>
  <details>
    <detail>
      &amp;quot;detail1.1&amp;quot;
      &amp;amp;
      detail1.2
    </detail>
    <detail>detail2.1&amp;#10;detail2.2&amp;#10;detail2.3</detail>
    <detail>
      <![CDATA[
        "detail3.1"
        &
        detail3.2
      ]]>
    </detail>
  </details>
</facilityParameter>
```

facilityFormItem – элемент формы услуги

Элементы формы определяют набор контролов, из которых строится диалоговое окно запуска услуги. Конфигурация задает типы элементов, а также их привязку к параметрам услуги. Существует несколько типов элементов. Тип каждого элемента задается в обязательном атрибуте **type**.

ВАЖНО. В виджете типа facilityShortcuts элементы формы могут задаваться динамически из источника данных в формате JSON. Требуемый формат данных источника см. в разделе «Тип виджета facilityShortcuts».

Доступные типы элементов:

Тип элемента	Дополнительные свойства, которые задаются в дочерних узлах
FacilityFormItem – базовый тип элемента формы	**hint** – подсказка. **parentName** – имя родительского элемента. В качестве родительского элемента может выступать группа или вкладка. Атрибут **name** – имя элемента формы. Для групп и вкладок атрибут является обязательным. Используется для указания в узле **parentName** и для управления доступностью элементов с типом **FacilityFormParameter** – скрытие, блокировка
FacilityFormTextblock – элемент формы типа «Произвольный текстовый блок»	[Обязательный] **content** – содержимое. Поддерживается работа с mustache-шаблоном. Данные берутся из элементов формы, ключами являются имена параметров формы. Дополнительно см. раздел «Лямбда-функции, доступные в mustache-шаблонах». **isHidden** – признак скрытого элемента
FacilityFormButton – элемент формы типа «Кнопка»	**text** – текст кнопки. **isEnabled** – признак доступности элемента. **isHidden** – признак скрытого элемента. Скрытые элементы не валидируются, даже если у параметра заданы ограничения. **icon** – имя иконки, отображаемой в интерфейсе. Можно указать только стандартные иконки решения.

Тип элемента	Дополнительные свойства, которые задаются в дочерних узлах
	onClickEventHandler – обработчик события нажатия на кнопку
FacilityFormParameter – элемент формы задания значения параметра услуги	**showHeaders** – признак отображения заголовков. **isReadOnly** – признак элемента только для чтения. **isHidden** – признак скрытого элемента. Скрытые элементы не валидируются, даже если у параметра заданы ограничения. **label** – метка для параметра. [Обязательный] **parameterName** – имя запрашиваемого параметра. **valueChangedEventHandler** – обработчик события изменения значения. **itemAddingEventHandler** – обработчик события начала добавления элемента в коллекцию. Для типа **FacilityParameterFile** вместо данного параметра используется **valueChangedEventHandler**. **itemAddedEventHandler** – обработчик события окончания добавления элемента в коллекцию. Для типа **FacilityParameterFile** вместо данного параметра используется **valueChangedEventHandler**. **itemRemovingEventHandler** – обработчик события начала удаления элемента из коллекции. Для типа **FacilityParameterFile** вместо данного параметра используется **valueChangedEventHandler**. **itemRemovedEventHandler** – обработчик события окончания удаления элемента из коллекции. Для типа **FacilityParameterFile** вместо данного параметра используется **valueChangedEventHandler**. **calculateFunction** – функция для вычисления значения. **placeholder** – подсказка ввода. Для интервала дат можно задать для каждого поля, используя разделитель «|». Пример: «Начало|Дней|Окончание»
FacilityFormTable – элемент формы типа «Таблица»	**title** – заголовок. **showHeaders** – признак необходимости отображения заголовков столбцов. **isHidden** – признак скрытой таблицы. Ячейки не валидируются, даже если у параметров заданы ограничения. **isReadOnly** – признак доступности только для чтения. **itemAddingEventHandler** – обработчик события начала добавления элемента в коллекцию. **itemAddedEventHandler** – обработчик события окончания добавления элемента в коллекцию. **itemRemovingEventHandler** – обработчик события начала удаления элемента из коллекции. **itemRemovedEventHandler** – обработчик события окончания удаления элемента из коллекции. [Обязательный] **columns** – столбцы, содержит список дочерних узлов **column**, которые описывают настройки столбца. Каждый узел имеет следующие дочерние узлы: • **header** – заголовок. Если значение не задано, то столбец не доступен для выбора в качестве первого при работе в мобильном представлении;

Тип элемента	Дополнительные свойства, которые задаются в дочерних узлах
	• **isReadOnly** – признак доступности данных в ячейках этого столбца только для чтения; • **isHidden** – признак скрытого столбца. Скрытые элементы не валидируются, даже если у параметра заданы ограничения; • [Обязательный] **parameterName** – имя параметра, значения которого формируют содержимое столбца по строкам. Ограничения: • не поддерживаются параметры типа FacilityParameterDateTimeInterval, FacilityParameterFile и FacilityParameterBoolean; • у параметров типа FacilityParameterString свойство **multiline** всегда считается равным **false**; • **valueChangedEventHandler** – обработчик события изменения значения; • **calculateFunction** – функция для вычисления значения; • **placeholder** – подсказка ввода; • **size** – ширина столбца. Возможные значения: • **число** – обрабатывается как доля от общей ширины таблицы. Значение по умолчанию **1**; • **проценты** – процент от общей ширины таблицы; • **usage** – вариант отображения столбца в мобильном представлении. Возможные значения: • **Default** – по умолчанию; • **TitleColumn** – первый столбец; • **PrimaryColumn** – второй столбец
FacilityFormGroup – элемент формы типа «Группа»	**label** – метка В группе не поддерживаются элементы с типами **FacilityFormTab**, **FacilityFormGroup** и **FacilityFormTable**. По умолчанию элементы в группе располагаются горизонтально. Элементы располагаются вертикально, если: • используется мобильное представление; • в группе больше трех элементов; • в группе используются элементы с типом, отличным от **FacilityFormParameter**; • в группе используются параметры с одним из следующих типов: • FacilityParameterDateTimeInterval; • FacilityParameterFile; • FacilityParameterBoolean; • в группе используется параметр с типом FacilityParameterString со свойством **multiline=true**; • в группе используются параметры с атрибутом **isCollection=true**
FacilityFormTab – элемент формы типа «Вкладка»	**label** – метка. **isHidden** – признак скрытой вкладки. Дочерние элементы не валидируются, даже если у параметров заданы ограничения. **navigatedToEventHandler** – обработчик события прихода на текущую вкладку. **navigatedFromEventHandler** – обработчик события ухода с текущей вкладки

Описание событий см. в разделе «Обработчики событий реквизитов».

Пример конфигурации двух элементов формы

Пример описывает:

- элемент формы типа «Таблица». Таблица с заголовком **Планируемые отпуска** содержит четыре столбца: **Вид отпуска**, **Дата начала**, **Количество дней**, **Дата окончания**. Все столбцы отображаются с заголовками и соответствуют параметрам услуги. Столбец **Дата окончания** доступен только для чтения;
- скрытый элемент формы для скрытого параметра **WorkItemId**, находящийся в группе **firstGroup**;
- группу **firstGroup**, находящуюся на вкладке **firstTab**;
- не скрытую вкладку **firstTab**.

```
<facilityFormItem type="FacilityFormTable">
  <title>Планируемые отпуска</title>
  <showHeaders>true</showHeaders>
  <columns>
    <column>
      <header>Вид отпуска</header>
      <isReadOnly>false</isReadOnly>
      <parameterName>VacationKindId</parameterName>
    </column>
    <column>
      <header>Дата начала</header>
      <isReadOnly>false</isReadOnly>
      <parameterName>DateBegin</parameterName>
    </column>
    <column>
      <header>Количество дней</header>
      <isReadOnly>false</isReadOnly>
      <parameterName>VacationDuration</parameterName>
    </column>
    <column>
      <header>Дата окончания</header>
      <isReadOnly>true</isReadOnly>
      <parameterName>DateEnd</parameterName>
    </column>
  </columns>
</facilityFormItem>
<facilityFormItem type="FacilityFormParameter" name="WorkItemIdFormItem">
  <isReadOnly>false</isReadOnly>
  <isHidden>true</isHidden>
  <parameterName>WorkItemId</parameterName>
  <calculateFunction><![CDATA[]]></calculateFunction>
  <valueChangedEventHandler><![CDATA[]]></valueChangedEventHandler>
  <parentName>firstGroup</parentName>
</facilityFormItem>
<facilityFormItem type="FacilityFormGroup" name="firstGroup">
  <hint />
  <label />
  <parentName>firstTab</parentName>
</facilityFormItem>
<facilityFormItem type="FacilityFormTab" name="firstTab">
  <hint />
  <label />
  <isHidden>false</isHidden>
  <navigatedToEventHandler><![CDATA[]]></navigatedToEventHandler>
  <navigatedFromEventHandler><![CDATA[]]></navigatedFromEventHandler>
</facilityFormItem>
```

widget – виджет

Узел **widget** предназначен для настройки виджета группы услуг. Атрибуты узла:

- [Обязательный] **name**. Имя виджета;
- [Обязательный] **type**. Тип виджета. Если значение отличается от тех, что используются для предопределенных типов виджетов, то виджет определяется как свободно конфигурируемый.

Структура узла:

```
<widget name="{Имя виджета}" type="{Тип виджета}">
  <icon>
    {Иконка}
  </icon>
  <title>
    {Заголовок}
  </title>
  <dataSourceUrl>
    {Источник данных}
  </dataSourceUrl>
  <properties>
    {Свойства виджета}
  </properties>
</widget>
```

icon – имя стандартной иконки, отображаемой в интерфейсе. Можно указать только иконки решения.

title – отображаемый заголовок виджета. В качестве значения указывается текст или ссылка на строку локализации.

[Обязательный] **dataSourceUrl** – адрес источника данных для формирования виджета.

properties – свойства виджета. Каждое свойство задается в дочернем узле **property** с атрибутами, названия которых определяет имя свойства, а содержание – его значение. В качестве значения свойства можно указать ссылку на строку локализации.

Структура виджета, ожидаемая модель данных и доступный набор свойств зависит от его типа:

Тип виджета	Описание
itemList	Список элементов
workItemList	Список рабочих элементов
facilityShortcuts	Список вариантов запуска услуг
templateWidget	Текст с поддержкой mustache-шаблона. Подробнее см. в документации Mustache статью Mustache Manual и раздел «Лямбда-функции, доступные в mustache-шаблонах»
embedded	Фрейм, например видеофайл или внешний сайт. Содержимое фрейма определяется настройкой

Также виджет может быть свободно сконфигурированным, при этом его структура не зависит от типа и включает заданные разработчиком контролы.

dataSourceUrl – адрес источника данных

Адрес сервиса, предоставляющего данные. Если в качестве источника используются сервисы личного кабинета, то допускается задание относительного пути, начиная с **/api**.

Если в качестве источника указана проекция прикладных объектов, то путь задается в формате **/api/<Имя проекции>**. В пути к проекции в параметрах запроса можно указать дополнительные параметры:

- сортировка по возрастанию:

  Пример для проекции **contracts** с сортировкой по возрастанию по свойству **Created**:

  ```
  /api/contracts?orderBy=Created
  /api/contracts?orderBy=+Created
  /api/contracts?orderBy=Created asc
  ```

- сортировка по убыванию:

  Пример для проекции **contracts** с сортировкой по убыванию по свойству **Created**:

  ```
  /api/contracts?orderBy=-Created
  /api/contracts?orderBy=Created desc
  ```

  ВАЖНО. При задании **dataSourceUrl** потребуется экранирование.

  Пример экранирования символа «+» в пути запроса с сортировкой по возрастанию:

  ```
  /api/contracts?orderBy=%2bCreated
  ```

- пропуск указанного количества записей.

  Пример для проекции **contracts** с пропуском первых 5 записей:

  ```
  /api/contracts?skip=5
  ```

- возврат указанного количества записей.

  Пример для проекции **contracts** возвращающий только 5 первых записей:

  ```
  /api/contracts?take=5
  ```

Если используется несколько параметров, то они применяются в следующей последовательности: **orderBy**, **skip**, **take**.
Пример для проекции **contracts** с сортировкой по возрастанию по свойству **Created**, пропуском первых 5 записей и возвратом первых 3 из оставшихся записей:

```
/api/contracts?orderBy=Created&skip=5&take=3
```

Если в виджете находится таблица, то данные для нее должны приходить в специальном формате. Для этого добавьте к адресу источника параметр **flags=1**.
Пример:

```
/api/workprocesses/HRSolution/StatementTask?filter=(Author eq
$(CurrentUser.DirectumRX.EmployeeId))&amp;flags=1
```

Тип виджета itemList

Тип виджета **itemList** предназначен для работы со списком элементов, которые загружаются из указанного в виджете источника данных.

Свойства виджета

Структура узла **properties**:

```
<properties>
  <defaultObjectCard>{Карточка объекта по умолчанию}</defaultObjectCard>
  <detailsTemplate>{Подзаголовок элементов}</detailsTemplate>
</properties>
```

defaultObjectCard – наименование карточки объекта по умолчанию, которая будет открываться из виджета. Подробнее см. в разделе «objectCard – карточка объекта». Используется, если не найдена карточка с именем открываемого элемента. Если свойство не указано или соответствующая карточка не сконфигурирована, используется карточка с именем **Default**.

detailsTemplate – подзаголовок элементов. Поддерживает mustache-шаблон, входными данными которого является модель элемента. Дополнительно см. раздел «Лямбда-функции, доступные в mustache-шаблонах».

Ожидаемая модель данных

В модели данных, получаемой из источника, ожидаются параметры:

- **title** – заголовок элемента;
- **icon** – иконка элемента. Можно указать только иконки решения. Если свойство не указано или в нем указано некорректное имя иконки, то по умолчанию используется иконка **memo**;
- **defaultActionName** – имя действия по умолчанию. По этому имени выполняется поиск кнопки в конфигурации карточки для выполнения ее обработчика;
- **defaultActionCaption** – заголовок кнопки действия по умолчанию;
- **itemLink** – ссылка на объект, который открывается при нажатии на элемент списка. Также используется для вызова действия по умолчанию.

Если в качестве источника данных указана проекция, то в свойства проекции для виджета добавьте свойства с типом **String** с соответствующими именами. Пример:

```
<property name="title" caption="Заголовок" type="String" />

<property name="icon" caption="Иконка" type="String" />
<property name="defaultActionName" caption="Имя действия" type="String" />
<property name="defaultActionCaption" caption="Заголовок действия" type="String" />
<property name="itemLink" caption="Ссылка на элемент" type="String" />
```

Значения для этих свойств заполняются через источники проекций. Пример:

```
<map sourceProperty="Title" destProperty="title" mode="PropertyToProperty" />

<map destProperty="icon" mode="ConstantToProperty" constValue="sign" />
<map destProperty="defaultActionName" mode="ConstantToProperty" constValue="Sign" />
```

```
<map destProperty="defaultActionCaption" mode="ConstantToProperty" constValue="Подписать" />
<map sourceProperty="Id" destProperty="itemLink" mode="PropertyToProperty"
converter="toEmployeeSignAssignmentLink" />
```

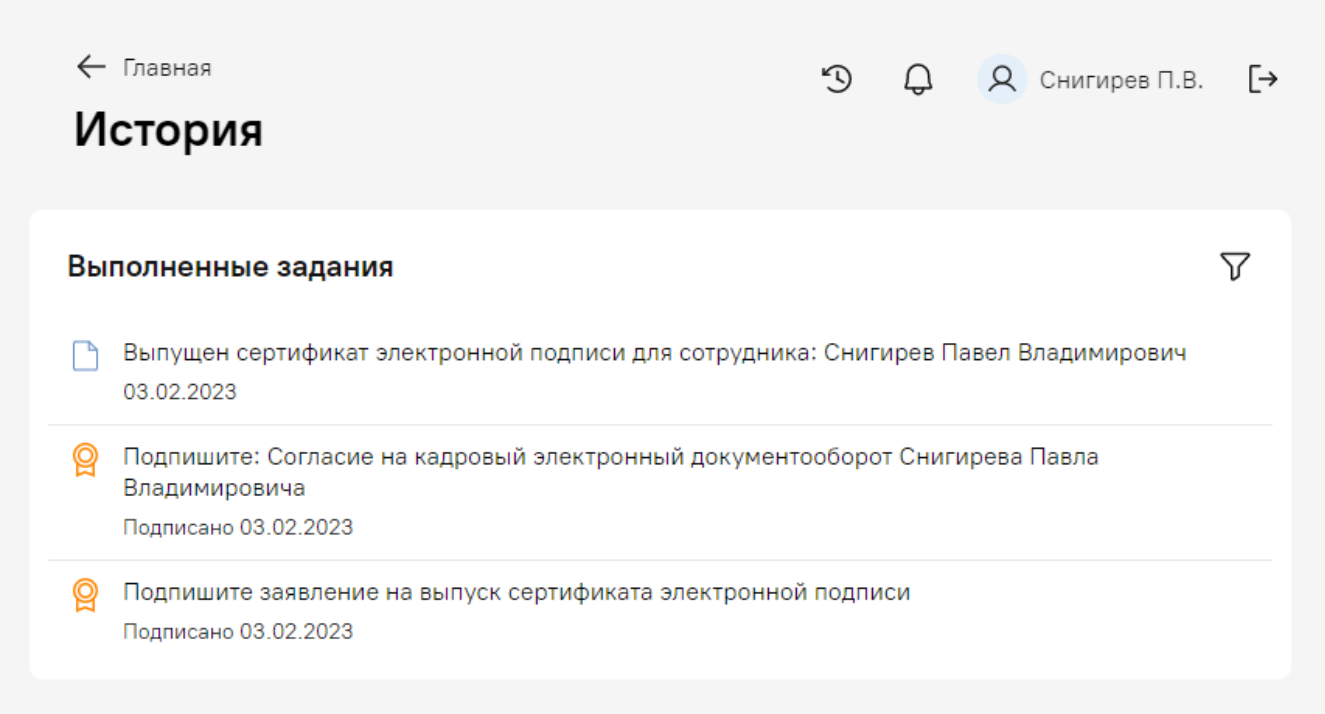
ПРИМЕЧАНИЕ. Для заполнения свойства элементов **itemLink** из проекции необходимо реализовать серверный конвертер. Пример:

```
<valueConverter name="toEmployeeSignAssignmentLink" runat="Server">
  <function>
    <![CDATA[
      return $"/workitems/DirectumRX/IEmployeeSignAssignments/{value}";
    ]]>
  </function>
</valueConverter>
```

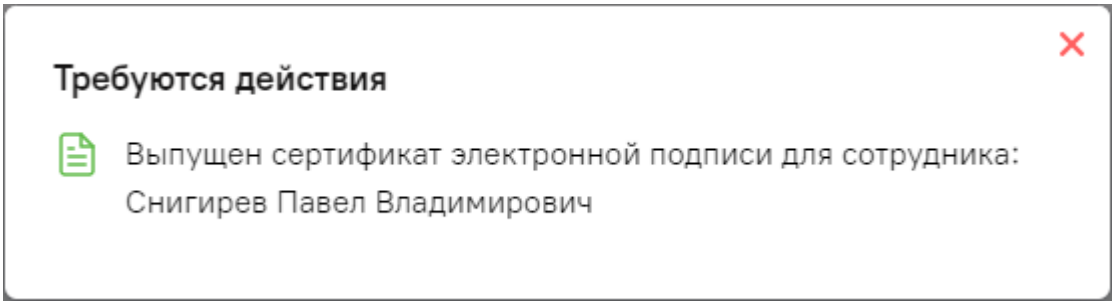
Пример настройки виджета «История»

```
<widget name="history" type="itemList">
  <icon>history</icon>
  <title>@.FACILITYGROUPS_WIDGET_HISTORY</title>
  <dataSourceUrl>...</dataSourceUrl>
  <properties>
    <defaultObjectCard>TestCard</defaultObjectCard>
    <detailsTemplate>{{result}} {{#localizeDate}}{{finished}}{{/localizeDate}}
</detailsTemplate>
  </properties>
</widget>
```

В результате виджет принимает вид:



При открытии элемента из виджета происходит получение карточки объекта по имени элемента:



Если такая карточка не найдена, то открывается карточка **TestCard**.

Тип виджета workItemList

Тип виджета **workItemList** – это обертка для типа itemList, в которой поддерживаются только элементы рабочего процесса.

Элементы загружаются в список из указанного в виджете источника данных. В качестве источника рекомендуется использовать проекции или API сервера приложения личного кабинета.

Свойства виджета

```
<properties>
  <defaultObjectCard>{Карточка объекта по умолчанию}</defaultObjectCard>
  <workItemSubtitle>{Подзаголовок элементов}</workItemSubtitle>
</properties>
```

Свойства виджета задаются в узле **properties** в дочерних узлах:

- **defaultObjectCard** – аналогично свойству **defaultObjectCard** типа виджета itemList;
- **workItemSubtitle** – аналогично свойству **detailsTemplate** типа виджета itemList.

Ожидаемая модель данных

В модели данных, получаемой из источника, ожидаются параметры:

- **title** – заголовок элемента;
- **icon** – иконка элемента. Можно указать только иконки решения. Если свойство не указано или в нем указано некорректное имя иконки, то по умолчанию используется иконка **paper**;
- **defaultActionName** – имя действия по умолчанию. По этому имени выполняется поиск кнопки в конфигурации карточки для выполнения ее обработчика;
- **defaultActionCaption** – заголовок кнопки действия по умолчанию;
- **system** – название системы, из которой загружаются данные;
- **name** – название типа элемента;
- **id** – ИД элемента.

  ПРИМЕЧАНИЕ. Из параметров **system**, **name** и **id** формируется ссылка на элемент, поэтому они обязательны в модели получаемых данных.

- **isRead** – признак того, что элемент прочитан. Если указано значение **false**, то строка в списке будет выделена полужирным шрифтом.

Пример настройки

```
<widget name="history" type="workItemList">
  <icon>history</icon>
  <title>@.FACILITYGROUPS_WIDGET_HISTORY</title>
  <dataSourceUrl>...</dataSourceUrl>
  <properties>
    <defaultObjectCard>TestCard</defaultObjectCard>
    <workItemSubtitle>{{result}} {{#localizeDate}}{{finished}}{{/localizeDate}}
</workItemSubtitle>
  </properties>
</widget>
```

Тип виджета facilityShortcuts

Тип виджета **facilityShortcuts** предназначен для работы с вариантами запуска услуги, которые загружаются в список из указанного источника данных. В качестве источника используются данные в формате JSON со списком вариантов услуги.

Параметры и элементы формы услуги могут отличаться для разных ее вариантов. Такие параметры называются динамическими.

Свойства виджета

У виджета типа **facilityShortcuts** нет предопределенных свойств.

Ожидаемая модель данных

В модели данных, получаемой из источника, ожидаются параметры:

- **facilityId** – идентификатор или имя услуги.
- **icon** – имя иконки, отображаемой в интерфейсе. Можно указать только иконки решения.
- **title** – заголовок варианта запуска услуги. Отображается в списке вариантов запуска виджета.
- **details** – детальное описание варианта запуска услуги. Параметр типа «Строка».
- **parameters** – описание динамических параметров услуги. Описание одного параметра состоит из:
  - **name** – имя параметра. Соответствует атрибуту name узла facilityParameter;
  - **type** – тип параметра. Соответствует атрибуту type узла facilityParameter;
  - **defaultValue** – значение по умолчанию. Соответствует атрибуту **defaultValue** узла **facilityParameter**.

    ПРИМЕЧАНИЕ. В динамических элементах значение по умолчанию для параметра с типом FacilityParameterDateTimeInterval дополнительно поддерживает строку с сериализованным объектом: **{ DateBegin: *string | null*, Duration: *string* }**. Также **DateBegin** поддерживают макросы;
  - **isRequired** – признак обязательности заполнения. Соответствует атрибуту **isRequired** узла **facilityParameter**;

- **isCollection** – признак того, что параметр является коллекцией. Соответствует атрибуту **isCollection** узла **facilityParameter**;
- **properties** – свойства параметра. Подробнее см. описание узлов в разделе «Узел facilityParameter».

  ВАЖНО. В конфигурации виджета для параметра с типом **FacilityParameterSelect** значения свойств **values**, **captions**, **details** и **icons** перечисляются либо через запятую, либо в виде сериализованного массива строк. Например:

```
{
  "values": "1,2,3",
  "captions": "[\"Caption1\",\"Caption2\",\"Caption3\"]"
}
```

- **form** – описание элементов формы с динамическими параметрами услуги. Описание формы состоит из списка элементов формы услуги **items**. Описание одного элемента формы состоит из:
  - **type** – тип элемента формы. Соответствует атрибуту **type** узла **facilityFormItem**;
  - **name** – имя элемента формы. Соответствует атрибуту **name** узла **facilityFormItem**;
  - **hint** – подсказка. Соответствует дочернему узлу **hint** узла **facilityFormItem**;
  - **parentName** – имя родительского элемента. Соответствует дочернему узлу **parentName** узла **facilityFormItem**;
  - **properties** – свойства элемента. Подробнее см. описание узлов в разделе «Узел facilityFormItem».

Пример настройки

Виджет **applications** отображает список вариантов запуска услуг, которые возвращаются прикладной функцией **GetJSON4WidgetStatementMock**.

```
<widget name="applications" type="facilityShortcuts">
  <icon />
  <title>@.FACILITYGROUPS_WIDGET_APPLICATION_TEMPLATE</title>
   <dataSourceUrl>/api/functions/GetJSON4WidgetStatementMock</dataSourceUrl>
  <properties />
</widget>
```

Пример ответа от прикладной функции GetJSON4WidgetStatementMock

Ответ содержит описание варианта запуска услуги «Прочее заявление». В примере к услуге добавляются динамические параметры **Текст заявления**, таблица со столбцами **Дата** и **Примечание** и элементы формы для них. Также в примере добавляются элементы формы группа и вкладка. Динамические элементы формы запуска услуги отображаются после элементов формы услуги, заданных в конфигурации.

Все значения свойств JSON-объекта должны быть строковыми. Для свойств-признаков также поддерживаются логические значения. Для необязательных свойств можно указать пустое значение **null**. Подробнее об обязательности свойств, их описании и возможных значениях см. в разделах «Узел facilityParameter» и «Узел facilityFormItem».

```
[
  {
    "facilityId": "0365B815-B108-429F-A9AF-E5A0C993B654",
    "icon": "error",
    "title": "Услуга с новым форматом динамических параметров",
    "details": "Детальная информация",
    "parameters": [
      {
```

```
      "name": "Description",
      "type": "FacilityParameterString",
      "defaultValue": null,
      "isRequired": true,
      "isCollection": false,
      "properties": {
        "maxLength": "1000",
        "multiline": "False"
      }
    },
    {
      "name": "Note",
      "type": "FacilityParameterString",
      "defaultValue": null,
      "isRequired": true,
      "isCollection": false,
      "properties": {
        "maxLength": "1000",
        "multiline": "False"
      }
    },
    {
      "name": "BeginDate",
      "type": "FacilityParameterDateTime",
      "defaultValue": null,
      "isRequired": true,
      "isCollection": false,
      "properties": {
        "minValue": "2000-01-01",
        "maxValue": "2024-01-01"
      }
    }
  ],
  "form": {
    "items": [
      {
        "type": "FacilityFormParameter",
        "hint": "Текст заявления",
        "name": "DescriptionItem",
        "parentName": "DescriptionGroup",
        "properties": {
          "parameterName": "Description",
          "showHeaders": "True",
          "isReadOnly": "False",
          "isHidden": "False",
          "label": "Описание",
          "itemAddedHandler": "",
          "itemAddingHandler": "",
          "itemRemovedHandler": "",
          "itemRemovingHandler": "",
          "valueChangedHandler": "",
          "calculateFunction": ""
        }
      },
      {
        "type": "FacilityFormGroup",
        "hint": "",
        "name": "DescriptionGroup",
        "parentName": "DescriptionTab",
        "properties": {
          "label": "Группа с описанием"
        }
      },
      {
        "type": "FacilityFormTab",
        "hint": "",
        "name": "DescriptionTab",
```

```
        "properties": {
          "isHidden": "False",
          "label": "Вкладка с описанием",
          "navigatedFromEventHandler": "",
          "navigatedToEventHandler": ""
        }
      },
      {
        "type": "FacilityFormTable",
        "hint": "",
        "name": "ExampleTable",
        "parentName": "",
        "properties": {
          "itemAddedHandler": "",
          "itemAddingHandler": "",
          "itemRemovedHandler": "",
          "itemRemovingHandler": "",
          "showHeaders": "True",
          "title": "Пример таблицы с заголовком",
          "columns": [
            {
              "parameterName": "BeginDate",
              "header": "Дата",
              "isReadOnly": "False",
              "isHidden": "False",
              "valueChangedHandler": "",
              "calculateFunction": "",
              "placeholder": "",
              "usage": "TitleColumn"
            },
            {
              "parameterName": "Note",
              "header": "Примечание",
              "isReadOnly": "False",
              "isHidden": "False",
              "valueChangedHandler": "",
              "calculateFunction": "",
              "placeholder": "",
              "usage": "PrimaryColumn"
            }
          ]
        }
      }
    ]
  }
]
```

Тип виджета templateWidget

Виджет типа **templateWidget** предназначен для отображения данных из источника в соответствии с заданным разработчиком шаблоном.

Свойства виджета

```
<properties>
  <template>{Mustache-шаблон для отображения данных}</template>
</properties>
```

template – mustache-шаблон, который накладывается на данные, получаемые виджетом из источника данных, указанного в **dataSourceUrl**. Результат подставляется в разметку виджета. Подробнее см. в документации Mustache статью Mustache Manual. Дополнительно см. раздел «Лямбда-функции, доступные в mustache-шаблонах».

Ожидаемая модель данных

Виджет принимает данные в соответствии с заданным шаблоном.

Пример настройки

Виджет отображает информацию из сервиса https://jsonplaceholder.typicode.com/users согласно указанному шаблону.

```
<widget name="MustacheTemplate" type="templateWidget">
  <icon>check-mark</icon>
  <title>Виджет с шаблоном</title>
  <dataSourceUrl>https://jsonplaceholder.typicode.com/users</dataSourceUrl>
  <properties>
    <template>
      <![CDATA[
        {{#.}}
          Name: {{name}} (<a href="mailto:{{email}}">{{email}}</a>)
        {{/.}}
      ]]>
    </template>
  </properties>
</widget>
```

Тип виджета embedded

Виджет типа **embedded** предназначен для отображения фреймов, например видеороликов или внешних сайтов.

Свойства виджета

У виджета типа **embedded** нет предопределенных свойств.

Ожидаемая модель данных

Виджет принимает прямую ссылку на встраиваемый сайт.

Пример настройки

Виджет с фреймом, который содержит видео из сервиса Youtube:

```
<widget name="videoFrame" type="embedded">
   <icon />
   <title>Видео</title>
   <dataSourceUrl>https://www.youtube.com/embed/WNESW8hu_-o</dataSourceUrl>
</widget>
```

Свободно конфигурируемый виджет

Структура свободно конфигурируемого виджета определяется контролами, которые заданы в узле **properties**, дочернем узле **controls**.

Структура узла:

```
<widget name="{Имя виджета}" type="{Тип виджета}">
  <title>{Заголовок виджета}</title>
  <dataSourceUrl>{Источник данных}</dataSourceUrl>
  <properties>
    <controls>
      <control name="{Имя контрола}">
        <properties>
          {Свойства контрола}
        </properties>
      </control>
    </controls>
  </properties>
</widget>
```

controls – список контролов виджета в виде узлов control.

Пример настройки

Виджет «Запланированные отпуска» отображает информацию из справочника **Отпуска** в виде таблицы. Каждая колонка, кроме последней, составляется из реквизитов справочника. Последняя колонка содержит действие, вызываемое по гиперссылке из строки таблицы.

```
<widget name="vacations" type="tableCard">
  <title>Запланированные отпуска</title>
  <dataSourceUrl>...</dataSourceUrl>
  <properties>
    <controls>
      <control name="dataTable">
        <properties>
          <columns>
            <column>
              <title>ИД</title>
              <type>int</type>
              <name>VacationId</name>
              <field>id</field>
              <size>1</size>
              <converter></converter>
              <template></template>
              <usage></usage>
              <maxCellLines></maxCellLines>
            </column>
            ...
          </columns>
          <maxRowCount>2</maxRowCount>
          <showHeaders>false</showHeaders>
          <rowTitleColumn>...</rowTitleColumn>
          <rowSubtitleColumn>...</rowSubtitleColumn>
          <rowPrimaryColumn>...</rowPrimaryColumn>
          <expandButton>
            <caption>...</caption>
            <dataSourceUrl>...</dataSourceUrl>
            <viewTitle>...</viewTitle>
          </expandButton>
        </properties>
```

```
      </control>
    </controls>
  </properties>
</widget>
```

control – контрол

control – контрол виджета. Содержит атрибут **name**, в котором задается тип контрола. Возможные значения:

- **keyValueBlocks** – список в формате <ключ-значение>. Во входных данных должен быть массив объектов со свойствами **key** и **value**. Для корректного применения стилей виджет, в котором указан контрол, должен иметь тип **keyValues**;
- **dataTable** – таблица;
- **textInfoBlocks** – информационные блоки. Во входных данных должен быть массив объектов со следующими свойствами:
  - **title** – заголовок блока;
  - **text** – текст с информацией;
  - **important** – признак важности информации. Если установлен, то блок выделится голубым фоном.

Узел **control** содержит дочерний узел **properties**, в котором задаются свойства контрола. Может содержать различные дочерние узлы в зависимости от типа контрола.

properties – свойства контрола

Свойства контрола задаются в узле **properties** в дочерних узлах. Состав доступных свойств определяется типом контрола.

Узлы, доступные для всех контролов

groupBy – узел с описанием группировки виджета:

- **field** – название поля в модели данных, по которому будет производиться группировка;
- **convert** – название функции, выполняющей группировку. Может принимать только значение **ByYear**;
- **order** – узел определяет направление сортировки сгруппированных значений. По умолчанию – прямое. Если необходимо задать обратное направление сортировки, то передается значение **desc**.

Узлы, доступные для всех контролов, кроме таблиц

showAllButton – узел с описанием кнопки **Показать все**. Свойство должно содержать дочерний узел **button**.

button – кнопка без границ и заливки. Содержит дочерние узлы:

- **caption** – заголовок кнопки;

- **action** – действие, которое выполняется при нажатии на кнопку. Содержит следующие узлы:

  type – тип действия для выполнения. Возможные значения:

  - **CHOOSE_FACILITY_GROUP** – выбор группы услуг;
  - **RUN_FACILITY** – запуск услуги;

  parameters – список параметров действия: может иметь разные дочерние узлы для разных типов действий:

  - **CHOOSE_FACILITY_GROUP**: **id** – ИД или имя группы услуг;
  - **RUN_FACILITY**: **id** – ИД или имя услуги, **bindings** – узел с привязками имен параметров услуги и имен параметров из табличных данных (имя параметра таблицы соответствует имени столбца). Дочерние узлы соответствуют именам параметров услуги, а их значения – именам параметров таблицы.

Кнопка будет отображаться без границ и заливки.

bindings – имя поля в модели данных, из которого используется значение. Поддерживаются форматы:

- **<поле>** – используется значение напрямую;
- **<поле>.<подполе>** – **<поле>** должно содержать в себе список подполей. Значение берется из свойства **value** подполя, у которого **name** равно **<подполе>**.

ПРИМЕЧАНИЕ. Если значение параметра не задано, то данные будут использоваться как есть.

Узлы, доступные для таблиц

maxRowCount – максимальное количество строк для отображения в таблице.

showHeaders – признак необходимости отображать заголовки столбцов.

rowTitleColumn – первый столбец, отображаемый в виде заголовка в мобильном представлении. Также значение этого столбца будет выступать заголовком в карточке строки таблицы.

rowSubtitleColumn – столбец, отображаемый в виде подзаголовка в мобильном представлении.

rowPrimaryColumn – второй столбец, отображаемый в мобильном представлении.

expandButton – узел с описанием кнопки открытия расширенного представления таблицы:

- **caption** – заголовок кнопки;
- **dataSourceUrl** – адрес источника данных. Заполняется по аналогии с **dataSourceUrl** виджета;
- **viewTitle** – заголовок контейнера расширенного представления.

columns – узел со списком столбцов таблицы.

column – узел с описанием столбца таблицы:

- **header** – заголовок столбца;

- **type** – тип данных в столбце. Возможные значения:

  date – значение будет отображаться в виде локализованной даты. Подробнее см. на сайте developer.mozilla.org статью Date.prototype.toLocaleDateString();

  bool – значение будет обрабатываться как логическое (см. **TrueValue** и **FalseValue** ниже);

  number – значение будет отображаться в виде числа. Если числовое значение состоит более чем из 16 знаков, то рекомендуется использовать тип столбца **string**;

  string – значение будет отображаться в виде строки;

  status – значение отображается в виде цветного маркера. Возможные значения:

  - **InProcess** – в работе, оранжевый маркер. В тексте отображается локализованное значение **В работе** или **In process**;
  - **Draft** – черновик, серый маркер. В тексте отображается локализованное значение **Черновик** или **Draft**;
  - **Suspended** – остановлена, красный маркер. В тексте отображается локализованное значение **Остановлена** или **Suspended**;
  - **Completed** – завершена, зеленый маркер. В тексте отображается локализованное значение **Завершена** или **Completed**;
  - **UnderReview** – на приемке, оранжевый маркер. В тексте отображается локализованное значение **На приемке** или **Under review**;
  - **Aborted** – прекращена, красный маркер. В тексте отображается локализованное значение **Прекращена** или **Aborted**;
  - остальные значения – синий маркер. В тексте отображается не измененное значение.

  Для других значений в узле **type** дополнительных преобразований не происходит;
- **name** – уникальное имя для идентификации столбца;
- **field** – имя поля в модели данных, из которого используется значение. Поддерживаются форматы:

  <поле> – используется значение напрямую;

  <поле>.<подполе> – **<поле>** должно содержать в себе список подполей. Значение берется из свойства **value** подполя, у которого **name** равно **<подполе>**;
- **size** – ширина столбца. Возможные значения:
  - число – обрабатывается как доля от общей ширины виджета, например **100**. Не может быть использовано с другими значениями ширины;
  - пиксели – устанавливается заданная ширина в пикселях, например **100px**. Если суммарная ширина столбцов, заданных в пикселях, выходит за пределы таблицы, то столбцы пропорционально сжимаются;
  - проценты – устанавливается заданная ширина в процентах, например **100%**;
  - **auto** – ширина вычисляется автоматически в зависимости от содержимого столбца. Значение используется по умолчанию, если **size** явно не указан;
- **converter** – имя конвертера. Приоритет конвертера выше приоритета стандартной обработки данных. Если узел не указан или указан некорректный конвертер, то для дат, логических значений и статуса используются стандартные механизмы конвертации. Доступны клиентские конвертеры.

Пример использования системного конвертера **toLongMonthYear** в конфигурации:

```
<converter>toLongMonthYear</converter>
```

Прикладной конвертер указывается в формате <Имя решения>.<Имя конвертера>. Пример использования прикладного конвертера с именем **booleanToOnOffState** из прикладного решения **Converters**:

```
<converter>Converters.booleanToOnOffState</converter>
```

ВАЖНО. Прикладные конвертеры возвращают строку. Исключением является конвертер, который используется для столбца с типом **status**. Для таких столбцов, помимо строк, конвертер может возвращать объект в формате **{ fillColor: string, textColor: string, title: string }**. Например:

```
class toStatus {
      convert(value, parameter, language) {
        return {
          title: value,
          fillColor: '#DCDCDC',
        };
      }
    }
```

- **usage** – в значении перечисляются места, где должен отображаться столбец. Возможные значения:

  Widget – в виджете;

  Card – в карточке;

  FullSize – в расширенном представлении.

  Если значение не заполнено, столбец будет скрыт;
- **maxCellLines** – максимальное количество строк в ячейке. По умолчанию **1**;
- **template** – шаблон для отображения. Должен содержать в себе описатель **{value}**, в который подставится значение, обработанное стандартными механизмами или конвертером;
- **trueValue** – значение для отображения, если значение реквизита равно **true**. Используется для данных с типом **bool**;
- **falseValue** – значение для отображения, если значение реквизита равно **false**. Используется для данных с типом **bool**.

buttons – узлы с описанием кнопок действий таблицы.

button – узел с описанием кнопки действия таблицы:

- **name** – имя действия;
- **icon** – имя иконки, отображаемой в интерфейсе. Можно указать только иконки решения;
- **caption** – заголовок действия;
- **modificator** – модификатор кнопки. Если значение не заполнено кнопка отображается с рамкой. Может принимать значения:

  string **minimal** – простая кнопка, без границ и заливки, только текст и иконки;

  string **primary** – кнопка с заливкой;
- **clickHandler** – обработчик события при клике.

handlers – узлы с описанием обработчиков событий таблицы. **dataLoadedHandler** – обработчик события загрузки данных. Описание события см. в разделе «В виджетах».

tile – плитка быстрого действия

Узел tile предназначен для настройки плиток быстрых действий. Атрибуты узла:

- [Обязательный] **name**. Имя плитки;
- **index**. Индекс, по которому определяется приоритет расположения плитки. Если индексы совпадают, то приоритет соответствует порядку загрузки плиток в базу данных.

Структура узла:

```
<tile name="{Имя виджета}" index="{Индекс}">
  <authorization>
    {Права доступа к плитке}
  </authorization>
  <backgroundColor>{Цвет фона}</backgroundColor>
  <foregroundColor>{Цвет текста}</foregroundColor>
  <title>{Заголовок плитки}</title>
  <icon>{Иконка}</icon>
  <navigationTargetUrl>{Адрес перехода}</navigationTargetUrl>
  <dataSourceUrl>{Источник данных} </dataSourceUrl>
  <contentTemplate>
    {Шаблон содержимого}
  </contentTemplate>
  <clickEventHandler>
    {Обработчик события}
  </clickEventHandler>
</tile>
```

authorization – права доступа к плитке.

backgroundColor – цвет фона. Поддерживаются форматы HEX ("#000000", "#000") и RGB ("0,0,0"). По умолчанию серый (#B4B3B3). Рекомендованные цвета:

#B4B3B3	#5CA2FC	#FCAE48
#3ACA7D	#8077FD	#FD91C0



#B4B3B3	#5CA2FC	#FCAE48
#40D7E1	#3BCD44	#68A5D0

foregroundColor – цвет текста. Поддерживает запись цвета в форматах HEX и RGB. По умолчанию белый (#FFF).

title – заголовок плитки.

icon – имя иконки. Можно указать только иконки решения.

navigationTargetUrl – адрес перехода при нажатии на плитку.

dataSourceUrl – адрес источника данных для содержимого плитки.

contentTemplate – mustache-шаблон для отображения содержимого. Подробнее см. раздел «Лямбда-функции, доступные в mustache-шаблонах». Для подстановки доступны данные, полученные с источника, указанного в параметре **dataSourceUrl**.

clickEventHandler – обработчик события нажатия на плитку. Доступные параметры:

- **tile** – свойства плитки;
- **data** – данные, полученные с источника, указанного в **dataSourceUrl**.

ПРИМЕЧАНИЕ. Если у плитки заполнены дочерние узлы **navigationTargetUrl** и **clickEventHandler**, то приоритетным является **clickEventHandler**. В этом случае перехода по ссылке, указанной в **navigationTargetUrl**, не произойдет.

Цвета, рекомендованные для фона плиток быстрых действий:

Пример настройки:

```
<tile name="StopThePlanet" index="0">
  <authorization>
    <allow roles="User" />
  </authorization>
  <backgroundColor>#FCAE48</backgroundColor>
  <foregroundColor>#000</foregroundColor>
  <title>@.TILE_STOP_PLANET</title>
  <icon>wave</icon>
  <navigationTargetUrl>/facilities/TestDropdown</navigationTargetUrl>
  <dataSourceUrl>https://contoso.directum.ru/tile.json</dataSourceUrl>
  <contentTemplate>
    Current planet: {{name}}
  </contentTemplate>
  <clickEventHandler><![CDATA[console.log('tile')]]></clickEventHandler>
</tile>
```

function – прикладная функция

Узел содержит конфигурацию прикладной функции. В нем поддерживаются прикладные функции следующих типов:

- **target** – выполняет запуск функции в указанной целевой системе;
- **remote** – выполняет HTTP запрос на удаленный хост.

В значениях параметров и заголовков функций используются переменные и константы.

Общие настройки

Атрибуты узла, общие для обеих прикладных функций:

- **id** – идентификатор;
- [Обязательный] **name** – имя прикладной функции, по которому она вызывается;
- [Обязательный] **type** – тип прикладной функции. Возможные значения: **Target** – прикладная функция в целевой системе, **Remote** – удаленная прикладная функция;
- [Обязательный] **methods** – список HTTP-методов, которые можно использовать для вызова функции. Возможные значения:
  - **GET** – функция доступна к вызову GET-методом;
  - **POST** – функция доступна к вызову POST-методом;
  - **GET,POST** – функция доступна к вызову GET и POST методами.

Общие дочерние узлы:

- **parameters** – параметры функции в виде набора узлов **parameter**.
- **result** – результат выполнения функции.

Для описания параметров прикладной функции используются узлы **parameter** с атрибутами:

- [Обязательный] **name** – имя параметра;
- [Обязательный] **type** – тип параметра. Возможные значения:
  - **String** – строковое значение;
  - **Boolean** – логическое значение;
  - **Integer** – целочисленное значение;
  - **Double** – значение с плавающей точкой;
  - **DateTime** – значение типа «Дата и время»;
  - **Object** – значение типа «Объект»;
- [Обязательный] **scope** – область доступа к параметру. Возможные значения:
  - **Server** – значение параметра может быть установлено только на стороне сервера;
  - **Client** – значение параметра может быть передано с клиента в момент вызова функции;
- **isArray** – признак, что значением параметра является массив значений. Возможные значения атрибута: **true**, **false**.

Узел **parameter** может содержать дочерний узел **defaultValue**, в котором указывается значение параметра по умолчанию.

Для описания возвращаемого значения прикладной функции используются узел **result** с атрибутами:

- [Обязательный] **type** – тип результата. Возможные значения:
  - **String** – строковое значение;
  - **Boolean** – логическое значение;
  - **Integer** – целочисленное значение;
  - **Double** – значение с плавающей точкой;
  - **DateTime** – значение типа «Дата и время»;
  - **Object** – значение типа «Объект»;
- **isArray** – задает признак того, что значением результата является массив значений. Возможные значения атрибута: **true**, **false**.

Узел **result** может содержать дочерний узел **fallbackValue**. В нем указывается значение результата, который будет возвращать функция в случае возникновения ошибки выполнения.

Индивидуальные настройки

Конфигурации параметров, индивидуальных для прикладной функции, указываются в отдельных узлах:

- **target** – описание функции. Обязательный узел для target-функций;
- **remote** – описание функции. Обязательный узел для remote-функций.

Узел **target** задает описание вызываемой целевой функции, которое задается в атрибутах узла:

- [Обязательный] **system** – имя целевой системы;
- [Обязательный] **function** – имя вызываемой функции в целевой системе;
- [Обязательный] **method** – HTTP-метод для вызова функции в целевой системе. Возможные значения:
  - **GET** – функция вызывается GET-методом;
  - **POST** – функция вызывается POST-методом.

Пример конфигурации target-функции:

```
<function name="GetEmployeeCertificates" type="Target" methods="GET">
  <parameters>
    <parameter name="employeeId" type="Integer" scope="Server">
      <defaultValue>$(CurrentUser.DirectumRX.ActorId)</defaultValue>
    </parameter>
  </parameters>
  <result type="Object" isArray="true" />
  <target system="EssBase" function="EssPlatform.GetEmployeeCertificates" method="GET" />
</function>
```

Узел **remote** задает описание вызываемой удаленной функции, которое задается атрибутах узла:

- [Обязательный] **url** – адрес для вызова удаленного запроса;
- [Обязательный] **method** – HTTP метод для вызова удаленного запроса. Возможные значения:
  - **GET** – функция вызывается GET-методом;
  - **POST** – функция вызывается POST-методом;
  - **PUT** – функция вызывается PUT-методом;
  - **DELETE** – функция вызывается DELETE-методом;
  - **PATCH** – функция вызывается PATCH-методом;
  - **HEAD** – функция вызывается HEAD-методом;
- **headers** – необязательный раздел. Содержит список заголовков, добавляемых к запросу. Каждый элемент списка является узлом **header**, который описывает заголовок запроса. В атрибуте **name** задается имя заголовка. Значение заголовка задается из значения узла.

Пример конфигурации remote-функции:

```
<function name="CreateUser" type="Remote" methods="POST">
  <parameters>
    <parameter name="name" type="String" scope="Client" />
    <parameter name="job" type="String" scope="Client" />
  </parameters>
  <result type="Object" />
  <remote url="https://reqres.in/api/users" method="POST">
    <headers>
      <header name="User-Agent">Directum.Core.FunctionService</header>
    </headers>
  </remote>
</function>
```

Переменные и константы в значениях параметров и заголовков

Переменные или прикладные константы можно использовать в:

- значениях по умолчанию;
- fallback-значении результата. Оно используется, если вызов функции завершился с ошибкой.
- заголовках remote-функции.

Для использования переменных имя переменной необходимо указать в формате **$(...)**. Пример: $(CurrentUser.DirectumRX.ActorId). Доступные переменные:

- **CurrentUser.<Удостоверение>.<Атрибут>** – возвращает атрибут из заданного удостоверения текущего пользователя. Пример: CurrentUser.DirectumRX.ActorId;
- **CurrentUser.GivenName** – имя собственное текущего пользователя;
- **CurrentUser.Surname** – фамилия;
- **CurrentUser.Patronym** – отчество;
- **CurrentUser.FullName** – полное имя;
- **CurrentUser.ShortName** – короткое имя (Фамилия И.О.)
- **CurrentUser.Initials** – инициалы (И.О.)
- **CurrentUser.Name** – идентифицирующее имя;

- **CurrentUser.Id** – внутренний идентификатор текущего пользователя;
- **Environment.Now** – текущее время в UTC;
- **Environment.Today** – текущий день в UTC.

Для использования константы ее имя указывается между символами **%**, пример: %TenantId%. Список констант задается в конфигурационном файле сервера приложения личного кабинета appsettings.json в секции **Constants**.

localizedStringValue – строка локализации

Узел **localizedStringValue** содержит информацию по значению локализуемой строки. Атрибуты узла:

- [Обязательный] **code**. Код строки. Должен быть в верхнем регистре;
- [Обязательный] **language**. Двухбуквенный код языка в верхнем регистре по стандарту ISO 639-1;
- **useat**. Место использования. Возможные значения:
  - **server** – на сервере;
  - **client** – на клиенте;
  - **both** – на сервере и на клиенте.

  Если атрибут не задан, строка доступна только на сервере.

Структура узла:

```
<localizedStringValue code="КодСтроки" language="КодЯзыка" useat="МестоИспользования">Локализованное значение</localizedStringValue>
```

Примеры настройки

Строки, указанные в узлах **localizedStringValue** с местом использования **server** или **both**, могут использоваться в XML-файлах в узлах, поддерживающих локализацию:

```
<localizedStringValue code="AGREE" language="RU" useat="server">Я ознакомлен с Соглашением и принимаю его условия</localizedStringValue>
<localizedStringValue code="AGREE" language="EN" useat="server">I have read the Agreement and accept its terms</localizedStringValue>
```

Строки в названиях пунктов пользовательского соглашения:

```
<agreementClause name="DigitalInteraction" isRequired="True">
  <text>@AGREEMENTS.AGREE</text>
</agreementClause>
```

Для упрощения конфигурирования, в XML-файлах доступны сочетания символов «@.» и «@@». Они заменяются на имя решения, в котором используются. При использовании в решении **AGREEMENTS** строки из примера выше можно заменить следующими строками:

```
<text>@.AGREE</text>
<text>@@AGREE</text>
```

Строки, указанные в узлах **localizedStringValue** с местом использования **client** или **both**, могут использоваться в прикладных событиях при использовании функции ESS.L:

```
<localizedStringValue code="CERTIFICATE_ISSUED" language="RU" useat="client">Сертификат успешно выпущен.</localizedStringValue>
<localizedStringValue code="CERTIFICATE_ISSUED" language="EN" useat="client">Certificate issued successfully.</localizedStringValue>
```

Такая строка может использоваться в обработчике события для принятия пользовательского соглашения:

```
<acceptedEventHandler>
  <![CDATA[
    ESS.toasts.showToast(ESS.L('@AGREEMENTS.CERTIFICATE_ISSUED'));
  ]]>
</acceptedEventHandler>
```

usageAgreement – пользовательское соглашение

Узел содержит конфигурацию пользовательского соглашения. Ниже приведен пример конфигурации с описанием параметров:

```
<usageAgreement id="10a4ef8b-23e0-4d5e-1a6f-08d8cf4a6ce8" title="Заголовок"
isAcceptionRequired="True" isConfirmationRequired="True">
  <authorization>
    <allow roles="User" />
  </authorization>
  <text><![CDATA[Текст соглашения в MD]]></text>
  <acceptedEventHandler><![CDATA[Код обработчика принятия соглашения на JS]]
></acceptedEventHandler>
  <refusedEventHandler><![CDATA[Код обработчика отказа от соглашения на JS]]
></refusedEventHandler>
  <openingEventHandler><![CDATA[Код обработчика события открытия соглашения]]
></openingEventHandler>
  <clauses>
    <agreementClause name="DigitalInteraction" isRequired="True">
      <text><![CDATA[Текст пункта соглашения]]></text>
    </agreementClause>
    <agreementClause name="CertificateIssuing" isRequired="True">
      <text><![CDATA[Текст пункта соглашения]]></text>
    </agreementClause>
  </clauses>
</usageAgreement>
```

Описание атрибутов **usageAgreement**:

- **id** – идентификатор соглашения;
- [Обязательный] **title** – заголовок соглашения, отображаемый в диалоге;
- [Обязательный] **isAcceptionRequired** – признак необходимости обязательного принятия соглашения, кнопка **Отказаться** в диалоге будет недоступна;
- [Обязательный] **isConfirmationRequired** – признак необходимости подтверждения соглашения. Требуется подтверждение вторым фактором, например, кодом из SMS-сообщения.

Описание узлов:

- **authorization** – содержит правила авторизации для пользовательского соглашения. Подробнее см. в разделе «Узел authorization»;
- [Обязательный] **text** – текст соглашения;
- **acceptedEventHandler** – обработчик события принятия соглашения;
- **refusedEventHandler** – обработчик события отказа от соглашения;
- **openingEventHandler** – обработчик события открытия соглашения;
- **clauses** – список узлов agreementClause, описывающих пункты соглашения.

Описание событий см.в разделе «В офертах».

Дочерний узел agreementClause

Узел содержит описание пункта соглашения.

Атрибуты **agreementClause**:

- [Обязательный] **name** – наименование пункта;
- [Обязательный] **isRequired** – признак обязательности принятия данного пункта соглашения, без его установки кнопка **Принять** будет недоступна.

Обязательное свойство **text** – текст пункта соглашения.

notificationTemplate – шаблон оповещения

Узел содержит конфигурацию шаблона оповещения. Шаблоны используются для преобразования сообщений, получаемых пользователями личного кабинета из сторонних систем, в оповещения для отображения в интерфейсе.

Входным объектом для формирования оповещения является объект сообщение (**Message**), которое содержит следующие свойства:

- **Title** – заголовок сообщения;
- **Content** – текст содержания сообщения;
- **IsRead** – признак прочитанности;
- **IsImportant** – признак важности.

Кроме того, сообщение может содержать список действий, дополнительные свойства и список вложений, которые автоматически будут добавляться в оповещение к тем действиям и вложениям, которые приходят вместе с сообщением.

Пример конфигурации шаблона оповещения:

```
<notificationTemplate     id="4c508877-4bb5-4526-af19-33333651eb56"     name="WorkItemReceived"
usageCondition="Event = &amp;quot;JobReceived&amp;quot;">
  <authorization>
    <allow roles="User" />
  </authorization>
  <template/>
</notificationTemplate>
```

Атрибуты узла:

- **id** – идентификатор;
- [Обязательный] **name** – имя шаблона;
- **usageCondition** – условие применения. Формулируется на основе выражения выбора (см. пример в настройках шага решения) и значения свойств сообщения.

Конфигурация включает дочерние узлы:

- **authorization** – содержит правила авторизации для шаблона оповещения. Подробнее см. в разделе «Узел authorization»;
- **template** – шаблон формируемого оповещения.

Дочерний узел template

Узел содержит описание оповещения, которое будет сформировано на основе данных входящего сообщения.

Узел определяет, как будут заполняться все свойства оповещения на основе данных свойств сообщения.

Ниже приведен пример шаблона. Оповещение по шаблону отображается для группы с идентификатором 9a661f41-ce53-4247-9f18-e22edc1967a3. Данные в оповещении (заголовок, текст, признак прочитанности и признак важности) берутся из соответствующих свойств входящего сообщения. В карточке оповещения будет кнопка с действием **В группу**, при нажатии на которую будет происходить переход в группу услуг с идентификатором fed3380e-9d25-471f-bade-965f80e8ad8d.

```
<notification>
  <properties>
  <property name="Title" binding="Title" />
  <property name="Content" binding="Content" />
  <property name="FacilityGroupId">9a661f41-ce53-4247-9f18-e22edc1967a3</property>
  <property name="IsRead" binding="IsRead"></property>
  <property name="IsImportant" binding="IsImportant"></property>
  <property name="Actions" type="Action[]">[{"name": "CHOOSE_FACILITY_GROUP", "title": "В
группу", "parameters": { "id": "fed3380e-9d25-471f-bade-965f80e8ad8d"]</property>
  </properties>
</notification>
```

Описание действия

- string **name** – тип действия для выполнения. Возможные значения:
  - **CHOOSE_FACILITY_GROUP** – выбор группы услуг;
  - **RUN_FACILITY** – запуск услуги;
- string **title** – заголовок действия;
- json **parameters** – объект со списком параметров действия: может иметь разную структуру для разных типов действий:
  - **CHOOSE_FACILITY_GROUP**: [required] string **id** – ИД или имя группы услуг;
  - **RUN_FACILITY**: [required] string **id** – ИД или имя услуги.

objectCard – карточка объекта

Узел содержит **objectCard** конфигурацию карточки объекта. Атрибуты узла:

- **id**. идентификатор;
- [Обязательный] **name**. Имя карточки;
- [Обязательный] **header**. Заголовок;
- **icon**. Имя иконки, отображаемой в интерфейсе. Можно указать только иконки решения.

Структура узла:

```
<objectCard id="{Идентификатор}" name="{Имя}" header="{Заголовок}" icon="{Иконка}">
  <authorization>
    {Права доступа}
  </authorization>
```

```
  <views>
    {Представления}
  </views>
  <buttons>
    {Кнопки}
  </buttons>
  <openingEventHandler><Обработчик события «Начало открытия формы»><openingEventHandler>
  <openedEventHandler><Обработчик события «Окончание открытия формы»></openedEventHandler>
  <closedEventHandler><Обработчик события «Окончание закрытия формы»></closedEventHandler>
  <referenceUrl>{Ссылка на веб-страницу справки}</referenceUrl>
</objectCard>
```

authorization – правила авторизации для карточки объекта.

[Обязательный] **views** – представления карточки.

buttons – кнопки на карточке.

openingEventHandler – обработчик события «Начало открытия формы».

openedEventHandler – обработчик события «Окончание открытия формы».

closedEventHandler – обработчик события «Окончание закрытия формы».

referenceUrl – адрес веб-справки по карточке объекта. Можно указать полный путь или путь относительно адреса справки, указанного в настройках сайта личного кабинета. Подробнее см. в руководстве администратора HR Pro, раздел «Настройка платформы личного кабинета».

Пример настройки

```
<objectCard    id="4c508877-4bb5-4526-af19-33333651eb56"    name="TestCard"    header="Тестовая
карточка" icon="sign">
  <authorization>
    <allow roles="User" />
  </authorization>
  <views/>
  <buttons/>
  <openingEventHandler><![CDATA[console.log('opening card')]]></openingEventHandler>
  <openedEventHandler><![CDATA[console.log('opened card')]]></openedEventHandler>
  <closedEventHandler><![CDATA[console.log('closed card')]]></closedEventHandler>
  <referenceUrl>https://webhelp.contoso.ru/card.html</referenceUrl>
</objectCard>
```

views – представления карточки объекта

Ниже приведен пример с описанием представления карточки объекта. Представление сконфигурировано для целевого устройства **Desktop**. В теле карточки отображаются поля, согласно настройке **fields**, содержание карточки **{{subject}}** и группа вложений **AttachmentGroup** с заголовком **TEST**.

```
<views>
  <view deviceIdioms="Desktop" deviceOrientations="Landscape, Portrait">
    <fields>
      <field caption="Тема">Тема задания</field>
      <field caption="Срок" binding="deadline" type="DateTime"
converter="EssBase.convertDeadline"/>
      <field caption="Статус" binding="status" type="Status"/>
      <field caption="Результат" binding="result"/>
      <field caption="Исполнитель" binding="PerformerInfo" type="Employee"/>
    </fields>
    <content>Контент карточки {{subject}}.</content>
    <attachmentsAreas>
       <attachmentsArea title="TEST">
```

```
        <documentGroups>
          <documentGroup>AttachmentGroup</documentGroup>
        </documentGroups>
      </attachmentsArea>
    </attachmentsAreas>
  </view>
</views>
```

Описание атрибутов **view**:

- **deviceIdioms** – идиомы целевого устройства. Возможные значения: **All**, **Phone**, **Desktop**. Значение по умолчанию **All**;
- **deviceOrientations** – ориентации целевого устройства. Возможные значения: **Landscape**, **Portrait**. Значение по умолчанию **Landscape, Portrait**.

Содержание узла **view**:

- **fields** – поля карточки. Содержит список узлов **field**;
- **content** – текст с поддержкой mustache-шаблона. Дополнительно см. раздел «Лямбда-функции, доступные в mustache-шаблонах»;
- **attachmentsAreas** – области вложений. Содержит список узлов **attachmentsArea**. При отсутствии узла карточка отображается без области вложений.

Подробнее о mustache-шаблоне см. в документации Mustache статью Mustache Manual.

Описание атрибутов узла **field**, поле карточки:

- **caption** – заголовок;
- **binding** – имя свойства объекта, из которого будет браться значение;
- **type** – тип поля. Возможные значения:
  - **String** – строка;
  - **Status** – статус;
  - **DateTime** – дата и время;
  - **Date** – дата;
  - **Employee** – сотрудник.

  Значение по умолчанию **String**.
- **converter** – имя конвертера. Доступны прикладные клиентские конвертеры. В качестве вспомогательного параметра передается модель с данными объекта.

Значение узла **field** считается значением по умолчанию. Пример:

```
<field caption="Поле со значением по умолчанию">Значение по умолчанию</field>
```

Оно используется если:

- не указано связанное свойство **binding**;
- связанное свойство отсутствует в модели объекта;
- значение связанного свойства пустое.

Значение из связанного свойства или установленное по умолчанию передается в конвертер. Значение по умолчанию и результат конвертера поддерживают mustache-шаблон. Дополнительно см. раздел «Лямбда-функции, доступные в mustache-шаблонах».

ПРИМЕЧАНИЕ. Если в атрибуте **type** указано **Status**, то значение отображается в том же виде, как для типа **status** в виджете с таблицей. При использовании конвертера также может быть возвращен объект со значением и цветом. Подробнее см. в разделе «Узлы, доступные для таблиц».

Для отображения поля с типом **Employee**, из свойства карточки, которое указано в атрибуте **binding**, используются свойства:

- **FullName** – полное имя;
- **Organization** – наша организация;
- **Department** – подразделение;
- **Position** – должность.

Описание атрибута **attachmentsArea**: [Обязательный] **title** – заголовок.

Содержание узла **attachmentsArea**: [Обязательный] **documentGroups** – области вложений. Содержит список узлов **documentGroup** с именем области вложения.

buttons – кнопки в карточке объекта

Пример описания кнопки карточки. Кнопка с заголовком **Подписать** является основной кнопкой. При клике вызывается обработчик clickEventHandler, который записывает информацию в консоль браузера.

```
<buttons>
  <button name="Sign" isPrimary="true">
    <icon>check-mark</icon>
    <caption>Подписать</caption>
    <clickEventHandler><![CDATA[console.log('sign')]]></clickEventHandler>
  </button>
</buttons>
```

Описание атрибутов **button**:

- [Обязательный] **name** – имя кнопки;
- **isPrimary** – признак основной кнопки.

Дочерние узлы **button**:

- **icon** – имя иконки, отображаемой в интерфейсе. Можно указать только иконки решения;
- **caption** – заголовок кнопки;
- **clickEventHandler** – обработчик события нажатия на кнопку.

projection – проекция

Узел **projection** содержит конфигурацию прикладного объекта **проекции**. Атрибуты узла:

- **id**. Идентификатор;
- [Обязательный] **name**. Имя проекции. Указывается во множественном числе;
- [Обязательный] **title**. Заголовок;
- **predicate**. Условие фильтрации данных в проекции.

Структура узла:

```
<projection name="{Имя}" title="{Заголовок}" id="{Идентификатор}" predicate="{Условие фильтрации}">
  <properties>
    {Свойства проекции}
  </properties>
  <sources>
    {Источники проекции}
  </sources>
</projection>
```

[Обязательный] **properties** – свойства проекции. Узел содержит набор дочерних узлов **property**.

sources – источники проекции. Узел содержит набор дочерних узлов **source**.

Пример настройки:

```
<projection name="Contracts" title="Документы" id="2E3FF1C0-1A99-4F71-A2CF-81B0EBD7AF38" predicate="signaturetype = &amp;quot;Endorsing&amp;quot;">
  <properties/>
  <sources/>
</projection>
```

properties – свойства проекции

Ниже приведен пример с описанием свойств проекции прикладного объекта. Свойство **Author** – строковое, свойство **IsRead** – логического типа.

```
<properties>
  <property name="Author" caption="Автор" type="String" />
  <property name="IsRead" caption="Признак прочитанности" type="Boolean" />
</properties>
```

Описание атрибутов **property**:

- [Обязательный] **name** – имя свойства;
- **caption** – заголовок;
- [Обязательный] **type** – тип свойства. Возможные значения: **String, Number**, **Boolean**, **DateTime**.

sources – источники проекции

Узел **sources** содержит источники проекции. Обязательный атрибут: **path** – путь до источника.

Структура узла:

```
<sources>
  <source path="{Путь до источника}">
    <authorization>
      {Права доступа}
    </authorization>
    <mapping>
      <map sourceProperty="{Свойство-источник}" destProperty="{Свойство-приемник}" mode="{Режим отображения}" constValue="{Константное значение}" fallbackValue="{Значение при null}" converter="{Имя конвертера}"/>
```

```
    </mapping>
  </source>
</sources>
```

authorization – права доступа для данных источника. Подробнее см. в разделе «Узел authorization»;

[Обязательный] **mapping** – список настроек отображения данных источника. Содержит дочерние узлы **map** с атрибутами:

- **sourceProperty**. Имя свойства-источника;
- [Обязательный] **destProperty**. Имя свойства-приемника;
- [Обязательный] **mode**. Режим отображения. Возможные значения: **PropertyToProperty**, **ConstantToProperty**;
- **constValue**. Константное значение;
- **fallbackValue**. Значение, устанавливаемое в свойстве объекта проекции, если исходное значение равно **null** или не определено;
- **convert**. Имя прикладного серверного конвертера. Применяется только для режима отображения **PropertyToProperty**. Подробнее см. в разделе «Узел valueConverter».

Каждый узел **map** определяет, как будет отображаться заданное значение в определенном свойстве объекта проекции – свойстве-приемнике. В качестве значения источника может выступать значение свойства объекта источника данных – свойство-источник, либо константное значение. Используемый вариант определяется режимом отображения, который задается в атрибуте **mode**.

Значение из источника может быть преобразовано с помощью прикладного конвертера до записи в свойство объекта проекции. При этом в конвертер также передаются остальные свойства в виде объекта типа IValueObject. Затем в конвертере можно обратиться к данному объекту. Пример:

```
using System;
using System.Globalization;
using System.Collections.Generic;
using Directum.SelfServiceOffice.Domain.Core;

public class converterUseParameter : IValueConverter
{
  public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
  {
    return (parameter as IValueObject)["userId"] + value.ToString();
  }
}
```

Если элемент является массивом, то каждый его элемент также можно привести к типу IValueObject. Пример:

```
(parameter as IValueObject)["requisites"] as IEnumerable<IValueObject>
```

В этом случае, если элемент является сложным объектом, к каждому свойству элемента массива также можно обращаться по строковому индексу:

```
foreach(var requisite in ((parameter as IValueObject)["requisites"] as
IEnumerable<IValueObject>))
  requisite["Name"].Value
```

Пример настройки

В списке один из источников с адресом **https://jsonplaceholder.typicode.com/todos/**. Для данных, выдаваемых источниками, задано соответствие свойствам проекции:

- свойство **AuthorId**, согласно режиму отображения **PropertyToProperty**, отображается как **Author**. К свойству применяется прикладной конвертер **ToLower**;
- свойство **IsRead**, согласно режиму отображения **ConstantToProperty**, берется из атрибута **constValue**.

```
<sources>
  <source path="https://jsonplaceholder.typicode.com/todos/">
    <authorization>
      <allow roles="User" />
    </authorization>
    <mapping>
      <map sourceProperty="AuthorId" destProperty="Author" mode="PropertyToProperty"
converter="ToLower"/>
      <map sourceProperty="IsRead" mode="ConstantToProperty" constValue="false" />
    </mapping>
  </source>
</sources>
```

extension – расширение объекта

Узел содержит конфигурацию расширения для объекта, который принадлежит к другому прикладному решению. С помощью расширения можно:

- разместить плитки быстрых действий;
- добавить источники к проекции;
- задать дополнительные права доступа к услугам, группам услуг, карточкам и плиткам быстрых действий.

ВАЖНО. Прикладное решение с узлами расширений устанавливается после прикладного решения, которое будет дополняться узлами расширений. Пример: прикладное решение ServiceRequests содержит группу услуг «Заявления» и расширение проекции для базового прикладного решения EssBase. В этом случае сначала необходимо установить прикладное решение EssBase, затем – ServiceRequests.

Размещение плиток быстрых действий

Используется, если в прикладном решении разработаны новые плитки быстрых действий и их нужно добавить в группу услуг другого прикладного решения.

Обязательный атрибут узла – **path**. В нем задается путь до представления (**view**), в котором нужно разместить плитки, в формате FacilityGroups/<ИмяГруппы>/views/<ИмяПредставления>.

В конфигурацию расширения включается дочерний узел **tileslots** с информацией о размещении плиток быстрых действий. Подробнее см. в разделе «Узел view».

Пример конфигурации расширения для размещения плиток быстрых действий в группе услуг **Applications** для представления с именем **WebView**, без указания размещения:

```
<extension path="FacilityGroups/Applications/views/WebView">
  <tileslots/>
</extension>
```

Добавление источников к проекции

Используется, если нужно добавить источник к проекции, которая относится к другому прикладному решению.

Обязательный атрибут узла – **path**. В нем задается путь до проекции в формате: Projections/<ИмяПроекции>.

В конфигурацию расширения включается дочерний узел **sources** с указанием источников проекции. Подробнее см. в разделе «Узел sources».

Пример конфигурации расширения для проекции **Contacts**, без указания источников:

```
<extension path="Projections/Contacts">
  <sources/>
</extension>
```

Расширение прав доступа к объектам

Используется, если нужно дополнить права доступа к объекту другого прикладного решения. Например, разрешить доступ для роли, которой нет в исходном решении. Расширить права можно для групп услуг, услуг, карточек и плиток быстрых действий.

Обязательный атрибут узла – **path**, в нем указывается путь до прикладного объекта. Формат пути зависит от типа объекта:

- FacilityGroups/<Имя группы>;
- FacilityGroups/<Имя группы>/facilities/<Имя услуги>;
- ObjectCards/<Имя карточки>;
- Tiles/<Имя плитки>.

В конфигурацию расширения включается дочерний узел **authorization** с указанием дополнительных прав доступа. Подробнее см. в разделе «Узел authorization».

Пример конфигурации расширения для услуги **SendApplication** из группы услуг **Applications** с добавлением разрешающей роли **Administrator**:

```
<extension path="FacilityGroups/Applications/facilities/SendApplication">
  <authorization>
    <allow roles="Administrator"/>
  </authorization>
</extension>
```

При определении итоговых прав доступа с учетом расширения используются правила:

1. Поддерживаются расширения только для разрешающих ролей в дочерних узлах **allow**.
2. Роли из узла **allow** добавляются к существующим ролям установленного прикладного объекта.

page – прикладная страница

Узел **page** содержит конфигурацию прикладной страницы. Атрибуты узла:

- **id**. Идентификатор;
- [Обязательный] **name**. Имя прикладной страницы;

- **title**. Заголовок.

Структура узла:

```
<page name="{Имя страницы}" title="{Заголовок}" id="{Идентификатор}">
  <content>{Содержимое страницы}</content>
  <pageLoadedHandler><Обработчик события «Открытие прикладной страницы»></pageLoadedHandler>
</page>
```

content – контент прикладной страницы с поддержкой mustache-шаблона. Дополнительно см. раздел «Лямбда-функции, доступные в mustache-шаблонах».

pageLoadedHandler – обработчик события открытия прикладной страницы. Подробнее см.в разделе «В прикладных страницах».

Пример настройки

```
<page name="testPage" title="Тестовая страница" id="4886B166-0EB1-4949-9EB9-AE25801CDB46">
  <content>Контент страницы</content>
  <pageLoadedHandler><![CDATA[console.log('pageLoadedHandler');resolve();]]
></pageLoadedHandler>
</page>
```

component – сторонний компонент

Узел **component** предназначен для добавления сторонних компонентов в прикладную конфигурацию решения. Контролы из таких компонентов можно встроить в элементы с поддержкой Mustache-шаблонов. Для работы с контролом в него при загрузке передается набор данных.

В репозитории GitHub доступен пример стороннего компонента с контролами – одностраничное веб-приложение, которое можно встроить в веб-клиент. Пример написан с использованием библиотеки React. Чтобы разработать свои контролы, скачайте пример стороннего компонента из репозитория и доработайте его.

Для корректной работы контролов:

- убедитесь, что пути до файлов в собранном компоненте формируются корректно. Шаблон пути указывается в файле public-path.js в формате:

  ```
  var src = new URL(document.currentScript.src);
  __webpack_public_path__ = src.origin + src.pathname + '/../';
  ```

- если планируется использовать несколько компонентов, убедитесь, что для них заданы разные публичные имена.

Добавление стороннего компонента в прикладную конфигурацию

Компоненты добавляются прикладную конфигурацию в узел **components**. Каждый компонент указывается в отдельном дочернем узле **component**.

Обязательный атрибут узла – **name**, в котором указывается имя компонента.

Структура узла:

```
<solution name="MySolution">
  <components>
    <component name="{Имя компонента}">
      <remoteEntry codeFile="{Путь до файла с конфигурацией компонента}" />
```

```
      <package>
        <entry file="{Файл компонента}" contentType="{Тип файла}" />
      </package>
    </component>
  </components>
  ...
</solution>
```

remoteEntry – конфигурация компонента. Задается в отдельном файле, путь к которому указывается в атрибуте **codeFile** относительно XML-файла решения.

package – дополнительные файлы компонента. Добавляются в дочерних узлах **entry**. Атрибуты узла **entry**:

- **file**. Путь до файла относительно XML-файла решения. В качестве имени файла можно указать маску;
- **contentType**. Тип ресурса.

Пример настройки:

```
<solution name="MySolution">
  <components>
    <component name="MyControl">
      <remoteEntry codeFile="component/dist/remoteEntry.js" />
      <package>
        <entry file="component/dist/chunks/*.js" contentType="text/javascript" />
      </package>
    </component>
  </components>
  ...
</solution>
```

Встраивание стороннего контрола

Контролы из добавленного компонента можно встроить в узлы с поддержкой Mustache-шаблонов:

- элемент формы услуги с типом **FacilityFormTextblock**;
- элемент диалога типа **TEXT**;
- виджет типа **templateWidget**;
- карточку объекта;
- плитку быстрого действия.

Для этого в Mustache-шаблон добавляется узел **component**. Атрибуты узла:

- [Обязательный] **name**. Имя компонента, указанное при его добавлении;
- [Обязательный] **public-name**. Публичное имя компонента. Указывается в файле webpack.config.js в свойстве **plugins[ModuleFederationPlugin].name**.

  ВАЖНО. В конфигурацию нельзя добавить несколько компонентов с одинаковыми публичными именами.
- [Обязательный] **module-name**. Имя модуля с загрузчиками. Указывается в файле webpack.config.js в свойстве **plugins[ModuleFederationPlugin].exposes**.

  ВАЖНО. В конфигурации компонента имя модуля с загрузчиками обязательно должно начинаться с символов **./**, например **./contoso-module**. В прикладной конфигурации можно указать имя модуля без этих символов.

- [Обязательный] **loader-name**. Имя загрузчика. Указывается ключ с нужным контролом из файла, указанного в модуле;
- **container-styles**. Стили для контейнера, в который встраивается контрол. Заполняется аналогично HTML-атрибуту **style**.

  СОВЕТ. Для настройки стилей можно использовать цветовую схему и тему, используемые на сайте личного кабинета. Их можно получить с помощью CSS-переменных, которые задаются в формате **--ess_<Параметр цветовой схемы>**. Подробнее см. в руководстве администратора HR Pro раздел «Настройка внешнего вида личного кабинета» (Linux, Windows).

При необходимости в узел можно добавить дополнительные атрибуты, которые необходимо передать в контрол.

ВАЖНО. Имена передаваемых атрибутов приводятся к нижнему регистру. Рекомендуется использовать формат kebab-case.

Пример встраивания контрола в форму услуги:

```
<facilityFormItem type="FacilityFormTextblock">
  <content>
    <![CDATA[
      <component
        name="MyControl"
        public-name="Contoso_MyControl_1_0"
        module-name="contoso-module"
        loader-name="contoso-loader"
        container-styles="background-color: var(--ess_Secondary50Color);"
      />
    ]]>
  </content>
</facilityFormItem>
```

Передаваемые данные

При загрузке стороннего контрола в него передаются данные:

- **initialContext** – контекст приложения:
  - **isMobile** – признак того, что используется мобильное представление;
  - **language** – локализация приложения;
  - **theme** – тема оформления личного кабинета;
  - **icons** – объект с SVG-иконками, использующимися в личном кабинете;
  - **classes** – объект с прикладными CSS-классами для типографики и классом для скелетона **skeleton**.
- **api** – API для взаимодействия с родительским элементом:
  - для формы услуги – пространство имен FormFunctions;
  - для диалога – API для работы с формой диалога;
  - для карточки объекта – пространство имен CardFunctions.
- **controlInfo** – контекст родительского элемента:
  - **context** – хранилище данных для прикладной разработки. Передается, если контрол встроен в форму услуги;

    - **data** – данные, полученные из источника данных родительского элемента. Передается, если контрол встроен в карточку объекта, виджет или плитку быстрого действия;
    - **configuration** – конфигурация. Передается, если контрол встроен в карточку объекта;
- **children** – дочерние элементы. Таким элементом может быть только текст;
- **onComponentUpdate** – обработчик события при обновлении контрола. Принимает в качестве аргумента функцию для отслеживания изменения входных свойств. Пример обработчика:

```
const [children, setChildren] = useState(props.children);

const handleComponentUpdate = useCallback(updateProps => {
  setChildren(updateProps.children);
}, []);

useEffect(() => {
  props.onComponentUpdate(handleComponentUpdate);
}, []);
```

valueConverter – прикладной конвертер

Узел содержит конфигурацию прикладного конвертера. Атрибуты узла:

- **id.** Идентификатор;
- [Обязательный] **name**. Имя прикладного конвертера;
- [Обязательный] **runat**. Среда выполнения кода. Возможные значения: **Server**, **Client**.

Структура узла:

```
<valueConverter name="{Имя}" runat="{Среда выполнения}" id="{Идентификатор}">
  <references>
    {Подключаемые библиотеки}
  </references>
...
</valueConverter>
```

Конфигурация включает в себя дочерний узел **references**, в котором могут содержаться узлы **reference**. Используется только для среды выполнения кода **Server** и варианта задания кода через класс или отдельный файл. Поддерживаются только системные библиотеки.

Атрибут узла **reference**: **include** – имя файла подключаемой библиотеки.

Пример конфигурации без задания кода конвертера:

```
<valueConverter name="codeConverter" runat="Server" id="2E3FF1C0-1A99-4F71-A2CF-81B0EBD7AF38">
  <references>
    <reference include="System.Drawing.Primitives.dll" />
  </references>
...
</valueConverter>
```

Варианты задания кода прикладного конвертера:

- функция конвертирования;
- класс конвертера;
- отдельный файл с классом конвертера.

Все классы серверных конвертеров (значение **runat="Server"**) пишутся на языке C# и реализуют интерфейс **IValueConverter** с методом:

```
object Convert(object value, Type targetType, object parameter, System.Globalization.CultureInfo culture)
```

Параметры:

- **value** – конвертируемое значение;
- **targetType** – тип, к которому надо преобразовать значение параметра value;
- **parameter** – вспомогательный параметр;
- **culture** – текущая культура приложения.

Метод должен возвращать результат конвертации значения.

Все классы клиентских конвертеров (значение runat="Client") пишутся на языке JavaScript и реализуют класс с методом convert с параметрами:

- **value** – конвертируемое значение;
- **parameter** – вспомогательный параметр;
- **language** – двухбуквенный код языка в нижнем регистре по стандарту ISO 639-1.

Метод должен возвращать результат конвертации значения.

В базовую поставку входят прикладные клиентские конвертеры:

- **EssBase.toLongMonthYear** – преобразует дату к формату <Январь 2020> с учетом локализации приложения;
- **EssBase.getDeadline** – возвращает mustache-шаблон для срока, используя свойства вспомогательного параметра **result**, **deadline** и **isExpired**.

Функция конвертирования

Ниже приведен пример конфигурации прикладного конвертера **functionConverter**. Код конвертера выполняется на сервере. Код конвертера указан в узле **function** на языке C#. Функция возвращает результат конвертации значения.

```
<valueConverter name="functionConverter" runat="Server">
<function>
<![CDATA[
return value?.ToString()?.ToLower();
]]>
</function>
</valueConverter>
```

Ниже приведен пример конфигурации прикладного конвертера **booleanToOnOffState**. Код конвертера выполняется на клиенте. Код конвертера указан в узле **function** на языке JavaScript. Функция возвращает результат конвертации значения.

```
<valueConverter name="booleanToOnOffState" runat="Client">
  <function>
    <![CDATA[
      return !!value ? 'On' : 'Off';
    ]]>
  </function>
</valueConverter>
```

Класс конвертирования

Ниже приведен пример конфигурации прикладного конвертера **codeConverter**. Код конвертера выполняется на сервере. Код конвертера указан в узле **code** на языке C#.

```
<valueConverter name="codeConverter" runat="Server">
  <code>
    <![CDATA[
      using System;
      using Directum.SelfServiceOffice.Domain.Core;

      public class WorkItemResultConverter : IValueConverter
      {
        public object Convert(object value, Type targetType, object parameter, CultureInfo
culture)
        {
          return value?.ToString()?.ToLower();
        }
      }
    ]]>
  </code>
</valueConverter>
```

Ниже приведен пример конфигурации прикладного конвертера **booleanToOnOffStateCode**. Код конвертера выполняется на клиенте. Код конвертера указан в узле **code** на языке JavaScript.

```
<valueConverter name="booleanToOnOffStateCode" runat="Client">
```

```
  <code>
    <![CDATA[
      class booleanToOnOffStateCode {
        convert(value, parameter, language) {
          return !!value ? 'On' : 'Off';
        }
      }
    ]]>
  </code>
</valueConverter>
```

Файл с описанием класса конвертера

Ниже приведен пример конфигурации прикладного конвертера **codeFileConverter**. Код конвертера выполняется на сервере. Код конвертера указан в файле CodeConverter.cs на языке C#. **codeFile** – путь к файлу с кодом прикладного серверного конвертера.

```
<valueConverter name="codeFileConverter" runat="Server" codeFile="CodeConverter.cs" />
```

Пример файла CodeConverter.cs.

```
using System;
using Directum.SelfServiceOffice.Domain.Core;

public class WorkItemResultConverter : IValueConverter
{
  public object Convert(object value, Type targetType, object parameter, CultureInfo
culture)
  {
    return value?.ToString()?.ToLower();
  }
}
```

Ниже приведен пример конфигурации прикладного клиентского конвертера **booleanToOnOffStateFile**. Код конвертера выполняется на клиенте. Код конвертера указан в файле ClientCodeConverter.js на языке JavaScript. **codeFile** – путь к файлу с кодом прикладного клиентского конвертера.

```
<valueConverter name="booleanToOnOffStateFile" runat="Client"
codeFile="ClientCodeConverter.js" />
```

Пример файла ClientCodeConverter.js:

```
class booleanToOnOffStateFile {
  convert(value, parameter, language) {
    return !!value ? 'On' : 'Off';
  }
}
```

Системные клиентские конвертеры

В личном кабинете доступны системные клиентские конвертеры:

- **getStatusDescription** – возвращает модель описания статуса по значению;
- **toLongMonthYear** – преобразует дату к формату **Январь 2020** с учетом локализации приложения. Рекомендуется использовать формат ISO. Подробнее см. на сайте developer.mozilla.org, статья Date.prototype.toISOString().

Базовые клиентские конвертеры

В базовом прикладном решении EssBase.xml доступны прикладные клиентские конвертеры:

- **EssBase.getDeadline** – возвращает mustache-шаблон для срока, используя свойства **result**, **deadline** и **isExpired** вспомогательного параметра.
- **EssBase.toLongMonthYear** – аналогичен системному конвертеру **toLongMonthYear**.
- **EssBase.simplifyDate** – преобразует дату к виду с упрощенным временем, указанным через запятую: **01.01.2020, 12:00**. Если часы и минуты равны 0, время не отображается. Рекомендуется использовать формат ISO. Подробнее см. на сайте developer.mozilla.org, статья Date.prototype.toISOString();
- **EssBase.toTaskScheme** – возвращает mustache-шаблон для стороннего контрола с маршрутом задачи, используя свойство **status** вспомогательного параметра. Если статус **Aborted** или **Completed**, то возвращает пустую строку.

script – прикладной сценарий

Узел содержит конфигурацию прикладного сценария. Атрибуты узла:

- **id** – идентификатор;
- [Обязательный] **name** – имя прикладного сценария;
- [Обязательный] **runat** – среда выполнения кода. Возможные значения: **Server**, **Client**.Пример конфигурации без задания кода сценария:

Структура узла:

```
<script name="{Имя}" runat="{Среда выполнения}" id="{Идентификатор}">
  <references>
    {Подключаемые библиотеки}
  </references>
...
</script>
```

references – список подключаемых библиотек, которые задаются в дочерних узлах **reference**. Используется только для среды выполнения кода **Server** и для варианта задания кода через класс или отдельный файл. Поддерживаются только системные библиотеки.

reference – подключаемая библиотека. В атрибуте узла **include** задается имя файла подключаемой библиотеки.

Пример настройки

```
<script name="AverageSequence" runat="Server" id="2E3FF1C0-1A99-4F71-A2CF-81B0EBD7AF38">
  <references>
    <reference include="System.Drawing.Primitives.dll" />
  </references>
...
</script>
```

Варианты задания кода прикладного сценария:

- функция;
- класс с кодом сценария;
- отдельный файл с классом сценария.

Все классы серверных сценариев (значение **runat="Server"**) пишутся на языке C# и реализуют интерфейс **IScript** с методом:

```
object Execute(params object[] arguments)
```

Параметр **arguments** – аргументы сценария. Метод возвращает результат выполнения сценария.

Все классы клиентских сценариев (значение **runat="Client"**) пишутся на языке JavaScript и реализуют класс с методом **execute** с параметром **args** – аргументы сценария. Метод должен возвращать объект **Promise** с результатом работы сценария. Подробнее см. на сайте developer.mozilla.org статью Promise.

Функция

Ниже приведен пример конфигурации прикладного сценария **CountItems**, который выполняется на сервере. Код функции сценария указан в узле **function** на языке C#. Функция сценария может вернуть некоторое значение.

```
<script name="CountItems" runat="Server">
  <function>
    <![CDATA[
      return arguments.Length;
    ]]>
  </function>
</script>
```

Ниже приведен пример конфигурации прикладного сценария **testClientScript**, который выполняется на клиенте. Код функции сценария указан в узле **function** на языке JavaScript. Функция сценария может вернуть некоторое значение.

```
<script name="testClientScript" runat="Client">
  <function>
    <![CDATA[
      return new Promise((resolve, reject) => {
        if (args["TestString"] === 'Test') {
          resolve('Yes');
        } else {
          resolve('No');
        }
      });
    ]]>
  </function>
</script>
```

Класс сценария

Ниже приведен пример конфигурации прикладного сценария AverageSequence, который выполняется на сервере. Код сценария указан в узле code на языке C#.

```
<script name="AverageSequence" runat="Server">
  <code>
    <![CDATA[
      using System;
      using System.Linq;
      using System.Globalization;
      using Directum.SelfServiceOffice.Domain.Core;
      using Directum.SelfServiceOffice.Domain.Core.Scripts;
```

```
      public class AverageIt : IScript
      {
        public object Execute(params object[] arguments)
        {
          return arguments.Average(v => (double)v);
        }
      }
    ]]>
  </code>
</script>
```

Ниже приведен пример конфигурации прикладного сценария **testClientScriptCode**, который выполняется на клиенте. Код сценария указан в узле **code** на языке JavaScript.

```
<script name="testClientScriptCode" runat="Client">
  <code>
    <![CDATA[
      class testClientScriptCode {
        execute(args) {
          return new Promise((resolve, reject) => {
            if (args["TestString"] === 'Test') {
              resolve('Yes');
            } else {
              resolve('No');
            }
          });
        }
      }
    ]]>
  </code>
</script>
```

Файл с описанием класса сценария

Ниже приведен пример конфигурации прикладного сценария **AverageSequence**, который выполняется на сервере. Код сценария указан в файле AverageSequence.cs на языке C#.

```
<script name="AverageSequence" runat="Server" codeFile="AverageSequence.cs" />
```

AverageSequence – путь к файлу с кодом прикладного конвертера.

Пример файла AverageSequence.cs:

```
using System;
using System.Linq;
using System.Globalization;
using Directum.SelfServiceOffice.Domain.Core;
using Directum.SelfServiceOffice.Domain.Core.Scripts;

public class AverageIt : IScript
{
  public object Execute(params object[] arguments)
  {
    return arguments.Average(v => (double)v);
  }
}
```

Ниже приведен пример конфигурации прикладного сценария **testClientScriptFile**. Код сценария выполняется на клиенте. Код сценария указан в файле ClientCodeScript.js на языке JavaScript.

```
<script name="testClientScriptFile" runat="Client" codeFile="ClientCodeScript.js" />
```

Пример файла ClientCodeScript.js:

```
class testClientScriptFile {
  execute(args) {
    return new Promise((resolve, reject) => {
      if (args["TestString"] === 'Test') {
        resolve('Yes');
      } else {
        resolve('No');
      }
    });
  }
}
```

role – роль

Узел содержит конфигурацию ролей. Атрибуты узла:

- [Обязательный] **name** – имя роли;
- **title** – заголовок роли;
- [Обязательный] **state** – состояние. Возможные значения: **Active**, **Closed**.

Структура узла:

```
<role name="{Имя}" title="{Заголовок}" state="{Состояние}">
  <condition>
    {Условие применения}
  </condition>
</role>
```

condition – условие для применения роли. Если узел пустой, то роль применяется ко всем пользователям. Условия записываются в виде выражения фильтрации, как в узле decisionStep. В узле можно использовать реквизиты удостоверений пользователя, полученные из сервиса идентификации.

Пример настройки

```
<role name="Administrator" title="Администратор" state="Active">
  <condition>DirectumRX.RoleId Contains "340103EA-A766-47A8-98AD-5260CA32DE46"</condition>
</role>
<role name="Developer" title="Разработчик" state="Active">
  <condition>DirectumRX.PersonId = 1480</condition>
</role>
```

Иконки решения

В прикладной конфигурации можно указывать имена иконок, которые будут отображаться в интерфейсе. Можно использовать:

- стандартные иконки решения, указанные в таблице;
- иконки из бандла, который загружается с адреса, указанного в секции **IconBundleUrl** конфигурационного файла сайта. Подробнее см. в руководстве администратора HR Pro, раздел «Изменение иконок».

 accountant	 application-settings	 applications	 arrow-left	 avatar-small
 avatar	 benefits	 book-open	 business-trip	 calendar
check-mark	 check	chevron-bottom	 chevron-left	 chevron-right
chevron-top	 clock	collapsed	 company	 contacts
 cross	 devtools	 diff	 discontinued	 documents
 download	 error	 exclamation-error	 exclamation-error-grey	 exit
 expand	 facilities	 file	 funnel	 gallery
 hamburger	 help-circle	 history	 home	 hr-pro
 in-progress	 info	kebab	kebab_vert	 key
 keylock	 mail	 memo	minus	 news

 notifications-new	 notifications	 paper	 paperclip	 pin-code
 plus	preview	 printer	 read-document	 refuse
 salary	 search	select-chevron	 services	 settings
 sick-leave	 sign	 statements	 suspended	 trash
 user-circle	 user	 user_large	 vacation-overlaps	 vacation
 wave				

Прикладные функции платформы личного кабинета

1. Для выполнения прикладной логики в диалогах доступны события.
2. В событиях используется язык JavaScript.
3. Во всех событиях доступны:
   - стандартное пространство имен **window**;
   - глобальная переменная **g_userProfile**, которая содержит информацию о профиле текущего пользователя;
   - глобальная переменная **g_userPerson** – содержит информацию о персоне текущего пользователя: ФИО, номер телефона, адрес электронной почты;
   - глобальная переменная **ESS** – пространство имен для работы с элементами страницы и данными личного кабинета. Также содержит вспомогательные функции;
   - прикладные клиентские конвертеры. Формат вызова:

```
converters.<Имя решения>.<Имя конвертера>(<Значение для конвертации>, <Вспомогательный параметр>)
```

     Подробнее см. в разделе «Узел valueConverter»;
   - прикладные клиентские сценарии. Формат вызова:

```
ESS.scripts.<Имя решения>.<Имя сценария>(<Аргументы>)
```

     Подробнее см. в разделе «Узел script».
4. На формах услуг также доступны:
   - список функций для работы с текущей формой;
   - хранилище данных для прикладной разработки. Является более приоритетным способ хранения необходимых данных, чем пространство имен **window**.
5. В событии после открытия карточки объекта также доступен список функций для работы с конфигурацией текущей карточки.
6. Для описания функций используется генератор jsDoc. Подробнее см. на сайте jsdoc.app.

Обработчики событий

Функция обработки событий вызывается, когда в указанном целевом объекте возникнет событие указанного типа.

Где используются	Обработчики событий
В офертах	openingEventHandler acceptedEventHandler refusedEventHandler
В услугах	Хранилище данных для прикладной разработки Обработчики событий формы: • openingEventHandler

Где используются	Обработчики событий
	• openedEventHandler • closingEventHandler • closedEventHandler Обработчики событий реквизитов: • valueChangedEventHandler • itemAddingEventHandler • itemAddedEventHandler • itemRemovingEventHandler • itemRemovedEventHandler • calculateFunction Обработчики событий вкладок: • navigatedFromEventHandler • navigatedToEventHandler Обработчики событий кнопок: • onClickEventHandler
В карточках объектов	formOpeningEventHandler formOpenedEventHandler formClosedEventHandler ClickEventHandler
В прикладных страницах	pageLoadedHandler
В виджетах	dataLoadedHandler

В офертах

openingEventHandler – обработчик события до открытия формы

Используется для актуализации данных перед показом формы. Является исполнителем Promise

Доступные параметры:

- **agreementData** – данные оферты;
- **resolve** – функция успешного выполнения. Аргументом необходимо передавать обновленные данные оферты в той структуре, в которой они пришли. Предварительно можно изменить значения свойств **agreementData**. Если аргумент не указан, то будут установлены изначальные данные. Вызов так же скрывает оверлей получения данных оферты. Пример вызова:

```
resolve(agreementData)
```

- **reject** – функция отклонения выполнения. Аргументом необходимо передавать объект с ошибкой в формате **{text: string, detail: string}**. Необходимо вызывать для показа страницы с ошибкой. Пример вызова:

```
reject({text: "error text", detail: "error detail"})
```

acceptedEventHandler – обработчик события после принятия оферты

Дополнительные параметры не передаются.

refusedEventHandler – обработчик события после отказа от принятия оферты

Дополнительные параметры не передаются.

Подробнее см. на сайте developer.mozilla.org, статья Promise.

В услугах

Хранилище данных для прикладной разработки

Предназначено для хранения любых данных, используемых в прикладной разработке. Создается в момент открытия формы перед вызовом openingEventHandler и удаляется после вызова closedEventHandler. Представляет собой мутируемый и уникальный для каждой формы объект. При изменении его содержимого новые данные можно использовать во всех событиях формы, а не только в том, где эти изменения совершены.

Пример использования:

```
</role>
<facilityFormItems>
  <facilityFormItem type="FacilityFormParameter">
    <parameterName>String</parameterName>
    <valueChangedEventHandler>
      <![CDATA[context.stringValueChanges = true;]]>
    </valueChangedEventHandler>
  </facilityFormItem>
</facilityFormItems>
<openingEventHandler>
  <![CDATA[
    context.stringValueChanges = false;
    context.customLog = value => console.log('From context', value);
    resolve();
  ]]>
</openingEventHandler>
<beforeSubmitEventHandler>
  <![CDATA[
    // Если был вызов события valueChangedEventHandler, то в консоли отобразится 'From
context, true', иначе 'From context, false'.
    context.customLog(context.stringValueChanges);
    resolve();
  ]]>
</beforeSubmitEventHandler>
```

Обработчики событий формы

openingEventHandler – обработчик события до открытия формы

Используется для актуализации данных перед показом формы. Является исполнителем Promise.

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;
- **resolve** – функция успешного выполнения. Аргумент игнорируется. Необходимо вызывать для обозначения окончания события и открытия формы;
- **reject** – функция отклонения выполнения. Аргумент игнорируется. Необходимо вызывать для отмены открытия формы.

ВАЖНО. В событии до открытия формы для параметров типа **FacilityParameterQuery** и **FacilityParameterReferenceRecord** устанавливается значение **null**, даже если для них есть значения по умолчанию. Попытка загрузки списка и установки значения по умолчанию выполняется только после завершения события до открытия формы.

openedEventHandler – обработчик события после открытия формы

Используется для работы с актуальными данными после вычислений на реквизитах. Подробнее см. в разделе «calculateFunction».

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки.

beforeValidationEventHandler – обработчик события перед началом валидации полей формы

Является исполнителем Promise.

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. раздел «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;
- **resolve** – функция успешного выполнения. Аргумент игнорируется. Необходимо вызывать для обозначения окончания события и начала валидации;
- **reject** – функция отклонения выполнения. Аргумент игнорируется. Необходимо вызвать для разблокировки формы и отмены платформенной валидации. Также предотвращает отправку формы и переход на следующую вкладку.

afterValidationEventHandler – обработчик события окончания валидации полей формы

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. раздел «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки.

beforeSubmitEventHandler – обработчик события перед отправкой формы

Является исполнителем Promise.

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. раздел «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;
- **resolve** – функция успешного выполнения. Аргумент игнорируется. Необходимо вызывать для обозначения окончания события и начала отправки формы;
- **reject** – функция отклонения выполнения. Аргумент игнорируется. Необходимо вызвать для разблокировки формы и отмены ее отправки.

afterSubmitEventHandler – обработчик события окончания отправки формы

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. раздел «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки.

closingEventHandler – обработчик события до закрытия формы

Используется для предотвращения закрытия формы. Является исполнителем Promise.

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;
- **resolve** – функция успешного выполнения. Аргумент игнорируется. Необходимо вызывать для обозначения окончания события и закрытия формы;
- **reject** – функция отклонения выполнения. Игнорируется.

closedEventHandler – обработчик события после закрытия формы

Используется для обработки результата выполнения услуги. Например, закрытие родительского элемента, обновление виджетов.

Доступные параметры:

- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;
- **sender** – данные вызывающего событие объекта. Содержит свойства:
    - **performingAction** – последнее выполняемое действие. Возможное значение **ESS.enums.FacilityAction.SUBMIT_FACILITY_PARAMETERS** устанавливается при нажатии на стандартную кнопку **Отправить**. Дополнительно может быть установлено при помощи функции addPerformingAction;

- **performingActions** – список всех выполняемых действий. Каждое действие указано в формате **{actionName: string, isApplied: boolean}**, где **isApplied** – признак того, что действие прикладное;
- **parent** – информация о родительском элементе. Используется для закрытия при помощи функции closeElement или для обновления при помощи функции updateElement. Может отсутствовать, если не был передан при вызове услуги.

Пример свойств параметра **sender**:

```
if (sender.performingAction === ESS.enums.FacilityAction.SUBMIT_FACILITY_PARAMETERS) {
  ESS.closeElement(sender.parent);
  ESS.widgets.updateWidgets();
}
```

Подробнее см. на сайте developer.mozilla.org, статья Promise.

Обработчики событий реквизитов

valueChangedEventHandler – обработчик события после изменения значения реквизита

Не вызывается при установке невалидного значения. Исключением являются реквизиты с датами, так как при невалидной дате происходит автоматическая очистка значения.

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;
- **formItem** – данные элемента. Свойства:
  - **key** – имя элемента формы или таблицы;
  - **name** – имя параметра формы или таблицы.
  - **value** – новое значение параметра;
  - **previousValue** – предыдущее значение параметра;
  - **index** – позиция значения в списке или индекс строки таблицы в формате GUID;
  - **columnName** – название колонки, используется для таблиц.

itemAddingEventHandler – обработчик события до добавления элемента в список или строку таблицы

Является исполнителем Promise.

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;

- **formItem** – данные элемента. Свойства:
  - **key** – имя элемента формы или таблицы;
  - **name** – имя параметра формы или таблицы;
- **resolve** – функция успешного выполнения. Аргумент игнорируется. Необходимо вызывать для добавления элемента в список или строку таблицы;
- **reject** – функция отклонения выполнения. Необходимо вызывать для предотвращения добавления элемента в список или строку таблицы. Аргументом можно передавать объект с ошибкой в формате **{message:** string**}** для логирования ошибки в консоль браузера. Пример вызова: **reject()** или **reject({message: "error text"})**.

itemAddedEventHandler – обработчик события после добавления элемента в список или строку таблицы

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;
- **formItem** – данные элемента. Свойства:
  - **key** – имя элемента формы или таблицы;
  - **name** – имя параметра формы или таблицы.

itemRemovingEventHandler – обработчик события до удаления элемента из списка или строки таблицы

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;
- **formItem** – данные элемента. Свойства:
  - **key** – имя элемента формы или таблицы;
  - **name** – имя параметра формы или таблицы;
  - **index** – позиция значения в списке или индекс строки таблицы в формате GUID;
- **resolve** – функция успешного выполнения. Аргумент игнорируется. Необходимо вызывать для добавления элемента в список или строку таблицы;
- **reject** – функция отклонения выполнения. Необходимо вызывать для предотвращения добавления элемента в список или строку таблицы. Аргументом можно передавать объект с ошибкой в формате **{message:** string**}** для логирования ошибки в консоль браузера. Пример вызова: **reject()** или **reject({message: "error text"})**.

itemRemovedEventHandler – обработчик события после удаления элемента из списка или строки таблицы

Доступные параметры:

- **functions** – прикладные функции. Подробнее см. в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;
- **formItem** – данные элемента. Свойства:
  - **key** – имя элемента формы или таблицы;
  - **name** – имя параметра формы или таблицы;
  - **index** – позиция значения в списке или индекс строки таблицы в формате GUID.

calculateFunction – выполнить вычисления для реквизита формы

Функция доступна для реквизитов. Является исполнителем Promise.

Доступные параметры:

- **value** – текущее значение реквизита;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;
- **resolve** – функция успешного выполнения. Аргументом необходимо передавать новое значение реквизита. Необходимо вызывать для обозначения окончания события и установки нового значения;
- **reject** – функция отклонения выполнения. Игнорируется.

Вызывается:

- для всех реквизитов:
  - после события **перед открытием формы** openingEventHandler, до события **после открытия формы** openedEventHandler;
  - при вызове функции updateValues;
- для одного реквизита при вызове функции executeItemCalculate.

Обработчики событий вкладок

navigatedFromEventHandler – обработчик события перехода с текущей вкладки

На время выполнения форма блокируется. Является исполнителем Promise.

Доступные параметры:

- **functions** – прикладные функции. Подробнее см.в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки;

- **resolve** – функция успешного выполнения. Аргумент игнорируется. Необходимо вызывать для обозначения окончания события и перехода на новую вкладку;
- **reject** – функция отклонения выполнения. Аргумент игнорируется. Необходимо вызывать для обозначения окончания события и снятия блокировки формы.

navigatedToEventHandler – обработчик события перехода на текущую вкладку

Доступные параметры:

- **functions** – прикладные функции. Подробнее см.в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки.

Обработчики событий кнопок

onClickEventHandler – обработчик события нажатия на кнопку

При обработке длительных или асинхронных операций, например при отправке запроса, рекомендуется блокировать кнопку перед вызовом и разблокировать после выполнения.

Доступные параметры:

- **functions** – прикладные функции. Подробнее см.в разделе «Пространство имен FormFunctions»;
- **context** – информация, указанная в объекте, передается в хранилище данных для прикладной разработки.

Подробнее см. на сайте developer.mozilla.org, статья Promise.

В карточках объектов

openingEventHandler – обработчик события до открытия карточки объекта

Используется для актуализации данных перед показом содержимого карточки. Является исполнителем Promise.

Доступные параметры:

- **data** – данные карточки;
- **resolve** – функция успешного выполнения. Аргументом необходимо передавать данные карточки в той структуре, в которой они пришли. Предварительно можно изменить значения свойств **data**. Аргумент обязателен. Необходимо вызывать для обозначения окончания события и обновления данных карточки. Пример вызова:

```
resolve(data)
```

- **reject** – функция отклонения выполнения. Аргументом необходимо передавать объект с ошибкой в формате **{message: string}**. Необходимо вызывать для логирования ошибки в консоль браузера и отмены обновления данных карточки. Пример вызова:

```
reject({message: "error text"})
```

- **sender** – описание вызывающего объекта в формате **{type: string, id: string}**.

Дополнительно в этом событии можно установить значение булевого свойства **data.showButtons**, отвечающего за необходимость отображения кнопок.

openedEventHandler – обработчик события после открытия карточки объекта

Используется для актуализации данных после показа карточки с использованием данных конфигурации карточки. Является исполнителем Promise.

Доступные параметры:

- **data** – данные карточки;
- **configuration** – конфигурация карточки;
- **functions** – прикладные функции. Подробнее см. в разделе «Пространство имен CardFunctions»;
- **resolve** – функция успешного выполнения. Аргументом необходимо передавать объект в формате **{configuration: object, data: object}**. Структуры свойств **configuration** и **data** должны соответствовать структурам параметров **configuration** и **data** обработчика события. Необходимо вызывать для обозначения окончания события и обновления данных и конфигурации карточки. Пример вызова:

```
resolve({configuration: configuration, data: data})
```

- **reject** – функция отклонения выполнения. Аргументом необходимо передавать объект с ошибкой в формате **{message: string}**. Необходимо вызывать для логирования ошибки в консоль браузера и отмены обновления данных и конфигурации карточки. Пример вызова:

```
reject({message: "error text"})
```

- **sender** – описание вызывающего объекта в формате **{type: string, id: string}**.

closedEventHandler – обработчик события после закрытия карточки объекта

Доступен параметр **data** – данные карточки.

clickEventHandler – обработчик события нажатия на кнопку

Доступные параметры:

- **data** – данные карточки;
- **sender** – описание вызывающего объекта в формате **{type: string, id: string}**. Используется в функции **ESS.actions.facilityActions.runFacility** для передачи описания родительского объекта в форму услуги, а также для закрытия при помощи функции closeElement или для обновления при помощи функции updateElement.

Пример:

```
ESS.actions.facilityActions.runFacility({id: 'FacilityId'}, {parameters: {ItemId: data.id, ItemName: data.name}}, sender);
```

Подробнее см. на сайте developer.mozilla.org, статья Promise.

В прикладных страницах

pageLoadedHandler – обработчик события открытия прикладной страницы

Используется для актуализации прикладной страницы перед отображением пользователю. Является исполнителем Promise.

Доступные параметры:

- **resolve** – функция успешного выполнения. Вызывается для обозначения окончания события. Пример вызова:

```
resolve()
```

- **reject** – функция отклонения выполнения. В качестве аргумента передается объект с ошибкой в формате **{message: string}**. Вызывается для отмены обновления статуса просмотра прикладной страницы и отображения страницы с ошибкой. Пример вызова:

```
reject({message: "error text"})
```

Подробнее см. на сайте developer.mozilla.org, статья Promise.

В виджетах

dataLoadedHandler – обработчик события загрузки данных таблицы

Используется для актуализации данных для отображения в таблице. Является исполнителем Promise.

Доступные параметры:

- **data** – данные таблицы;
- **resolve** – функция успешного выполнения. Аргументом необходимо передавать данные таблицы в той структуре, в которой они пришли. Предварительно можно изменить значения свойств **data**. Аргумент обязателен. Необходимо вызывать для обозначения окончания события и обновления данных таблицы.

Пример вызова:

```
resolve(data)
```

Пространство имен ESS

Пространство имен для работы с элементами страницы и данными личного кабинета. Также содержит вспомогательные функции, доступные в событиях.

СОВЕТ. Если текст необходимо отображать пользователю сайта личного кабинета, используйте для работы с ним функцию L. Например:

- в элементах диалога, созданного функцией createDialog (**ESS.dialogs.createDialog**);
- в функциях отображения toast-уведомлений showToast (**ESS.toasts.showToast**);

- при создании прикладной ошибки в событиях формы с помощью функции showError (**functions.showError**);
- при установке значения параметра формы функцией setValue, которое будет отображаться в mustache-шаблоне текстового блока формы (**functions.setValue**).

Элементы	Описание
actions	Действия
activities	Функции для работы с индикатором активности
cloneObject	Клонировать объект
closeElement	Закрыть элемент, используя соответствующую функцию
dialogs	Диалоги
downloadDocument	Скачать документ с сервера по его описанию
downloadFile	Скачать по ссылке файл с сервера
enums	Перечисления
formatString	Сформировать строку с описателями типа {0}
getColor	Получить цвета из палитры по токену
hints	Функции для работы с панелью подсказок на странице
L	Возвращает строку локализации из XML-файлов, расположенных в папке Localizations сайта.
network	Функции для работы с сетевыми запросами
overlay	Функции для работы с оверлеем
parseDate	Функции для получения даты из строки
parseFloat	Расширенная функция выделения числа с плавающей точкой
preview	Функции по работе с предпросмотром
replaceSubstrings	Заменить все вхождения указанной подстроки на новую подстроку
scripts	Прикладные сценарии
signing	Функции подписания
simplifyDate	Вернуть локализованную дату в виде строки
toasts	Функции для работы с toast-уведомлениями
widgets	Функции по работе с виджетами

actions – действия

chooseFacilityGroup – выполнить переход в указанную группу услуг

@param {string} [idName='''] – ид или имя группы.

@param {boolean} [clearWidgets=false] – признак необходимости очистки виджетов.

@param {boolean} [closeModal=true] – признак необходимости закрытия модальных окон.

Пример:

```
ESS.actions.chooseFacilityGroup(idName, clearWidgets, closeModal)
```

facilityActions – действия для работы с услугами

Пример:

```
ESS.actions.facilityActions
```

runFacility – запустить процесс получения услуги

@param {Object} actionParameters – параметры выполняемого действия. Свойства:

- **@param {string} actionParameters.id** – идентификатор или имя услуги;
- **@param {Object} [actionParameters.form]** – описание формы услуги для динамических параметров. Подробнее см. раздел «Тип виджета facilityShortcuts»;
- **@param {Array<Object>} [actionParameters.parameters]** – динамические параметры услуги. Подробнее см. раздел «Тип виджета facilityShortcuts».

@param {Object} [context={}] – данные контекста, из которого запущено действие. Свойство **@param {Object} context.parameters** – значения параметров из контекста в формате ключ/значение.

@param {Object} [parent] – родительский элемент, из которого запущено действие. Используется для закрытия или обновления по завершению процесса запуска услуги. Например, sender из событий карточки. Подробнее см. разделы «closeElement – закрывает элемент» и «updateElement – обновляет элемент».

Пример запуска услуги в действии карточки объекта:

```
const actionParameters = {
  id: 'Application',
  parameters: [
    {
      name: "ActiveText",
      type: "FacilityParameterString",
      defaultValue: null,
      isRequired: true,
      isCollection: false,
      properties: {
        Multiline: true,
      },
    },
  ],
  form: {
    items: [
      {
        type: "FacilityFormParameter",
        hint: "",
        properties: {
          ParameterName: "ActiveText",
          IsReadOnly: "False",
          IsHidden: "False",
          Label: "Текст заявления",
          ItemAddedHandler: "",
          ItemAddingHandler: "",
          ItemRemovedHandler: "",
          ItemRemovingHandler: "",
          ValueChangedHandler: "",
          CalculateFunction: "",
```

```
        },
      },
    ],
  },
};
const context = {
  parameters: {
    // data - данные карточки, доступные в действиях карточки.
    ItemId: data.id,
  },
};
ESS.actions.facilityActions.runFacility(actionParameters, context, sender);
```

submitFacilityParameters – запустить услугу с заданными параметрами

@param {Object} actionParameters – параметры выполняемого действия. Свойство **@param {string} actionParameters.facilityId** – идентификатор или имя выполняемой услуги.

@param {Object} [values={}] – значения параметров в формате ключ/значение.

@returns {Promise} – промис результата работы функции. При успешном выполнении операция завершается без возврата результата. При завершении операции с ошибкой возвращается текст ошибки.

Пример запуска услуги из действия карточки объекта:

```
const values = {
  // data - данные карточки, доступные в действиях карточки.
  ActiveText: data.ActiveText,
};
ESS.actions.facilityActions.submitFacilityParameters({ facilityId: 'Application' }, values)
  .then(() => ESS.closeElement(sender))
  .catch(error => ESS.toasts.showToast(error, ESS.enums.ToastType.ERROR));
```

workProcessActions

Действия для работы с рабочими процессами.

Пример:

```
ESS.actions.workProcessActions
```

changeWorkItemState – изменить состояние рабочего элемента

@param {Object} workItem – рабочий элемент. Свойства:

- **@param {string} workItem.system** – название системы;
- **@param {string} workItem.name** – имя рабочего элемента;
- **@param {string} workItem.id** – идентификатор рабочего элемента.

@param {string} state – новое состояние. Возможные значения см. раздел «WorkItemStatus».

@param {string} reason – причина изменения состояния. Возможные значения см. раздел «WorkItemChangingReason».

@returns {Promise} – промис запроса на изменение состояния элемента рабочего процесса без обработки.

Пример изменения состояния рабочего элемента из действия карточки объекта:

```
const workItem = {
  // data - данные карточки, доступные в действиях карточки.
  system: 'EssBase',
  name: data.WorkItemName,
  id: data.WorkItemId,
};
ESS.actions..workProcessActions.changeWorkItemState(
  workItem,
  ESS.enums.WorkItemStatus.COMPLETE,
  ESS.enums.WorkItemChangingReason.REFUSE,
)
  .then(() => ESS.closeElement(sender))
  .catch(error => ESS.toasts.showToast(ESS.network.getRequestErrorMessage(error),
ESS.enums.ToastType.ERROR));
```

completeWork – завершить элемент рабочего процесса

@param {Object} actionParameters – параметры выполняемого действия. Свойства:

- **@param {string} actionParameters.SystemName** – имя целевой системы;
- **@param {string} actionParameters.WorkItemName** – имя элемента рабочего процесса;
- **@param {string} actionParameters.WorkItemId** – идентификатор элемента рабочего процесса;
- **@param {string} actionParameters.State** – новое состояние элемента рабочего процесса. Возможные значения см. раздел «WorkItemStatus»;
- **@param {string} actionParameters.Reason** – причина изменения состояния элемента рабочего процесса. Возможные значения см. раздел «WorkItemChangingReason».

@returns {Promise} – промис завершения элемента рабочего процесса. При успешном выполнении операция завершается без возврата результата. При завершении операции с ошибкой возвращается текст ошибки.

Пример завершения элемента рабочего процесса из действия карточки объекта:

```
const actionParameters = {
  // data - данные карточки, доступные в действиях карточки.
  SystemName: 'EssBase',
  WorkItemName: data.WorkItemName,
  WorkItemId: data.WorkItemId,
  State: ESS.enums.WorkItemStatus.COMPLETE,
  Reason: ESS.enums.WorkItemChangingReason.REFUSE,
};
ESS.actions.workProcessActions.completeWork(actionParameters)
  .then(() => ESS.closeElement(sender))
  .catch(error => ESS.toasts.showToast(error, ESS.enums.ToastType.ERROR));
```

Подробнее см. на сайте developer.mozilla.org, статья Promise.

activities – функции для работы с индикатором активности

addActivity – добавить выполняемое действие в список активности

@returns {string} – идентификатор выполняемого действия.

Пример:

```
ESS.activities.addActivity()
```

removeActivity – удалить выполняемое действие из списка активности

@param {string} id – идентификатор выполняемого действия.

Пример:

```
ESS.activities.removeActivity(id)
```

cloneObject – клонирует объект

Функция используется для предотвращения мутирования исходного объекта.

@param {Object} [inObject] – исходный объект.

@returns {Object} – склонированный объект.

Пример:

```
ESS.cloneObject(inObject)
```

closeElement – закрывает элемент

При закрытии элемента используется соответствующая функция. Дополнительно см. раздел «updateElement – обновляет элемент».

@param {Element} element – элемент, который необходимо закрыть. Подробнее см. раздел «Element – элемент». Например, объект sender в событиях карточки или объект sender.parent в событии закрытия формы услуги.

Пример закрытия карточки по кнопке **Закрыть**:

```
ESS.closeElement(sender);
```

Пример закрытия карточки при закрытии формы услуги:

```
ESS.closeElement(sender.parent);
```

constants – системные константы

- **ALL** – все;
- **INVALID_DATE** – неверная дата;
- **INITIATOR** – инициатор.

Пример:

```
ESS.constants.ALL
```

dialogs – диалоги

closeDialog – закрыть диалог

Закрыть диалог созданный функцией createDialog.

@param {string} id – ИД закрываемого диалога.

@param {string} [actionName='''] – имя действия, по которому закрывается диалог. Передается в события **onClosing** и **onClosed**.

Пример:

```
ESS.dialogs.closeDialog(id, actionName)
```

createDialog – создать новый диалог через конструктор

@param {string} id – идентификатор диалога для закрытия при помощи функции closeDialog.

@param {string} title – заголовок.

@param {Array<TDialogItem>} items – элементы диалога.

@param {Array<TDialogButton>} buttons – кнопки диалога.

@param {TDialogProperties} [properties={}] – свойства диалога.

@param {TDialogHandlers} [handlers={}] – обработчики событий диалога.

Пример

```
const dialogId = 'new-dialog';
const inputFieldName = 'inputField';
const inputField2Name = 'inputField2';
const textFieldName = 'textItem';
const textField2Name = 'textItem2';

const textItem = {
  type: ESS.enums.DialogItemType.TEXT,
  name: textFieldName,
  properties: {
    text: 'test text',
  },
};

const textWithTemplateItem = {
  type: ESS.enums.DialogItemType.TEXT,
  name: textField2Name,
  properties: {
    text: `
      <p style="font-weight: 700;">
        Введено значение: "
        <span style="color: {{#getColor}}Primary50Color{{/getColor}};">
          {{${inputField2Name}}}
        </span>
        ".
      </p>
    `,
```

```
  },
};

const onInputChange = (inputName, value, dialogApi) => {
  console.log(`${inputName} change:`, value, dialogApi);
  dialogApi.items.setProperty({
    itemName: textFieldName,
    propertyName: ESS.enums.DialogItemProperties[ESS.enums.DialogItemType.TEXT].TEXT,
    value: `Введено значение "${value}"`,
  });
};

const onInputBlur = (inputName, value, dialogApi) => {
  console.log(`${inputName} blur:`, value, dialogApi);
  if (!window[dialogId][inputName]) {
    window[dialogId][inputName] = dialogApi.errors.add(`${inputName} error`);
  }
};

const items = [
  textItem,
  {
    type: ESS.enums.DialogItemType.INPUT,
    name: inputFieldName,
    properties: {
      title: 'input title',
      initialValue: '123',
      onChange: (value, dialogApi) => onInputChange(inputFieldName, value, dialogApi),
      onBlur: (value, dialogApi) => onInputBlur(inputFieldName, value, dialogApi),
      hint: `Подсказка для ${inputFieldName}`,
    },
  },
  {
    type: ESS.enums.DialogItemType.BUTTON,
      name: 'button-item',
      properties: {
          title: `Установить значение в ${inputFieldName}`,
          hint: 'Подсказка для кнопки',
          icon: 'check-mark',
          onClick: dialogApi => dialogApi.items.setValue(inputFieldName, 'Значение из
кнопки'),
    },
  },
  {
    type: ESS.enums.DialogItemType.CHECKBOX,
    name: 'checkbox',
    properties: {
      title: 'checkbox title',
      onChange: (value, dialogApi) => {
        if (value) {
          dialogApi.items.setDisabled('referenceField');
        } else {
          dialogApi.items.setEnabled(ESS.constants.ALL);
        }
      },
      hint: 'Подсказка для checkbox',
    },
  },
  {
    type: ESS.enums.DialogItemType.CHECKBOX,
    name: 'checkbox_init',
    properties: {
      title: 'checkbox init title',
      initialValue: true,
    },
  },
```

```
  textWithTemplateItem,
  {
    type: ESS.enums.DialogItemType.INPUT,
    name: inputField2Name,
    properties: {
      title: 'input 2 title',
      initialValue: '123',
      onBlur: (value, dialogApi) => onInputBlur(inputField2Name, value, dialogApi),
      hint: `Подсказка для ${inputField2Name}`,
    },
  },
  {
    type: ESS.enums.DialogItemType.FILE_INPUT,
    name: 'inputFile',
    isRequired: true,
    properties: {
      title: 'inputFile title',
      multiple: true,
      fileTypes: [
        '.jpg',
        '.png',
      ],
      maxFileCount: 2,
      maxFileSize: 1200,
      onChange: (value, dialogApi) => {
        console.log('inputFile change:', value, dialogApi);
      },
      hint: 'Подсказка для inputFile',
    },
  },
  {
    type: ESS.enums.DialogItemType.SELECT,
    name: 'selectField',
    properties: {
      title: 'select title',
      onChange: (value, dialogApi) => console.log('dropdown change:', value,
dialogApi),
      options: [
        { caption: 'test1', value: '1' },
        { caption: 'test2', value: '2' },
        { caption: 'test3', value: '3' },
        { caption: 'test4', value: '4' },
        { caption: 'test6', value: '5' },
        { caption: 'test7', value: '6' },
        { caption: 'test8', value: '7' },
      ],
      initialValue: '1',
      hint: 'Подсказка для selectField',
    },
  },
  {
    type: ESS.enums.DialogItemType.KEY_VALUE_LIST,
    name: 'kvlist',
    properties: {
      items: [
        { key: 'Издатель:', value: 'Crypto.pro' },
        { key: 'Действителен до:', value: '2023-01-19' },
      ],
    },
  },
  {
    type: ESS.enums.DialogItemType.SELECT_WITH_INFO_BLOCK,
    name: 'certField',
    properties: {
      title: 'select with info block',
      valueField: 'ObjId',
      captionField: 'Subject',
```

```
      infoKeyValues: {
        Issuer: 'Издатель:',
        ValidToDate: 'Действителен до:',
      },
      options: [
        {
          Issuer: 'crypto.pro',
          Subject: 'cert1',
          ValidToDate: '2023-01-19',
          ObjId: '1',
        },
        {
          Issuer: 'crypto.pro',
          Subject: 'cert2',
          ValidToDate: '2022-01-29',
          ObjId: '2',
        },
      ],
      initialValue: '2',
      onChange: (value, dialogApi) => console.log('combined select change:', value,
dialogApi),
      hint: 'Подсказка для certField',
    },
  },
  {
    type: ESS.enums.DialogItemType.RADIO_GROUP,
    name: 'radio',
    properties: {
      title: 'Сколько будет 2x2?',
      options: [
        {
          label: '3',
          value: '3',
        },
        {
          label: '4',
          value: '4',
        },
        {
          label: '5',
          value: '5',
        },
      ],
      initialValue: '4',
      hint: 'Подсказка для radio',
    },
  },
  {
    type: ESS.enums.DialogItemType.DATE,
    name: 'date',
    properties: {
      title: 'Date input',
      initialValue: '2023-01-01',
      hint: 'Подсказка для date',
    },
  },
  {
    type: ESS.enums.DialogItemType.DATE_INTERVAL,
    name: 'date_interval',
    properties: {
      initialValue: {
        begin: '2023-01-01',
        end: '2023-12-31',
      },
      onChange: (value, dialogApi) => {
        console.log(value, dialogApi);
      },
```

```
      hint: 'Подсказка для date_interval',
    },
  },
  {
    type: ESS.enums.DialogItemType.INPUT,
    name: 'blocked',
    properties: {
      title: 'blocked',
      initialValue: '2',
      isReadOnly: true,
      hint: 'Подсказка для blocked',
    },
  },
  {
    type: ESS.enums.DialogItemType.NUMBER_INPUT,
    name: 'blockedNumber',
    properties: {
      title: 'blocked number',
      initialValue: 2,
      isReadOnly: true,
      hint: 'Подсказка для blockedNumber',
    },
  },
  {
    type: ESS.enums.DialogItemType.NUMBER_INPUT,
    name: 'number',
    validators: [
      value => {
        if (!value) return;
        const numberValue = Number(value);
        if (numberValue < 0 || !Number.isInteger(numberValue)) {
          return {
            text: ESS.L('FORM.DATETIMEINTERVAL_DURATION_INVALID'),
            type: ESS.enums.DialogErrorType.FORM_VALIDATION
          };
        }
      }
    ],
    properties: {
      title: 'number',
      initialValue: 2,
      hint: 'Подсказка для number',
    },
  },
  {
    type: ESS.enums.DialogItemType.QUERY_SELECT,
    name: 'queryField',
    properties: {
      title: 'query title',
      onChange: (value, dialogApi) => console.log('queryField change:', value,
dialogApi),
      initialValue: '2791',
      dataSource: 'https://jsonplaceholder.typicode.com/users',
      dataSourceCaptionField: 'name',
      dataSourceValueField: 'id',
      hint: 'Подсказка для queryField',
    },
  },
  {
    type: ESS.enums.DialogItemType.REFERENCE_SELECT,
    name: 'referenceField',
    properties: {
      title: 'reference title',
      onChange: (value, dialogApi) => console.log('referenceField change:', value,
dialogApi),
      isUserQuerySupported: true,
      displayRequisite: 'Name',
```

```
      storeRequisite: 'Id',
      systemName: 'EssBase',
      referenceName: 'IEmployees',
      initialValue: '103',
      hint: 'Подсказка для referenceField',
    },
  },
  {
    type: ESS.enums.DialogItemType.COUNTDOWN,
    name: 'countdown',
    properties: {
      countdownText: ESS.L('@DEVELOPMENT.COUNTDOWN'),
      countdownInitValue: 63,
      refreshButtonCaption: ESS.L('@DEVELOPMENT.RESTART'),
      onRefreshButtonClick: refreshCallback => refreshCallback(),
      isReadOnly: false,
    },
  }
];

const submitButton = {
  name: 'submit-button',
  type: 'submit',
  caption: 'Submit button',
  icon: 'check-mark',
  onClick: (dialogApi, resolve, reject) => {
    console.log('submit:', dialogApi);
    resolve();
  },
  isPrimary: true,
};

const buttons = [
  submitButton,
  {
    name: 'remove-error',
    caption: 'Remove form error',
    icon: 'cross',
    onClick: dialogApi => {
      Object.values(window[dialogId]).forEach(value => {
        dialogApi.errors.remove(value);
      });
      window[dialogId] = {};
    },
  },
];

ESS.dialogs.createDialog({
  id: dialogId,
  title: 'test',
  items,
  buttons,
  properties: {
    height: '576px',
  },
  handlers: {
    onFormChange: dialogApi => {
      const error = Object.values(dialogApi.errors.getAll());
      if (!!error.length) {
        dialogApi.buttons.setDisabled('submit-button');
      } else {
        dialogApi.buttons.setEnabled('submit-button');
      }
      console.log('formState change handler', dialogApi);
      console.log(error.length ? `common error found: ${error}` : 'no common error');
    },
```

```
    onClosed: actionName => console.log('closed', actionName),
    onClosing: (closingResolve, closingReject, dialogApi, actionName) => {
      console.log('before close', dialogApi, actionName);
      delete window[dialogId];
      closingResolve();
    },
    onOpened: dialogApi => console.log('opened', dialogApi),
    onOpening: (openingResolve, openingReject) => {
      console.log('before open');
      window[dialogId] = {};
      openingResolve();
    },
  }
});
```

showConfirmationDialog – отобразить диалог подтверждения

@param {TSystemDialog} params – параметр функции.

Пример:

```
const params = {
  title: 'Подтвердите действие',
  text: 'Для подтверждения действия нажмите на кнопку OK',
  buttonsType: ESS.enums.ButtonsType.OK_CANCEL,
  onCloseCallback: button => {
    if (button === ESS.enums.ConfirmationDialogButtonName.OK) {
      ESS.toasts.showToast('Вы подтвердили действие');
    } else {
      ESS.toasts.showToast('Вы отменили действие');
    }
  },
  dialogSettings: {
    small: true,
    height: 'auto',
  }
};
ESS.dialogs.showConfirmationDialog(params);
```

showConfirmationDialogWithCode – отобразить диалог подтверждения действия вторым фактором

@param {Object} [callbacks={}] – обработчики нажатия кнопки. Свойства:

- **@param {function} [callbacks.confirmationCallback]** – callback подтверждения действия. Параметры:
  - **@param {string} value** – значение кода;
  - **@param {Function} resolve** – функция подтверждения. Необходимо вызывать при удачном завершении операции;
  - **@param {Function} reject** – функция отмены. Необходимо вызывать, если во время выполнения подтверждения произошла ошибка. В качестве параметра принимает объект с ошибкой;
- **@param {function} [callbacks.codeResendingCallback]** – callback повторной отправки кода. Параметры:
  - **@param {Function} resolve** – функция подтверждения успешной отправки. Необходимо вызывать при удачном завершении операции;

    - **@param {Function} reject** – функция отмены повторной отправки. Необходимо вызывать, если во время повторной отправки произошла ошибка. В качестве параметра принимает объект с ошибкой;
- **@param {function} [callbacks.cancelationCallback]** – callback отмены выполнения подтверждения.

@param {Object} [options={}] – конфигурация элементов диалога. Свойства:

- **@param {string} [options.title]** – заголовок диалога;
- **@param {string} [options.inputCaption]** – заголовок поля ввода кода;
- **@param {string} [options.resendingButtonCaption]** – текст кнопки повторной отправки кода;
- **@param {string} [options.confirmationButtonCaption]** – заголовок кнопки подтверждения;
- **@param {string} [options.cancelButtonCaption]** – заголовок кнопки закрытия диалога;
- **@param {string} [options.text]** – текст с уточняющей информацией;
- **@param {Array<TDialogItem>} [options.items]** – элементы диалога. Добавляются между текстом с уточняющей информацией и полем для ввода кода;
- **@param {string} [options.countdownText]** – текст при обратном отсчете. Необходимо, чтобы в тексте был описатель в формате **{0}** для подстановки в таймер. При подстановке учитывается локализация;
- **@param {number} [options.countdownInitValue]** – начальное значение обратного отсчета в секундах. Если не указано, используется значение из переменной **g_RefreshCodeInterval**. Минимальное значение – **60**. Если значение меньше 60, используется значение 60.

@returns – идентификатор диалога. Используется для закрытия при помощи функции closeDialog.

Пример:

```
const callbacks = {
  confirmationCallback: (value, resolve, reject) => {
    console.log('confirmationCallback', value);
    setTimeout(() => {
      if (value > 100) {
        resolve();
      } else {
        reject(new Error('Значение должно быть больше 100'));
      }
    }, 3000);
  },
  codeResendingCallback: (resolve, reject) => {
    console.log('codeResendingCallback');
    setTimeout(() => reject({ message: 'test' }), 3000);
  },
  cancelationCallback: () => {
    console.log('cancelationCallback');
  },
};
const options = {
  title: 'Подтвердите действие',
  inputCaption: ESS.L('DEFAULT.ATTENTION'),
  resendingButtonCaption: 'Выслать код заново на почту',
  confirmationButtonCaption: 'Кнопка отправки кода',
  cancelButtonCaption: 'Отменить операцию',
```

```
    text: 'Мы отправили вам сообщение с кодом на телефон. Введите полученный код.',
    countdownText: 'Запросить код повторно можно будет через {0}',
    countdownInitValue: 65,
  };
  ESS.dialogs.showConfirmationDialogWithCode(callbacks, options);
```

showMessageBox – отобразить диалог с сообщением

@param {string} title – заголовок диалога.

@param {string} text – текст сообщения.

Пример:

```
ESS.dialogs.showMessageBox(title, text)
```

downloadDocument – скачивает документ с сервера по его описанию

@param {Object} document – описание документа. Свойства:

- **@param {string} [document.binaryDataId]** – идентификатор документа в хранилище. Если он указан, скачивание происходит напрямую из сервиса хранилищ, иначе – из целевой системы;
- **@param {string} [document.id]** – идентификатор документа в целевой системе;
- **@param {string} document.name** – наименование документа.

  Если указано свойство **binaryDataId**, используется как имя файла. Иначе – используется как наименование типа документа в целевой системе. Имя файла будет взято из целевой системы.

@param {string} [system] – имя целевой системы. Используется, если задано свойство **document.binaryDataId**.

Примеры:

```
const document = {
  binaryDataId: '623a645f-dac6-4992-a970-6aa35ef3bd24',
  name: 'Заявление_об_увольнении_Снигирев_С._А._от_03.11.2022.pdf',
};
downloadDocument(document);
```

```
const document = {
  id: '413',
  name: 'IElectronicDocuments',
};
downloadDocument(document, 'DirectumRX');
```

downloadFile – скачивает по ссылке файл с сервера

@param {string} url – ссылка на файл.

@param {boolean} [fromStorageService=false] – признак того, что файл скачивается через сервис хранилищ.

@param {boolean} [encoded=false] – признак того, что ссылка уже экранирована. В этом случае ссылка оборачивается в функцию encodeURI. Подробнее см. на сайте developer.mozilla.org, статья «encodeURI()».

Пример:

```
ESS.downloadFile(url, fromStorageService, encoded)
```

enums – объект, который содержит перечисления или дочерние объекты с перечислениями

ButtonsType – набор кнопок для диалога подтверждения

- **OK_CANCEL** – кнопки **OK** и **Отмена**;
- **YES_NO** – кнопки **Да** и **Нет**;
- **YES_NO_CANCEL** – кнопки **Да**, **Нет** и **Отмена**;
- **RETRY_CANCEL** – кнопки **Повторить** и **Отмена**;
- **ABORT_RETRY_IGNORE** – кнопки **Прервать**, **Повторить** и **Пропустить**.

Пример:

```
ESS.enums.ButtonsType.OK_CANCEL
```

Colors – элементы цветов из палитры

- **Primary20Color** – цвет Primary20Color из конфигурации сайта;
- **Primary30Color** – цвет Primary30Color из конфигурации сайта;
- **Primary40Color** – цвет Primary40Color из конфигурации сайта;
- **Primary50Color** – цвет Primary50Color из конфигурации сайта;
- **Primary60Color** – цвет Primary60Color из конфигурации сайта;
- **Primary70Color** – цвет Primary70Color из конфигурации сайта;
- **Secondary20Color** – цвет Secondary20Color из конфигурации сайта;
- **Secondary50Color** – цвет Secondary50Color из конфигурации сайта;
- **Secondary70Color** – цвет Secondary70Color из конфигурации сайта;
- **Success20Color** – цвет Success20Color из конфигурации сайта;
- **Success30Color** – цвет Success30Color из конфигурации сайта;
- **Success40Color** – цвет Success40Color из конфигурации сайта;
- **Success50Color** – цвет Success50Color из конфигурации сайта;

- **Success70Color** – цвет Success70Color из конфигурации сайта;
- **Info20Color** – цвет Info20Color из конфигурации сайта;
- **Info30Color** – цвет Info30Color из конфигурации сайта;
- **Info40Color** – цвет Info40Color из конфигурации сайта;
- **Info50Color** – цвет Info50Color из конфигурации сайта;
- **Warning20Color** – цвет Warning20Color из конфигурации сайта;
- **Warning30Color** – цвет Warning30Color из конфигурации сайта;
- **Warning40Color** – цвет Warning40Color из конфигурации сайта;
- **Warning50Color** – цвет Warning50Color из конфигурации сайта;
- **Danger20Color** – цвет Danger20Color из конфигурации сайта;
- **Danger30Color** – цвет Danger30Color из конфигурации сайта;
- **Danger40Color** – цвет Danger40Color из конфигурации сайта;
- **Danger50Color** – цвет Danger50Color из конфигурации сайта;
- **Surface50Color** – цвет Surface50Color из конфигурации сайта;
- **Surface60Color** – цвет Surface60Color из конфигурации сайта;
- **Surface70Color** – цвет Surface70Color из конфигурации сайта;
- **SurfaceContainer50Color** – цвет SurfaceContainer50Color из конфигурации сайта;
- **SurfaceContainer60Color** – цвет SurfaceContainer60Color из конфигурации сайта;
- **SurfaceContainer70Color** – цвет SurfaceContainer70Color из конфигурации сайта;
- **SurfaceContainer80Color** – цвет SurfaceContainer80Color из конфигурации сайта;
- **Supporting20Color** – цвет Supporting20Color из конфигурации сайта;
- **Supporting50Color** – цвет Supporting50Color из конфигурации сайта;
- **TextPrimaryColor** – цвет TextPrimaryColor из конфигурации сайта;
- **TextSecondary30Color** – цвет TextSecondary30Color из конфигурации сайта;
- **TextSecondary50Color** – цвет TextSecondary50Color из конфигурации сайта;
- **ShadowColor** – цвет ShadowColor из конфигурации сайта;
- **ScrollColor** – цвет ScrollColor из конфигурации сайта;
- **FocusVisibleColor** – цвет FocusVisibleColor из конфигурации сайта.

Пример:

```
ESS.enums.Colors.Primary20Color
```

Подробнее см. в руководстве администратора HR Pro, раздел «Изменение цветов»

ConfirmationDialogButtonName – названия кнопок системного диалога

- **OK** – кнопка **OK**;
- **CANCEL** – кнопка **Отмена**;
- **YES** – кнопка **Да**;
- **NO** – кнопка **Нет**;
- **ABORT** – кнопка **Отмена**;

- **RETRY** – кнопка **Повторить**;
- **IGNORE** – кнопка **Пропустить**.

Пример:

```
ESS.enums.ConfirmationDialogButtonName.OK
```

DefaultIcons – системные иконки для сообщений

- **ERROR** – иконка ошибки;
- **INFO** – иконка информационного сообщений;
- **WARNING** – иконка предупреждения.

Пример:

```
ESS.enums.ButtonsType.OK_CANCEL
```

DialogErrorType – типы ошибок

Перечислены в порядке уменьшения приоритета:

- **SYSTEM** – системная ошибка;
- **APPLICATION** – прикладная ошибка;
- **FORM_VALIDATION** – ошибка валидации;
- **FORM_REQUIRED** – ошибка заполнения;
- **FORM_SUBMITTED** – ошибка после отправки;
- **UNSPECIFIED** – прочие ошибки.

Перечисление используется для диалогов.

Пример:

```
ESS.enums.DialogErrorType.SYSTEM
```

DialogItemProperties – объект, содержащий перечисления с именами свойств элементов диалога, которые доступны для изменения

Имена перечислений соответствуют значениям DialogItemType:

Перечисления	Доступные свойства элемента
[DialogItemType.BUTTON]	**TITLE** – заголовок **ICON** – иконка **HINT** – подсказка Пример: `ESS.enums.DialogItemProperties[DialogItemType.BUTTON].TITLE`
[DialogItemType.CHECKBOX]	**TITLE** – заголовок **HINT** – подсказка

Перечисления	Доступные свойства элемента
	Пример: `ESS.enums.DialogItemProperties[ESS.enums.DialogItemType.CHECKBOX].TITLE`
[DialogItemType.CODE_INPUT]	**TITLE** – заголовок **MAX_LENGTH** – максимальная длина значения **HINT** – подсказка Пример: `ESS.enums.DialogItemProperties[DialogItemType.CODE_INPUT].TITLE`
[DialogItemType.COUNTDOWN]	**TITLE** – заголовок Пример: `ESS.enums.DialogItemProperties[DialogItemType.COUNTDOWN].TITLE`
[DialogItemType.DATE_INTERVAL]	**TITLE** – заголовок **MIN_DATE** – минимальная дата **MAX_DATE** – максимальная дата **MAX_LENGTH** – максимальное количество дней между датами **HINT** – подсказка Пример: `ESS.enums.DialogItemProperties[DialogItemType.DATE_INTERVAL].TITLE`
[DialogItemType.DATE]	**TITLE** – заголовок **MIN_DATE** – минимальная дата **MAX_DATE** – максимальная дата **HINT** – подсказка Пример: `ESS.enums.DialogItemProperties[DialogItemType.DATE].TITLE`
[DialogItemType.FILE_INPUT]	**TITLE** – заголовок **FILE_TYPE** – список допустимых расширений файлов **MAX_FILE_COUNT** – максимальное количество файлов **MAX_FILE_SIZE** – максимальный размер файла в байтах **HINT** – подсказка Пример: `ESS.enums.DialogItemProperties[DialogItemType.FILE_INPUT].TITLE`
[DialogItemType.IMAGE]	**SRC** – адрес изображения **ALT** – альтернативное текстовое описание изображения **WITH_PREVIEW** – признак, который определяет, можно ли открыть изображение в режиме предпросмотра

Перечисления	Доступные свойства элемента
	MAX_WIDTH – максимальная ширина изображения **MIN_WIDTH** – минимальная ширина изображения Пример: `ESS.enums.DialogItemProperties[DialogItemType.IMAGE].SRC`
[DialogItemType.INPUT]	**TITLE** – заголовок **MAX_LENGTH** – максимальная длина значения **HINT** – подсказка Пример: `ESS.enums.DialogItemProperties[DialogItemType.INPUT].TITLE`

[DialogItemType.KEY_VALUE_LIST]	**ITEMS** – список элементов для отображения в формате:
	{ key: string, value: string }
	Пример:
	`ESS`
	`.enums`
	`.DialogItemProperties[DialogItemType.KEY_VALUE_LIST].ITEMS`

[DialogItemType.MULTILINE_LINK]	**TEXT** – текст ссылки
	HREF – адрес ссылки
	LINES – количество строк в которые отображается ссылка
	IS_BLANK – признак, что страницу нужно загружать в новом окне браузера
	Пример:
	`ESS.enums.DialogItemProperties[DialogItemType.MULTILINE_LINK]`
	`.TEXT`

[DialogItemType.NUMBER_INPUT]	**TITLE** – заголовок
	IS_INTEGER – признак того, что число должно быть целочисленным
	PRECISION – точность значения
	MIN_VALUE – минимально допустимое значение
	MAX_VALUE – максимально допустимое значение
	HINT – подсказка
	Пример:
	`ESS`
	`.enums`
	`.DialogItemProperties[DialogItemType.NUMBER_INPUT].TITLE`

[DialogItemType.PREVIEW_LINK]	**TITLE** – заголовок ссылки
	Пример:
	`ESS`
	`.enums`
	`.DialogItemProperties[DialogItemType.PREVIEW_LINK].TITLE`

[DialogItemType.QUERY_SELECT]	**TITLE** – заголовок **HINT** – подсказка Пример:

Перечисления	Доступные свойства элемента
	ESS .enums .DialogItemProperties[DialogItemType.QUERY_SELECT].TITLE
[DialogItemType.RADIO_GROUP]	**TITLE** – заголовок **OPTIONS** – массив объектов с вариантами выбора в формате: **{ label: string, value: string }** **HINT** – подсказка Пример: ESS .enums.DialogItemProperties[DialogItemType.RADIO_GROUP].TITLE
[DialogItemType.REFERENCE_SELECT]	**TITLE** – заголовок **HINT** – подсказка Пример: ESS .enums .DialogItemProperties[DialogItemType.REFERENCE_SELECT].TITLE
[DialogItemType.SELECT_WITH_INFO_BLOCK]	**TITLE** – заголовок **INFO_KEY_VALUES** – объект с элементами для отображения в формате ключ-значение **OPTIONS** – массив объектов с вариантами в формате ключ-значение **HINT** – подсказка Пример: ESS .enums .DialogItemProperties [DialogItemType.SELECT_WITH_INFO_BLOCK].TITLE
[DialogItemType.SELECT]	**TITLE** – заголовок **OPTIONS** – массив объектов с вариантами выбора в формате: **{ caption: string, value: string }** **HINT** – подсказка Пример: ESS.enums.DialogItemProperties[DialogItemType.SELECT].TITLE
[DialogItemType.TEXT_AREA]	**TITLE** – заголовок **ROWS** – максимальное количество строк, по умолчанию равно 3 **MAX_LENGTH** – максимальная длина текста **HINT** – подсказка Пример: ESS .enums.DialogItemProperties[DialogItemType.TEXT_AREA].TITLE
[DialogItemType.TEXT]	**TEXT** – текст для отображения Пример: ESS.enums.DialogItemProperties[DialogItemType.TEXT].TEXT

DialogItemType – типы элемента диалога

- **BUTTON** – кнопка;
- **CHECKBOX** – флажок;
- **CODE_INPUT** – поле для ввода кода. Позволяет вводить только цифры;
- **COUNTDOWN** – обратный отсчет;
- **DATE_INTERVAL** – выбор интервала дат;
- **DATE** – выбор даты;
- **FILE_INPUT** – выбор файла;
- **IMAGE** – изображение;
- **INPUT** – поле ввода;
- **KEY_VALUE_LIST** – список пар ключ-значение;
- **MULTILINE_LINK** – ссылка с поддержкой многострочного текста;
- **NUMBER_INPUT** – поле для ввода числового значения;
- **PREVIEW_LINK** – ссылка для предпросмотра;
- **QUERY_SELECT** – выпадающий список с запросом;
- **RADIO_GROUP** – группа кнопок-переключателей;
- **REFERENCE_SELECT** – выпадающий список с выбором из справочника;
- **SELECT_WITH_INFO_BLOCK** – выпадающий список с информационным блоком;
- **SELECT** – выпадающий список;
- **TEXT_AREA** – многострочное поле ввода;
- **TEXT** – текстовое поле с поддержкой mustache-шаблона.

Пример:

```
ESS.enums.DialogItemType.INPUT
```

DialogWidth – ширина диалога

- **DEFAULT** – размер диалога по умолчанию;
- **WIDE** – расширенный диалог;
- **LAYOUT** – размер диалога стремится к ширине содержимого сайта.

Пример:

```
ESS.enums.DialogWidth.DEFAULT
```

FacilityAction – действия для работы с услугой

SUBMIT_FACILITY_PARAMETERS – отправка запроса услуги.

Пример:

```
ESS.enums.FacilityAction.SUBMIT_FACILITY_PARAMETERS
```

FormAction – действия для работы с формой

- **SUBMIT** – действие отправки формы;
- **VALIDATION** – действие валидации полей;
- **NAVIGATED_FROM_TAB** – действие при перемещении по вкладкам.

Пример:

```
ESS.enums.FormAction.SUBMIT
```

FormButtonType – типы кнопок на форме

SUBMIT – отправка формы.

Пример:

```
ESS.enums.FormButtonType.SUBMIT
```

FormErrorType – типы ошибок

Перечислены в порядке уменьшения приоритета:

- **SYSTEM** – системная ошибка;
- **APPLICATION** – прикладная ошибка;
- **FORM_VALIDATION** – ошибка валидации формы;
- **FORM_REQUIRED** – ошибка заполнения формы;
- **FORM_SUBMITTED** – ошибка после отправки формы;
- **UNSPECIFIED** – прочие ошибки.

Перечисление используется для форм.

Пример:

```
ESS.enums.FormErrorType.SYSTEM
```

FormItemType – типы контролов формы

- **TEXT_BLOCK** – текстовый блок;
- **SEPARATOR** – разделитель;
- **PICTURE** – картинка;
- **PARAMETER** – запрашиваемое значение;
- **TABLE** – таблица;
- **GROUP** – группа;
- **TAB** – вкладка;
- **BUTTON** – кнопка.

Пример:

```
ESS.enums.FormItemType.TEXT_BLOCK
```

FormParameterType – типы параметров формы

Типы параметров формы:

- **NUMBER** – число;
- **STRING** – строка;
- **DATE_TIME** – дата и время;
- **SELECT** – выбор из списка значений;
- **REFERENCE_RECORD** – запись справочника;
- **BOOLEAN** – логическое значение;
- **DATE_TIME_INTERVAL** – интервал дат;
- **FILE** – файл;
- **QUERY** – выбор из произвольного источника.

Пример:

```
ESS.enums.FormParameterType.NUMBER
```

HintType – типы подсказки

- **NONE** – подсказка без типа;
- **WARNING** – предупреждение;
- **ERROR** – ошибка.

Пример:

```
ESS.enums.HintType.NONE
```

Parent – родительский элемент для стороннего контрола new

- **DEFAULT** – значение по умолчанию;
- **FORM** – форма;
- **DIALOG** – диалог;
- **CARD** – карточка;
- **WIDGET** – виджет;
- **TILE** – плитка быстрого доступа.

Пример:

```
ESS.enums.Parent.DEFAULT
```

RequestErrorCode – коды ошибок запросов

- **CONFIRMATION_CODE_REQUIRED** – для подтверждения операции требуется код;
- **CONFIRMATION_REQUIRED** – требуется подтверждение операции;
- **PHONE_NOT_UNIQUE** – номер телефона уже привязан к другому пользователю;
- **SERVICE_IS_BUSY** – сервис занят;

- **CONFIRMATION_MOBILE_REQUIRED** – для подтверждения операции требуется мобильное приложение;
- **SIGN_REJECTED** – пользователь отклонил запрос на подписание.

Пример:

```
ESS.enums.RequestErrorCode.CONFIRMATION_CODE_REQUIRED
```

ToastType – типы toast-уведомлений

- **ERROR** – ошибка;
- **INFO** – информирование;
- **SUCCESS** – подтверждение;
- **WARNING** – предупреждение.

Пример:

```
ESS.enums.ToastType.ERROR
```

Visibility – признаки видимости элемента

- **HIDDEN** – скрытый;
- **VISIBLE** – видимый.

Пример:

```
ESS.enums.Visibility.HIDDEN
```

WorkItemChangingReason – причины изменения элемента рабочего процесса

- **REFUSE** – отказано;
- **SIGN** – подписано.

Пример:

```
ESS.enums.WorkItemChangingReason.REFUSE
```

WorkItemStatus – статусы элемента рабочего процесса

- **COMPLETE** – завершено;
- **IN_PROCESS** – в процессе.

Пример:

```
ESS.enums.WorkItemStatus.COMPLETE
```

WorkProcessStatus – статусы задачи new

- **IN_PROCESS** – в работе;
- **DRAFT** – черновик;
- **SUSPENDED** – приостановлена;
- **COMPLETED** – завершена;
- **UNDER_REVIEW** – на приемке;
- **ABORTED** – прекращена.

Пример:

```
ESS.enums.WorkProcessStatus.IN_PROCESS
```

formatString – форматирует строку с описателями типа {0}

@param {string} inputString – исходная строка.

@param {...string} values – значения для подстановки.

@returns {string} – отформатированная строка.

Пример:

```
const result = ESS.formatString('{0}, {1}.', 'Первое значение', 'второе значение');
```

getColor – получает цвета из палитры по токену

@param {Colors} colorTokenId – идентификатор токена из палитры.

@returns {string} – цвет из палитры или изначальное значение, если цвет в палитре не найден.

Пример:

```
ESS.getColor(ESS.enums.Colors.Primary50Color);
```

hints – функции для работы с панелью подсказок на странице

hideHint – скрыть подсказку со страницы

@param {string} id – идентификатор подсказки.

Пример:

```
ESS.hints.hideHint(id)
```

showHint – отобразить подсказку на странице

@param {string} text – текст подсказки.

@param {string} title – заголовок подсказки.

@param {HintType} [hintType] – тип подсказки.

@param {number} [showTime] – время отображения подсказки в секундах. Если не указано, подсказка не исчезает автоматически.

@returns {string} – идентификатор подсказки.

Пример:

```
ESS.hints.showHint('Текст подсказки', 'Заголовок подсказки', ESS.enums.HintType.WARNING, 5);
```

L – возвращает строку локализации

Используется язык сайта. Строки загружаются из базы данных сервиса и из XML-файлов, расположенных в папке Localizations сайта.

@param {string} code – код строки локализации:

- для получения строки из базы данных сервиса код должен быть в формате **"@<Имя решения в верхнем регистре>.<Имя строки локализации в исходном регистре>"**. Строка должна быть доступна для использования на клиенте. Подробнее см. раздел«localizedStringValue – строка локализации»;
- для получения строки из XML-файлов код должен быть в формате **"<Имя группы в XML в верхнем регистре>.<Имя строки локализации в верхнем регистре>"**.

@param {...string} values – значения для подстановки описателей типа {0}.

@returns {string} – строка локализации на языке сайта или исходный код, если строка не найдена.

Пример получения строки из прикладного решения из базы данных:

```
// Узел в конфигурационном файле:
// <solution name="MySolution">
// ...
//   <localizedStringValues>
//     <localizedStringValue code="CardTitle" language="EN" useat="client">Card title: {0}
</localizedStringValue>
//     <localizedStringValue code="CardTitle" language="RU" useat="client">Заголовок карточки:
{0}</localizedStringValue>
//   </localizedStringValues>
// </solution>
// Вызов в прикладном событии:
ESS.L('@MYSOLUTION.CardTitle', 'Название карточки');
```

Пример получения строки из строк локализации сайта:

```
// В папке Localizations будет 2 файла: mySolution.ru.xml и mySolution.en.xml.
// Узел в каждом из файлов(в файле mySolution.en.xml будет английский текст):
// <Localization group="mySolution">
//   <s key="CardTitle">Заголовок карточки</s>
// </Localization>
// Вызов в прикладном событии:
```

```
ESS.L('MYSOLUTION.CARDTITLE');
```

methods – методы

facilityMethods – методы для отправки запросов по работе с услугами

getFacilityMethod – отправка запроса на получение услуги

@param {string} id – идентификатор или имя услуги.

@returns {Promise} – промис выполнения запроса без обработки. При успешном выполнении вернутся данные услуги. Подробнее см. на сайте developer.mozilla.org, статья Promise.

Пример:

```
ESS.methods.facilityMethods.getFacilityMethod(id)
```

runFacilityMethod – отправка запроса на запуск услуги

@param {string} facilityId – идентификатор или имя услуги.

@param {RequestOptions**} requestOptions** – опции запроса.

@returns **{Promise}** – промис выполнения запроса без обработки. Подробнее см. на сайте developer.mozilla.org, статья Promise.

Пример:

```
ESS.methods.facilityMethods.runFacilityMethod(facilityId, requestOptions)
```

notificationMethods – методы для отправки запросов по работе с оповещениями

getNotificationMethod – отправка запроса на получение оповещения

@param {string} id – идентификатор оповещения.

@returns {Promise} – промис выполнения запроса без обработки. При успешном выполнении вернутся данные оповещения. Подробнее см. на сайте developer.mozilla.org, статья Promise.

Пример:

```
ESS.methods.notificationMethods.getNotificationMethod(id)
```

workProcessMethods – методы для отправки запросов по работе с рабочими процессами

ChangeWorkItemStateMethod – изменение состояния элемента рабочего процесса

@param {string} systemName – название системы.

@param {string} workItemName – имя рабочего элемента.

@param {string} workItemId – идентификатор рабочего элемента.

@param {string} state – новое состояние. Возможные значения см. в WorkItemStatus.

@param {string} reason – причина изменения состояния. Возможные значения см. в WorkItemChangingReason.

@returns {Promise} – промис выполнения запроса на изменение состояния элемента рабочего процесса без обработки. Подробнее см. на сайте developer.mozilla.org, статья Promise.

Пример:

```
ESS.methods.workProcessMethods.ChangeWorkItemStateMethod(systemName, workItemName, workItemId, state, reason)
```

StopWorkProcessMethod – остановка рабочего процесса

@param {string} systemName – название системы.

@param {string} processName – имя процесса.

@param {string} processId – идентификатор процесса.

@returns {Promise} – промис выполнения запроса на остановку рабочего процесса без обработки. Подробнее см. на сайте developer.mozilla.org, статья Promise.

Пример:

```
ESS.methods.workProcessMethods.StopWorkProcessMethod(systemName, processName, processId)
```

UpdateWorkItemMethod – изменение элемента рабочего процесса

@param {string} systemName – название системы.

@param {string} workItemName – имя рабочего элемента.

@param {string} workItemId – идентификатор рабочего элемента.

@param {Object} parameters – параметры рабочего элемента.

@returns {Promise} – промис выполнения запроса на изменение элемента рабочего процесса без обработки. Подробнее см. на сайте developer.mozilla.org, статья Promise.

Пример:

```
ESS.methods.workProcessMethods.UpdateWorkItemMethod(systemName, workItemName, workItemId, parameters)
```

UpdateWorkProcessMethod – изменение рабочего процесса

@param {string} systemName – название системы.

@param {string} processName – имя процесса.

@param {string} processId – идентификатор процесса.

@param {Object} parameters – параметры рабочего процесса.

@returns {Promise} – промис выполнения запроса на изменение рабочего процесса без обработки. Подробнее см. на сайте developer.mozilla.org, статья Promise.

Пример:

```
ESS.methods.workProcessMethods.UpdateWorkProcessMethod(systemName, processName, processId, parameters)
```

network – функции для работы с сетевыми запросами

getRequestErrorMessage – вернуть текст из ошибки запроса

Возвращает текст ошибки из результата запроса.

Обрабатывается недоступность сервера и формат ошибки вида **{ isError, reason }**.

Если ошибка стандартного вида, то, при наличии, возвращается значение свойства **message**, иначе свойства **statusText**.

Сервер приложения личного кабинета может вернуть ошибку в формате FailureResult. Возможные значения свойства **code** см. в разделе «RequestErrorCode».

@param {RequestResult**} [requestResult]** – результат запроса.

@returns {string} – текст ошибки. Если в параметр **requestResult** передан не объект, возвращается строка с исходным значением.

Пример:

```
ESS.network.getRequestErrorMessage(requestResult)
```

refreshToken

Функция обновления токена пользователя.

Пример:

```
ESS.network.refreshToken()
```

sendRequest – выполнить запрос

@param {string} url – адрес запроса. В параметре можно указать полный или относительный адрес. Если относительный адрес начинается с символа «/», то он добавляется к адресу API-шлюза (переменная **g_ApiGatewayEndpoint**), иначе – к адресу сайта личного кабинета.

@param {RequestOptions} [options={}] – опции запроса. Метод по умолчанию **GET**.

@param {boolean} [isRetry=false] – признак повторного запроса. Если не указан, используется значение признака **options.isRetry**. Если признак **options.isRetry** также не указан, используется значение **false**.

@returns {Promise} – промис запроса. При ошибке возвращается оригинальный объект ошибки с актуализированным текстом, записанным в свойство объекта **message**. Текст формируется аналогично результату функции getRequestErrorMessage.

Пример:

```
ESS.network.sendRequest('/api/functions/GetTaskActiveText?assignmentId=' + data.id, { method:
'GET' })
  .then(activeText => {
    ESS.dialogs.showMessageBox('Текст задачи', activeText);
  })
  .catch(error => {
    ESS.toasts.showToast(error.message, ESS.enums.ToastType.ERROR, false);
  });
```

sendRequestWithBlockUI

Выполняет запрос с отображением оверлея во время выполнения. Оверлей отображается только во время выполнения основного запроса.

Если в цепочке вызова будут другие длительные операции, то при необходимости отображения оверлея, его следует отображать и скрывать вручную.

Входные параметры аналогичны параметрам функции sendRequest.

ВАЖНО. Для признака **options.blockUI** принудительно устанавливается значение **true**.

Пример:

```
ESS.network.sendRequestWithBlockUI('/api/functions/GetTaskActiveText?assignmentId=' + data.id,
 { method: 'GET' })
  .then(activeText => {
    ESS.dialogs.showMessageBox('Текст задачи', activeText);
  })
  .catch(error => {
    ESS.toasts.showToast(error.message, ESS.enums.ToastType.ERROR, false);
  });
```

Подробнее см. на сайте developer.mozilla.org, статья Promise.

overlay – функции для работы с оверлеем

hideOverlay – скрыть оверлей

@param {string} id – идентификатор инициатора показа.

Пример:

```
ESS.overlay.hideOverlay(id)
```

showOverlay – отобразить оверлей

@param {string} id – идентификатор инициатора показа.

@param {string} text – текст. Если не указан, будет отображаться текст из строки локализации с кодом **DEFAULT.PLEASE_WAIT**.

@param {TOverlayOptions**} [options]** – опции оверлея.

Пример:

```
const options = {
  blockUnload: false,
```

```
};
ESS.overlay.showOverlay('test', undefined, options);
```

parseDate – функции для получения даты из строки

Получает дату из строкового формата. В отличие от конструкции **"new Date()"**, корректно обрабатывает короткую запись с учетом часового пояса.

@param {string} [str=''] – дата в формате строки.

@param {string} [locale] – локализация исходной строки. По умолчанию локализация сайта. Рекомендуется использовать ISO-формат, в таком случае локализацию можно не указывать.

@returns {Date} – объект даты.

Пример:

```
ESS.parseDate('1988-08-15');
```

parseFloat – расширенная функция выделения числа с плавающей точкой

Стандартная функция **parseFloat** выделяет первые символы строки, разрешенные для указания в числах, а не всю строку. Поэтому из строки **"12e3qwer"** будет выделено число **12000**, а не **NaN**.

@param {any} value – входное значение.

@returns {number} – выделенное число или **NaN**.

Пример:

```
ESS.parseFloat(value)
```

preview – функции по работе с предпросмотром

openDocumentPreview – открыть предпросмотр документа

@param {StoredDocument | Document} document – модель документа. Подробнее см. разделы «StoredDocument» и «Document».

Пример 1:

```
openDocumentPreview({
 binaryDataId: '623a645f-dac6-4992-a970-6aa35ef3bd24',
 name: 'Заявление_об_увольнении_Снигирев_С._А._от_03.11.2022.pdf',
});
```

где:

- **StoredDocument** – документ, находящийся в файловом хранилище;
- {string} **StoredDocument.binaryDataId** – идентификатор бинарных данных документа;
- {string} **StoredDocument.name** – наименование документа.

Пример 2:

```
openDocumentPreview({
 id: '413',
 name: 'IElectronicDocuments',
 title: 'Заявление_об_увольнении_Снигирев_С._А._от_03.11.2022.pdf',
 system: 'DirectumRX',
});
```

где:

- **Document** – документ целевой системы;
- {UniqueId} **Document.id** – идентификатор документа;
- {string} **Document.name** – наименование типа документа;
- {string} **Document.title** – заголовок документа;
- {string} **Document.system** – имя системы.

openPreview – открыть предпросмотр документа

@param {string} binaryDataId – идентификатор бинарных данных документа.

@param {string} name – наименование документа.

Пример:

```
ESS.preview.openPreview(binaryDataId, name)
```

replaceSubstrings – заменяет все вхождения указанной подстроки на новую подстроку

@param {string} inputString – исходная строка.

@param {string} substring – подстрока, которую нужно заменить.

@param {string} newSubstring – подстрока, на которую нужно заменить.

@returns {string} – строка с замененными подстроками.

Пример:

```
ESS.replaceSubstrings(inputString, substring, newSubstring)
```

scripts – прикладные сценарии

Список сценариев определяется конфигурацией узла scripts.

Пример:

```
ESS.scripts
```

signing – функции подписания

createPackageSignatures – сформировать подписи для пакета документов из сервиса хранилищ

@param {Object} params – параметры функции. Свойства:

- **@param {Array<SystemCertificate>} params.certificates** – сертификаты;
- **@param {Array<StoredDocument>} params.documents** – документы;
- **@param {string} [params.packageTitle=params.documents[0].name]** – название пакета подписываемых документов. Если не указано, используется имя первого документа;
- **@param {function} [params.afterSigningCallback]** – функция для выполнения после успешного подписания. Входным параметром является массив строк подписей в формате Base64;
- **@param {function} [params.cancelationCallback]** – функция для выполнения после отмены подписания пользователем. Без входных параметров.

@returns {Promise} – промис формирования подписей. При успешном выполнении операции возвращается массив строк подписей в формате Base64. При завершении операции с ошибкой возвращается объект ошибки в формате **{code, message}**.

Пример класса для прикладного клиентского сценария, который используется в шаге движка услуг с типом actionStep:

```
class createSignature {
  execute(params) {
    const document = JSON.parse(params.SigningDocument);
    const serviceUrl = '/api/functions/GetEmployeeCertificates';
    return new Promise(resolve => {
      ESS.network.sendRequest(serviceUrl)
        .then(certs => ESS.signing.createPackageSignatures({
          certificates: certs,
          documents: [{
            binaryDataId: document.BinaryDataId,
            name: document.Name,
          }],
          cancelationCallback: params.cancelationCallback,
        }))
        .then(signatures => {
          resolve(JSON.stringify({
            ...document,
            Signature: signatures[0],
            SignatureType: params.SignatureType,
          }));
        })
        .catch(console.log);
    });
  }
}
```

createSignature – сформировать подпись для документа из сервиса хранилищ

@param {Object} params – параметры функции. Свойства:

- **@param {Array<SystemCertificate>} params.certificates** – сертификаты;
- **@param {StoredDocument} params.document** – документ;
- **@param {function} [params.afterSigningCallback]** – функция для выполнения после успешного подписания. Входным параметром является строка подписи в формате Base64;
- **@param {function} [params.cancelationCallback]** – функция для выполнения после отмены подписания пользователем. Без входных параметров.

@returns {Promise} – промис формирования подписи. При успешном выполнении операции возвращается строка подписи в формате Base64. При завершении операции с ошибкой возвращается объект ошибки в формате **{code, message}**.

Пример класса для прикладного клиентского сценария, который используется в шаге движка услуг с типом actionStep:

```
class createSignature {
  execute(params) {
    const document = JSON.parse(params.SigningDocument);
    const serviceUrl = '/api/functions/GetEmployeeCertificates';
    return new Promise(resolve => {
      ESS.network.sendRequest(serviceUrl)
        .then(certs => ESS.signing.createSignature({
          certificates: certs,
          document: {
            binaryDataId: document.BinaryDataId,
            name: document.Name,
          },
          cancelationCallback: params.cancelationCallback,
        }))
        .then(signature => {
          resolve(JSON.stringify({
            ...document,
            Signature: signature,
            SignatureType: params.SignatureType,
          }));
        })
        .catch(console.log);
    });
  }
}
```

createSigningCertificate – сгенерировать пользовательский сертификат для подписания

@returns {Promise} – промис генерации сертификата. При успешном выполнении операции возвращается идентификатор сгенерированного сертификата. При завершении операции с ошибкой возвращается текст ошибки.

Пример:

```
ESS.signing.createSigningCertificate()
```

sign – подписать документы

@param {Object} params – параметры функции. Свойства:

- **@param {Array<SystemCertificate>} params.certificates** – сертификаты;
- **@param {Array<Document>} params.documents** – список подписываемых документов;
- **@param {string} params.documentGroup** – группа подписываемых документов;
- **@param {string} params.systemName** – система, в которой находятся документы;
- **@param {string} params.signatureType** – тип подписи;
- **@param {string} params.packageTitle** – название пакета подписываемых документов;
- **@param {string} [params.signedBy]** – идентификатор подписанта. Поскольку подписание документов ведется из карточки задания, для параметра необходимо передавать значение **data.performer**;
- **@param {function} [params.callback]** – функция для выполнения после успешного подписания. Без входных параметров;
- **@param {function} [params.cancelationCallback]** – функция для выполнения после отмены подписания пользователем;
- **@param {function} [params.errorCallback]** – функция для выполнения после ошибки подписания.

Входным параметром является объект ошибки. Для получения текста ошибки можно использовать функцию getRequestErrorMessage. Если функция не указана, ошибка будет выведена в toast-уведомлении. Полная информация об ошибке в обоих случаях фиксируется в консоль.

Пример класса для прикладного клиентского сценария, который используется в действии карточки объекта:

```
class signDocument {
  execute(params) {
    const [data, sender] = params.arguments;
    const serviceUrl = '/api/functions/GetEmployeeCertificates';
    ESS.network.sendRequest(serviceUrl)
      .then(certs => {
        const documents = [];
        const documentGroups = data.attachments.filter(x => x.group === 'DocumentGroup' ||
x.group === 'AddendaGroup');
        documentGroups.forEach(a => documents.push({ id: a.id, title: a.title }));
        const signatureType = data.requisites.find(_ref => {
          const { name } = _ref;
          return name === 'SignatureType';
        }).value;

        function callback() {
          const overlayId = 'signDocument_changeWorkItemState';
          ESS.overlay.showOverlay(overlayId);
          ESS.actions.workProcessActions.changeWorkItemState(data, 'Complete',
'Sign').then(() => {
            ESS.closeElement(sender);
            ESS.toasts.showToast('Документы подписаны.', ESS.enums.ToastType.SUCCESS);
            ESS.widgets.updateWidgets();
          }).catch(error => {
            const text = 'При выполнении задания произошла ошибка. Обратитесь к
администратору системы.';
```

```
            ESS.toasts.showErrorToastWithLog(text, error.message);
          }).finally(() => {
            ESS.overlay.hideOverlay(overlayId);
          });
        }
        ESS.signing.sign({
          certificates: certs,
          documents,
          documentGroup: 'IElectronicDocuments',
          systemName: data.system,
          signatureType,
          packageTitle: data.title,
          callback,
          signedBy: data.performer,
          cancelationCallback: () => ESS.toasts.showToast('Подписание было отменено.'),
          errorCallback: err => {
            ESS.toasts.showToast(`При подписании возникла ошибка:
${ESS.network.getRequestErrorMessage(err)}`);
          },
        });
      })
      .catch(err => {
        ESS.toasts.showErrorToastWithLog(err);
      });
  }
}
```

Подробнее см. на сайте developer.mozilla.org, статья Promise.

simplifyDate – возвращает локализованную дату в виде строки

Если время нулевое, строка будет в упрощенном виде (без времени).

@param {Date} date – объект даты для преобразования.

@returns {string} – строка с локализованной датой.

Пример:

```
ESS.simplifyDate(date)
```

toasts – функции для работы с toast-уведомлениями

hideToast – скрыть toast-уведомление

@param {string} id – идентификатор toast-уведомления.

Пример:

```
ESS.toasts.hideToast(id)
```

showErrorToastWithLog – отобразить toast-ошибку и зафиксировать данные

Отображает toast-ошибку и фиксирует подробные данные в консоль.

@param {string} message – сообщение для отображения в toast-уведомлении.

@param {string | Object} [details=""] – подробные данные для записи в консоль.

@param {boolean} [autoHide=true] – признак необходимости автоматического скрытия toast-уведомления.

@returns {string} – идентификатор toast-уведомления.

Пример:

```
ESS.toasts.showErrorToastWithLog(message, details, autoHide)
```

showToast – отобразить toast-уведомление

@param {string} message – сообщение для отображения.

@param {ToastType} [type=ToastType.INFO] – тип toast-уведомления.

@param {boolean} [autoHide=true] – признак необходимости автоматического скрытия toast-уведомления. Для установки длительности отображения используется значение из переменной **g_ToastAutoHideTimeout**. Оно устанавливается из конфигурационного файла сайта личного кабинета, секции **ClientConfiguration**, настройки **ToastAutoHideTimeout**. Если значение не указано в настройке или указано **0**, то toast-уведомление будет отображаться 5 секунд.

@returns {string} – идентификатор toast-уведомления.

Пример:

```
ESS.toasts.showToast('Сообщение для отображения', ESS.enums.ToastType.SUCCESS, false);
```

showToastEx – отобразить toast-уведомление для blueprint toast

Отображает toast-уведомление. На вход принимает пропсы компонента blueprint toast.

@param {Object} props – пропсы компонента blueprint toast. Подробнее см. в документации на сайте blueprintjs.com, раздел Toast.

@returns {string} – идентификатор toast-уведомления.

Пример:

```
ESS.toasts.showToastEx(props)
```

showWarningToastWithLog – отобразить toast–предупреждение и зафиксировать данные

Отображает toast–предупреждение и фиксирует подробные данные в консоль.

@param {string} message – сообщение для отображения в toast-уведомлении.

@param {string | Object} [details=""] – подробные данные для записи в консоль.

@param {boolean} [autoHide=true] – признак необходимости автоматического скрытия toast-уведомления.

@returns {string} – идентификатор toast-уведомления.

Пример:

```
ESS.toasts.showWarningToastWithLog(message, details, autoHide)
```

updateElement – обновляет элемент

Обновляет элемент, используя соответствующую функцию. Дополнительно см. раздел «closeElement – закрывает элемент».

@param {[Element]} element – элемент, который необходимо обновить. Например, объект sender в событиях карточки или объект sender.parent в событии закрытия формы услуги.

Пример обновления данных карточки по кнопке **Обновить**:

```
ESS.updateElement(sender);
```

Пример обновления данных карточки закрытии формы услуги:

```
ESS.updateElement(sender.parent);
```

widgets – функции по работе с виджетами

updateWidgets – принудительно обновить виджеты

Пример:

```
ESS.widgets.updateWidgets()
```

Пространство имен FormFunctions

Функции для работы с формой услуги. Доступны только в событиях формы и вызываются из обработчика события с помощью параметра **functions**.

addItem – добавить строку в параметр формы типа «Список» или элемент формы типа «Таблица»

Если необходимо дождаться окончания добавления строки перед следующей операцией, то следует использовать функцию addItemAsync. Для удаления строки используется функция removeItem.

@param {string} name – имя параметра формы типа «Список» или элемента формы типа «Таблица».

@returns {string} – индекс добавляемого значения, используется для таблиц.

Пример добавления и удаления в таблице:

```
const tableName = 'Param&Param2Grid';
const index = functions.addItem(tableName);
functions.removeItem(tableName, index);
```

Пример добавления и удаления в списке:

```
functions.addItem('List');
functions.removeItem('List', 0);
```

ПРИМЕЧАНИЕ. Если необходимо дождаться окончания добавления строки перед следующей операцией, используйте функцию addItemAsync.

addItemAsync – добавить строку в параметр формы типа «Список» или элемент формы типа «Таблица» с помощью асинхронной функции

Позволяет дождаться окончания процесса добавления строки перед последующими операциями.

@param {string} name – имя параметра формы типа «список» или элемента формы типа «таблица».

@returns {string} – индекс добавляемого значения.

Пример добавления и удаления в таблице:

```
const tableName = 'Param&Param2Grid';
const rowIndex = await functions.addItemAsync(tableName);
functions.removeItem(tableName, rowIndex);
```

Пример добавления и удаления в списке:

```
const index = await functions.addItemAsync('List');
functions.removeItem('List', 0);
```

ПРИМЕЧАНИЕ. Если нет необходимости дожидаться окончания добавления строки перед следующей операцией, используйте функцию addItem. Для удаления строки используйте функцию removeItem.

addPerformingAction – добавить выполняемое прикладное действие на форму

Имя действия передается в событие closedEventHandler для определения, в рамках какого действия была закрыта форма. Для удаления действия используется функция removePerformingAction.

@param {string} actionName – имя выполняемого действия.

Пример:

```
functions.addPerformingAction(actionName)
```

executeItemCalculate – вызвать вычисления поля формы

@param {string} formItemName – имя элемента формы.

@param {string} [tableName] – имя таблицы, в которой находится элемент формы.

Подробнее см. описание функции calculateFunction.

Пример:

```
functions.executeItemCalculate(formItemName, tableName)
```

getValues – вернуть текущие значения элементов формы

Значение интервала дат возвращается в виде строки с сериализованным объектом в следующем формате: **{ DateBegin: string | null, Duration: string, DateEnd: string | null }**.

@returns {Object} – список значений элементов в формате ключ/значение.

Пример:

```
functions.getValues()
```

hideHint – скрыть подсказку с формы

@param {string} hintId – ИД подсказки, который был указан при ее отображении.

Пример:

```
functions.hideHint('hint_01')
```

hideLoader – скрыть индикатор прогресса с формы

@param {string} loaderId – ИД индикатора, который был указан при его отображении.

Пример:

```
functions.hideLoader('loader_01')
```

isItemDisabled – проверить, заблокирован ли элемент формы

Для установки признака блокировки используется функция setIsItemDisabled.

@param {string} formItemName – имя элемента формы.

@param {string} columnName – название столбца. Используется для таблиц.

@returns {boolean} – результат проверки.

Пример:

```
functions.isItemDisabled(formItemName, columnName)
```

isItemHidden – проверить, скрыт ли элемент формы или вкладка

Для установки признака скрытия используется функция setIsItemHidden.

@param {string} formItemName – имя элемента формы или вкладка.

@param {string} columnName – название столбца. Используется для таблиц.

@returns {boolean} – результат проверки.

Пример:

```
functions.isItemHidden(formItemName, columnName)
```

removeError – удалить ошибку с формы

Для добавления ошибки используется функция showError.

@param {string} errorId – идентификатор ошибки.

Пример добавления ошибки в событии изменения ячейки таблицы.:

```
const errorType = ESS.enums.FormErrorType.FORM_VALIDATION;
const tableName = 'Param&Param2Grid';
// Индекс и имя колонки берутся из платформенного параметра, передающегося в событие формы
с моделью элемента формы.
const errorId = functions.showError('Ошибка', 5, errorType, tableName, formItem.index,
formItem.columnName);
// Удаление ошибки.
functions.removeError(errorId);
```

removeItem – удалить строку из параметра формы типа «Список» или элемента формы типа «Таблица»

Есть возможность дождаться окончания процесса удаления строки. Для добавления строки используется функция addItem или addItemAsync.

@param {string} name – имя параметра формы типа «Список» или элемента формы типа «Таблица».

@param {number | string} index – индекс удаляемого значения. Число для списка или строка для таблицы.

Примеры добавления и удаления в таблице с ожиданием:

```
const tableName = 'Param&Param2Grid';
const index = functions.addItem(tableName);
await functions.removeItem(tableName, index);
```

Примеры добавления и удаления в списке без ожидания:

```
functions.addItem('List');
functions.removeItem('List', 0);
```

ПРИМЕЧАНИЕ. Для добавления строки используйте функцию addItem или addItemAsync.

removePerformingAction – удалить выполняемое прикладное действие с формы

Для добавления действия используется функция addPerformingAction.

@param {string} [actionName] – имя прикладного действия для удаления. Если не указано, будут удалены все прикладные действия.

Пример:

```
functions.removePerformingAction(actionName)
```

setCellDataSource – установить новый источник данных для выпадающего списка в ячейке таблицы

Устанавливает новый источник данных для параметра с типом FacilityParameterQuery в ячейке таблицы. Этот источник данных имеет приоритет над источником данных, указанным для параметра.

Для восстановления используемого источника данных параметра необходимо повторно вызвать эту функцию с передачей пустой строки в параметре **newDataSource**. Для установки источника данных для параметра используется функция setDataSource.

Подробнее про параметр типа **FacilityParameterQuery** см. раздел «facilityParameter – параметр услуги».

Варианты вызов функции с разными параметрами:

1. Функция принимает несколько аргументов.

   @param {string} gridName – имя таблицы.

   @param {string} gridRow – индекс строки таблицы.

   @param {string} gridColumn – имя колонки.

   @param {string} newDataSource – новый источник данных для ячейки. Если новый источник данных совпадает с установленным для ячейки ранее, функция прекращает выполнение. Если новый источник данных совпадает с уже установленным для параметра, функция не прекращается.

   @param {string | null} [newValue=null] – новое значение ячейки, которое устанавливается после получения данных из нового источника. Особенности работы:

   - значение устанавливается, только если оно присутствует в новых данных;
   - если используется постраничная загрузка, то значение подставляется только когда находится на первой странице, или передан параметр **newDataSourceSingle**, по которому доступно новое значение. Иначе значение сбросится;
   - если **newValue** не указано, значение в ячейке сбросится в любом случае. Для сохранения ранее выбранного значения необходимо использовать вариант функции с передачей объекта и установкой флага **shouldSaveValue** в значении **true**.

   @param {string} [newDataSourceSingle='''] – новый источник данных для получения одиночного элемента. Используется для получения устанавливаемого значения, если оно находится вне загруженных данных. Например, если источник поддерживает постраничную загрузку документов, а устанавливаемое значение находится не на первой странице. Особенности работы:

   - если **newDataSourceSingle** не указано, то источник данных для получения одиночного элемента будет сброшен, даже если ранее он уже был установлен;
   - если **newDataSourceSingle** не указано, и новое значение ячейки оказалось вне диапазона новых данных, оно не будет установлено, даже если входило в изначальный список.

Пример изменения источника данных в соседней ячейке таблицы:

```
const newUrl = '/api/references?systemName=EssBase&referenceName=ICities';
const newSingleUrl = '/api/references?systemName=EssBase&referenceName=ICities&filter=Id%20eq%20{value}&takeCount=1';
const tableName = 'Param&QueryGrid';
// Индекс и значение берутся из платформенного параметра, передающегося в событие формы с моделью элемента формы.
functions.setCellDataSource(tableName, formItem.index, 'Query', newUrl, formItem.value, newSingleUrl);
```

2. Функция принимает аргумент в виде объекта.

   @param {TSetCellDataSourceParams} params – параметры функции.

   Пример изменения источника данных в соседней ячейке таблицы:

```
const newUrl = '/api/references?systemName=EssBase&referenceName=ICities';
const newSingleUrl = '/api/references?systemName=EssBase&referenceName=ICities&filter=Id%20eq%20{value}&takeCount=1';
const tableName = 'Param&QueryGrid';
// Индекс и значение берутся из платформенного параметра, передающегося в событие формы с моделью элемента формы.
functions.setCellDataSource({
  gridName: tableName,
  gridRow: formItem.index,
  gridColumn: 'Query',
  newDataSource: newUrl,
  newValue: formItem.value,
  newDataSourceSingle: newSingleUrl,
  shouldSaveValue: true,
});
```

setDataSource – установить новый источник данных для выпадающего списка

Устанавливает новый источник данных для параметра с типом FacilityParameterQuery. Для установки источника данных для ячейки таблицы используется функция setCellDataSource.

Подробнее про параметр типа **FacilityParameterQuery** см. раздел «facilityParameter – параметр услуги».

Варианты вызов функции с разными параметрами:

1. Функция принимает несколько аргументов.

   @param {string} parameterName – имя параметра.

   @param {string} newDataSource – новый источник данных.

   @param {string | null} [newValue=null] – новое значение параметра. Устанавливается после получения данных из нового источника. Особенности работы:

   - значение устанавливается, только если оно присутствует в новых данных;

- если используется постраничная загрузка, то значение подставляется только когда находится на первой странице, или передан параметр **newDataSourceSingle**, по которому доступно новое значение. Иначе значение сбросится;
- если **newValue** не указано, значение в ячейке сбросится в любом случае. Для сохранения ранее выбранного значения необходимо использовать вариант функции с передачей объекта и установкой флага **shouldSaveValue** в значении **true**.

@param {string} [newDataSourceSingle='''] – новый источник данных для получения одиночного элемента. Используется для получения устанавливаемого значения, если оно находится вне загруженных данных. Например, если источник поддерживает постраничную загрузку документов, а устанавливаемое значение находится не на первой странице. Особенности работы:

- если **newDataSourceSingle** не указано, то источник данных для получения одиночного элемента будет сброшен, даже если ранее он уже был установлен;
- если **newDataSourceSingle** не указано, и новое значение параметра оказалось вне диапазона новых данных, оно не будет установлено, даже если входило в изначальный список.

Пример:

```
const newUrl = '/api/references?systemName=EssBase&referenceName=ICities';
const newSingleUrl = '/api/references?systemName=EssBase&referenceName=ICities&filter=Id%
20eq%20{value}&takeCount=1';
functions.setDataSource('Query', newUrl, '8', newSingleUrl);
```

2. Функция принимает аргумент в виде объекта.

   @param {TSetParameterDataSourceParams} params – параметры функции.

   Пример:

```
const newUrl = '/api/references?systemName=EssBase&referenceName=ICities';
const newSingleUrl = '/api/references?systemName=EssBase&referenceName=ICities&filter=Id%
20eq%20{value}&takeCount=1';
functions.setDataSource({
  parameterName: 'Query',
  newDataSource: newUrl,
  newValue: '8',
  newDataSourceSingle: newSingleUrl,
  shouldSaveValue: true,
});
```

setIsItemDisabled – установить признак блокировки элемента формы

Для проверки, заблокирован ли элемент, используется функция isItemDisabled.

@param {string} name – имя элемента формы.

@param {boolean} value – значение признака.

@param {string} columnName – название столбца. Используется для таблиц.

Пример:

```
functions.setIsItemDisabled(name, value, columnName)
```

setIsItemHidden – установить признак скрытия элемента формы или вкладки

Для проверки, скрыт ли элемент, используется функция isItemHidden.

@param {string} name – имя элемента формы или вкладки.

@param {boolean} value – значение признака.

@param {string} columnName – название столбца. Используется для таблиц.

Пример:

```
functions.setIsItemHidden(name, value, columnName)
```

setValue – установить значение параметра на форме

Установка значения для параметра типа «Файл» недоступна.

@param {string} name – имя параметра или таблицы.

@param {string | null} value – значение параметра. Для дат и интервала дат действуют те же ограничения, что и для значения по умолчанию. Подробнее см. раздел «facilityParameter – параметр услуги».

@param {number | string} [index] – позиция значения в списке или индекс строки таблицы в формате GUID.

@param {string} [columnName] – название столбца, используется для таблиц.

Пример изменения значения в соседней ячейке таблицы:

```
const tableName = 'Param&Param2Grid';
// Индекс берется из платформенного параметра, передающегося в событие формы с моделью элемента формы.
functions.setValue(tableName, 'Новое значение', formItem.index, 'Param2');
```

setValueQuiet – установить значение параметра на форме без вызова события изменения

Входные параметры аналогичны параметрам функции setValue.

Пример изменения значения в соседней ячейке таблицы:

```
const tableName = 'Param&Param2Grid';
// Индекс берется из платформенного параметра, передающегося в событие формы с моделью элемента формы.
functions.setValueQuiet(tableName, 'Новое значение', formItem.index, 'Param2');
```

showError – добавить ошибку на форму

Ошибка может относиться как ко всей форме целиком, так и к отдельным параметрам. К одному и тому же параметру или форме в целом может относиться несколько ошибок. Для удаления ошибки используется функция removeError.

@param {string} text – текст ошибки.

@param {number} [showTime] – время отображения ошибки в секундах. Если не указано, то ошибка не скрывается автоматически.

@param {FormErrorType} [type=FormErrorType.UNSPECIFIED] – тип ошибки.

@param {string} [controlName] – имя параметра.

@param {number | string} [controlIndex] – позиция значения в списке или индекс строки таблицы в формате GUID.

@param {string} [columnName] – имя столбца таблицы.

@returns {string} –идентификатор ошибки.

Пример добавления ошибки в событии изменения ячейки таблицы:

```
const errorType = ESS.enums.FormErrorType.FORM_VALIDATION;
const tableName = 'Param&Param2Grid';
// Индекс и имя колонки берутся из платформенного параметра, передающегося в событие формы
с моделью элемента формы.
const errorId = functions.showError('Ошибка', 5, errorType, tableName, formItem.index,
formItem.columnName);
// Удаление ошибки.
functions.removeError(errorId);
```

showHint – отобразить подсказку к форме

Подсказка отображается, только если форма заблокирована или отображается индикатор прогресса. При этом ошибки, относящиеся ко всей форме, скрываются.

@param {string} hintId – ИД подсказки. Должен быть уникальным в пределах формы. Используется для скрытия подсказки.

@param {string} text – текст подсказки.

Пример:

```
functions.showHint('hint_01', 'Идет получение данных. Пожалуйста, подождите.')
```

showLoader – отобразить индикатор прогресса

@param {string} loaderId** – ИД индикатора. Должен быть уникальным в пределах формы. Используется для скрытия индикатора.

Пример:

```
functions.showLoader('loader_01')
```

updateValues – вызвать код на вычисляемых полях формы

Подробнее см. описание функции calculateFunction.

Пример:

```
functions.updateValues()
```

waitQueryValueChange – ожидание окончание установки значения в выпадающем списке

@param {string} name – имя параметра или имя таблицы.

@param {string} [index] – индекс строки в таблице.

@param {string} [columnName] – имя колонки таблицы.

Пример ожидания установки нового значения после смены источника данных:

```
const newUrl = '/api/references?systemName=EssBase&referenceName=ICities';
const newSingleUrl = '/api/references?systemName=EssBase&referenceName=ICities&filter=Id%
20eq%20{value}&takeCount=1';
functions.setDataSource('Query', newUrl, '8', newSingleUrl);
await functions.waitQueryValueChange('Query');
```

Пространство имен CardFunctions

Функции для работы с карточкой объекта. Доступны только в событиях карточки и вызываются из обработчика события **openedEventHandler** с помощью параметра **functions**.

setVisibility – установить видимость кнопки

Видимость учитывается только в случае, если для карточки свойство **data.showButtons** имеет значение **true**. В ином случае все кнопки будут скрыты.

@param {string | Array<string>} nameOrNames – имя или массив имен кнопок, для которых устанавливается видимость.

@param {Visibility} [visibility=Visibility.VISIBLE] – новое состояние видимости.

Пример скрытия кнопки в событии после открытия карточки объекта:

```
functions.setVisibility('Button', ESS.enums.Visibility.HIDDEN);
```

Используемые типы

В разделе приведены описания типов параметров, используемых в объектах, и их свойств.

Document – документ

@property {string} id – идентификатор документа.

@property {string} name – наименование типа документа.

@property {string} title – заголовок документа.

@property {string} system – имя системы.

Element – элемент

@property {string} id – идентификатор элемента.

@property {string} type – тип элемента.

FailureResult – результат отказа

@property {boolean} isError – признак отказа из-за ошибки.

@property {string} reason – причина отказа.

@property {string} details – детальное описание проблемы.

@property {string} code – код ошибки.

FormError – ошибка на форме

@property {string} errorId – идентификатор ошибки.

@property {string} text – текст ошибки.

@property {FormErrorType} type – типы ошибок.

@property {string} [controlName] – имя параметра или таблицы.

@property {string | number} [controlIndex] – индекс элемента списка или строки таблицы.

@property {string} [columnName] – имя столбца в таблице.

@property {boolean} [isApplied] – признак того, что ошибка добавлена через прикладную функцию.

RequestOptions – опции запроса

@property {string} [body] – тело запроса. Подробнее см. на сайте developer.mozilla.org , статья Using the Fetch API.

@property {string} [method] – метод запроса. По умолчанию **GET**. Подробнее см. на сайте developer.mozilla.org , статья Using the Fetch API.

@property {AbortSignal} [signal] – объект для прекращения запроса. Подробнее см. на сайте developer.mozilla.org , статья AbortController.signal.

@property {boolean} [skipContentType] – признак необходимости пропустить установку значения по умолчанию для заголовка **Content-Type**. Если не указан или равен **false**, устанавливается значение по умолчанию: **application/json; charset=utf-8**. Если равен **true**, то устанавливается значение из свойства **options.contentType**.

@property {string} [contentType] – значение для заголовка **Content-Type**. Если не указано, то заголовок **Content-Type** не устанавливается. Подробнее см. на сайте developer.mozilla.org , статья Using the Fetch API.

@property {string} [token] – токен для авторизации.

@property {boolean} [nocache] – признак выполнения запроса без кэширования.

@property {boolean} [skipGettingToken] – признак выполнения запроса без использования токена.

@property {boolean} [isRetry] – признак повторного запроса.

@property {boolean} [blockUI] – признак отображения оверлея.

ВАЖНО. Оверлей отображается только во время выполнения основного запроса. Если в цепочке вызова будут другие длительные операции, то оверлей нужно отображать и скрывать вручную.

@property {boolean} [blockUnload] – признак блокировки события **unload** страницы. Учитывается, только если признак **blockUI** равен **true**. Подробнее см. в описании типа TOverlayOptions.

RequestResult – результат запроса new

Объединение типов FailureResult, **Response**, **Error** и **string**.

Описание типов **Response** и **Error** см. на сайте developer.mozilla.org, статьи Response и Error.

StoredDocument – документ в файловом хранилище

@property {string} binaryDataId – идентификатор бинарных данных документа.

@property {string} name – наименование документа.

SystemCertificate – сертификат

@property {string} providerId – провайдер подписания.

@property {string} pluginId – плагин подписания.

@property {string} [issuerInfo] – издатель сертификата.

@property {string} [serialNumber] – серийный номер сертификата.

@property {string} subjectInfo – сведения о владельце.

@property {string} thumbprint – отпечаток сертификата.

@property {Object} [validPeriod] – период действия сертификата. Свойства:

- **@property {string} validPeriod.from** – начало действия;
- **@property {string} validPeriod.to** – окончание действия.

TDateInterval – интервал дат

@property {string | null} begin – начало интервала.

@property {string | null} end – окончание интервала.

СОВЕТ. Рекомендуется указывать значения в формате ISO. Подробнее см. на сайте developers.mozilla.org, статья Date.prototype.toISOString().

TDialogBaseItemProperties – базовые свойства элемента диалога

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

TDialogButton – кнопка диалога

@property {string} name – имя кнопки.

@property {'submit' | 'reset' | 'button'} [type=button] – тип кнопки. Определяет действие с формой при нажатии. Возможные значения:

- **submit** - отправка формы;
- **reset** - очистка формы от введенных значений и ошибок;
- **button** - только действия из обработчика события **onClick**. Используется по умолчанию.

@property {string} caption – заголовок кнопки.

@property {string} [icon] – имя иконки кнопки. По умолчанию кнопка отображается без иконки.

@property {boolean} [isPrimary=false] – признак того, что кнопка основная. По умолчанию кнопка не является основной.

@property {boolean} [disabled=false] – признак того, что кнопка недоступна для нажатия. По умолчанию кнопка доступна для взаимодействия.

@property {boolean} [hidden=false] – признак того, что кнопка скрыта. По умолчанию кнопка отображается.

@property {function} onClick – обработчик события нажатия на кнопку. Входные параметры:

- **{TFormApi} formApi** – API для работы с диалогом;

- **{Function} [resolve]** – функция успешного выполнения. Передается только в обработчик кнопки с типом **submit**. Необходимо вызвать, чтобы обозначить завершение события и закрытие диалога. В качестве имени действия, в рамках которого закрывается диалог, используется **submit**. Аргумент игнорируется;
- **{Function} [reject]** – функция отклонения выполнения. Передается только в обработчик кнопки с типом **submit**. Необходимо вызвать, чтобы обозначить завершение события и разблокировку диалога. Если аргумент указан, в диалог добавляется общая ошибка с текстом из объекта ошибки.

ВАЖНО. При использовании кнопки типа **reset**:

- очищаются ошибки диалога. Если они были установлены, то в обработчике события необходимо вручную очистить их сохраненные идентификаторы;
- состояния **isReadOnly** и **isHidden** не сбрасываются до исходных значений. Если они менялись у элементов, то необходимо сбрасывать их вручную в обработчике кнопки.

TDialogButtonItemProperties – свойства кнопки

Наследуется от TDialogBaseItemProperties.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {string} [hint] – подсказка.

@property {string} [title] – текст кнопки.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге.

@property {string} [icon] – имя иконки, отображаемой в интерфейсе. Можно указать только стандартные иконки решения.

@property {function} [onClick] – обработчик нажатия на кнопку. Входные параметры:

- **{TFormApi} formApi** – API для работы с диалогом.

TDialogCheckboxProperties – флажок

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {boolean} [initialValue] – начальное значение.

@property {function} [onChange] – обработчик события изменения. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

TDialogCodeInputProperties – свойства поля ввода кода

Позволяет вводить только цифры. Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {number} [initialValue] – начальное значение.

@property {function} [onChange] – обработчик события изменения. Вызывается при вводе каждого символа. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {function} [onBlur] – обработчик события смены фокуса. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {number} [maxLength] – максимальное количество символов. По умолчанию не имеет ограничения. Принимает значение в виде целого положительного числа, иначе свойство будет игнорироваться.

TDialogCountdownProperties – свойства контрола обратного отсчета

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {string} countdownText – текст для отображения при обратном отсчете. Необходимо, чтобы в тексте был описатель в формате **{0}** для подстановки в таймер. При подстановке учитывается локализация.

@property {number} countdownInitValue – начальное значение обратного отсчета в секундах.

@property {string} refreshButtonCaption – заголовок кнопки сброса обратного отсчета.

@property {function} onRefreshButtonClick – обработчик нажатия на кнопку сброса обратного отсчета. Входные параметры:

- **refreshCallback** – callback для сброса обратного отсчета;

- **{TFormApi} formApi** – API для работы с диалогом.

TDialogDateIntervalProperties – свойства контрола выбора интервала дат

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {TDateInterval} [initialValue] – начальное значение интервала дат.

@property {function} [onChange] – обработчик события изменения. Вызывается при вводе или выборе корректной даты, удалении всех символов, потере фокуса с промежуточным значением. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {string} [minDate] – минимальная дата. Если значение не задано, то дата по умолчанию 1 января. Год высчитывается в соответствии с текущей датой и значением свойства **maxDate**:

- если значение **maxDate** не задано, из текущего года вычитается 10 лет. Например, для любой даты в 2024 году для **minDate** устанавливается значение 2014-01-01;
- если значение **maxDate** задано, из указанного в нем года вычитается 20 лет. Например, если значение **maxDate** 2024-06-01, то для **minDate** устанавливается значение 2004-01-01.

@property {string} [maxDate] – максимальная дата. Если значение не задано, то дата по умолчанию 31 декабря. Год высчитывается в соответствии с текущей датой и значением свойства **minDate**:

- если значение **minDate** не задано, к текущему году прибавляется 10 лет. Например, для любой даты в 2024 году для **maxDate** устанавливается значение 2034-12-31;
- если значение **minDate** задано, к указанному в нем году прибавляется 20 лет. Например, если значение **minDate** 2024-06-01, то для **maxDate** устанавливается значение 2044-12-31.

@property {number} [maxLength] – максимальное количество дней между датами. По умолчанию ограничения нет. Принимает значение в виде целого положительного числа, иначе будет использоваться значение по умолчанию.

TDialogDateProperties – свойства контрола выбора даты

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {string} [initialValue] – начальное значение. Рекомендуется указывать значения в формате ISO. Подробнее см. на сайте developers.mozilla.org, статья Date.prototype.toISOString().

@property {function} [onChange] – обработчик события изменения. Вызывается при вводе или выборе корректной даты, удалении всех символов, потере фокуса с промежуточным значением. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {string} [minDate] – минимальная дата. Если значение не задано, то дата по умолчанию 1 января. Год высчитывается в соответствии с текущей датой и значением свойства **maxDate**:

- если значение **maxDate** не задано, из текущего года вычитается 10 лет. Например, для любой даты в 2024 году для **minDate** устанавливается значение 2014-01-01;
- если значение **maxDate** задано, из указанного в нем года вычитается 20 лет. Например, если значение **maxDate** 2024-06-01, то для **minDate** устанавливается значение 2004-01-01.

@property {string} [maxDate] – максимальная дата. Если значение не задано, то дата по умолчанию 31 декабря. Год высчитывается в соответствии с текущей датой и значением свойства **minDate**:

- если значение **minDate** не задано, к текущему году прибавляется 10 лет. Например, для любой даты в 2024 году для **maxDate** устанавливается значение 2034-12-31;
- если значение **minDate** задано, к указанному в нем году прибавляется 20 лет. Например, если значение **minDate** 2024-06-01, то для **maxDate** устанавливается значение 2044-12-31.

TDialogFileInputProperties – свойства поля выбора файла.

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {function} [onChange] – обработчик события изменения. Входные параметры:

- **value** – новое значение элемента;

- **{TFormApi} formApi** – API для работы с диалогом.

@property {Array<string>} [fileTypes] – список допустимых расширений файлов. По умолчанию ограничений по типу файла нет.

@property {number} [maxFileCount=1] – максимальное количество файлов. По умолчанию равно **1**. Чтобы убрать ограничение, укажите **0**. Принимает положительное целое число, иначе будет использоваться значение по умолчанию.

@property {number} [maxFileSize] – максимальный размер файла в байтах. По умолчанию ограничение по размеру файла нет. Принимает положительное целое число, иначебудет использоваться значение по умолчанию.

TDialogHandlers – обработчики событий диалога

@property {function} onFormChange – обработчик события изменения диалога. Входные параметры:

- **{TFormApi} formApi** – API для работы с диалогом.

@property {function} onOpening – обработчик события, срабатывающий перед открытием диалога. Входные параметры:

- **resolve** – функция для открытия диалога;
- **reject** – функция для предотвращения дальнейшего открытия диалога.

@property {function} onOpened – обработчик события, срабатывающий после открытия диалога. Входные параметры:

- **{TFormApi} formApi** – API для работы с диалогом.

@property {function} onClosing – обработчик события, срабатывающий перед закрытием диалога. Входные параметры:

- **resolve** – функция для закрытия диалога;
- **reject** – функция для предотвращения дальнейшего закрытия диалога;
- **{TFormApi} formApi** – API для работы с диалогом;
- **actionName** – имя действия, в рамках которого закрывается диалог.

@property {function} onClosed – обработчик события, срабатывающий после закрытия диалога. Входные параметры:

- **actionName** – имя действия, в рамках которого закрывается диалог.

TDialogHint – подсказка к диалогу

@property {string} id – ИД подсказки.

@property {string} text – текст подсказки.

TDialogImageProperties – свойства изображения

@property {string} src – URL-адрес изображения.

@property {string} [alt] – альтернативное текстовое описание изображения.

@property {boolean} [withPreview=false] – признак того, что изображение можно открыть в режиме предпросмотра. Используется только на мобильных устройств. При значении **true** изображение отображается в миниатюре с текстом-подсказкой. При нажатии на миниатюру изображение откроется для предпросмотра в отдельном диалоге.

@property {string} [maxWidth=256px] – максимальная ширина изображения. Устанавливается при работе на компьютере. Используется при работе на мобильном устройстве, если не установлен признак **withPreview** или изображение открыто в диалоге предпросмотра. Значение рекомендуется указывать в пикселях. Значение по умолчанию **256px**.

@property {string} [minWidth=100px] – минимальная ширина изображения. Устанавливается при работе на компьютере. Используется при работе на мобильном устройстве, если не установлен признак **withPreview** или изображение открыто в диалоге предпросмотра. Значение рекомендуется указывать в пикселях. Значение по умолчанию **100px**.

TDialogInputProperties – свойства поля ввода

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {string} [initialValue] – начальное значение.

@property {function} [onChange] – обработчик события изменения. Вызывается при вводе каждого символа. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {function} [onBlur] – обработчик события смены фокуса. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {number} [maxLength] – максимальное количество символов. По умолчанию ограничения нет. Принимает значение в виде целого положительного числа, иначе свойство будет игнорироваться.

TDialogItem – элемент диалога

@property {DialogItemType} type – тип элемента диалога.

@property {string} name – имя элемента. Должно быть уникальным.

@property {boolean} [isRequired] – признак обязательности элемента. Если установлено значение **true**, выполняется проверка на заполненность элемента. Если установлено значение **false**, добавляется подсказка, о том что элемент необязателен для заполнения. По умолчанию свойство не заполнено: элемент необязателен для заполнения, при этом подсказка о необязательности не отображается.

@property {Array<Validator>} [validators] – дополнительные проверки валидности значения элемента.

@property {TDialogInputProperties **|** TDialogNumberInputProperties **|** TDialogCodeInputProperties**|** TDialogTextProperties **|** TDialogSelectProperties **|** TDialogKeyValueListProperties **|** TDialogSelectWithInfoBlockProperties **|** TDialogPreviewLinkProperties **|** TDialogRadioGroupProperties **|** TDialogDateProperties **|** TDialogDateIntervalProperties **|** TDialogCountdownProperties **|** TDialogCheckboxProperties **|** TDialogTextareaProperties **|** TDialogFileInputProperties **|** TDialogQuerySelectProperties **|** TDialogReferenceSelectProperties **|** TDialogImageProperties **|** TDialogMultilineLinkProperties **|** TDialogButtonItemProperties **} properties** – свойства элемента. Различаются в зависимости от типа элемента.

TDialogKeyValueListProperties – свойства списка пар ключ/значение

@property {Array<Object>} items – список элементов для отображения в формате **{ key: string, value: string }**.

TDialogMultilineLinkProperties – свойства ссылки с поддержкой многострочного текста

@property {string} href – URL-адрес ссылки.

@property {string} [text] – отображаемый текст ссылки. По умолчанию равен URL-адресу ссылки.

@property {number} [lines] – количество строк, в которых отображается ссылка. По умолчанию длина ссылки не меняется.

@property {boolean} [isBlank=true] – признак того, что страница по ссылке открывается в новом окне браузера. По умолчанию ссылка открывается в новом окне.

TDialogNumberInputProperties – свойства поля для ввода числового значения

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {string} [initialValue] – начальное значение.

@property {function} [onChange] – обработчик события изменения. Вызывается при вводе каждого символа. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {function} [onBlur] – обработчик события смены фокуса. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {boolean} [isInteger=false] – признак того, что в поле можно указать только целое число. По умолчанию допустимы дробные значения.

@property {number} [precision] – признак, указывающий на точность значения. Принимает целочисленные положительные значения. При указании некорректного значения или признака **isInteger=true** свойство не учитывается.

@property {number} [minValue] – минимально допустимое значение. Значение должно быть числом и меньше значения **maxValue**, если оно указано, иначе свойство не будет учитываться.

@property {number} [maxValue] – максимально допустимое значение. Значение должно быть числом и больше значения **minValue**, если оно указано, иначе свойство не будет учитываться.

TDialogPreviewLinkProperties – свойства ссылки для предпросмотра

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {StoredDocument **|** Document**} document** – документ для отображения в предпросмотре.

TDialogProperties – свойства диалога

@property {string} [height] – значение минимальной высоты диалога в веб-представлении. Указывается в пикселях. Для сжатия высоты по размеру содержимого используется значение **"auto"**.Ограничивается максимальной высотой в 740px. При указании в свойстве **width** значения **DialogWidth.LAYOUT** ограничение максимальной высоты для веб-представления увеличивается до 9/16 от максимальной ширины содержимого сайта. По умолчанию диалог растягивается, стремясь к максимальной высоте.

@property {boolean} [small=false] – признак уменьшенного размера диалога в мобильном представлении. Рекомендуется использовать для диалогов с выводом информации либо с минимальным количеством интерактивных элементов. По умолчанию в мобильном представлении отображается диалог стандартного размера.

@property {DialogWidth**} [width=DialogWidth.DEFAULT]** – ширина диалога. Значение по умолчанию – **DEFAULT**.

@property {boolean} [useConfirmationClose=true] – признак необходимости подтверждения при закрытии диалога. Окно подтверждения выводится, если хотя бы одно поле в диалоге было изменено.

@property {string} [referenceUrl] – ссылка на справку, которая отображается в диалоге. Можно указать адрес относительно параметра HelpServerUrl или полный адрес.

TDialogQuerySelectProperties – свойства выпадающего списка с запросом

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {string} [initialValue] – начальное значение.

@property {function} [onChange] – обработчик события изменения. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {string} dataSource – адрес источника данных. Источник должен возвращать массив строк в формате JSON или массив объектов.

@property {string} dataSourceCaptionField – имя поля в источнике данных, которое отображается для выбора в интерфейсе.

@property {string} dataSourceValueField – имя поля в источнике данных, которое сохраняется как значение параметра.

Если источник данных поддерживает поиск, то в этом параметре **dataSource** можно отобразить пользовательский ввод. Для этого используется макрос **{query}**, вместо которого подставляется введенное пользователем значение. Например:

```
/api/references?query={query}
```

В параметр запроса **query** передается введенное пользователем значение, и источник обрабатывает его как значение для поиска.

Если источник данных поддерживает постраничную выгрузку, то в этом параметре можно включить постраничную загрузку данных. Для этого используется макрос **{top}**, вместо которого подставляется количество элементов, которое необходимо загрузить. Например:

```
/api/references?takeCount={top}
```

В параметре **takeCount** указывается макрос **{top}**, отвечающий за количество элементов, которое необходимо загрузить. При открытии списка загружается **25** элементов. Когда достигнут конец списка, это количество увеличивается на 25, и происходит дозагрузка элементов.

Также при постраничной загрузке поддерживается работа с механизмом пропуска. Для этого используется макрос **{skip}**, вместо которого подставляется количество элементов, которое необходимо пропустить при загрузке. Например:

```
/api/references?takeCount={top}&skipCount={skip}
```

В параметре **skipCount** указывается макрос **{skip}**, отвечающий за количество элементов, которое необходимо пропустить. В приведенном примере **{top}** всегда равен **25**, а **{skip}** увеличивается на **25**.

Также поддерживается одновременная работа с поиском и постраничной загрузкой:

```
/api/references?query={query}&takeCount={top}&skipCount={skip}
```

TDialogRadioGroupProperties – свойства группы переключателей

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {string} [initialValue] – начальное значение.

@property {Array<Object>} options – опции для выбора. Массив объектов с опциями в формате { label: string, value: string }.

@property {function} [onChange] – обработчик события изменения. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

TDialogReferenceSelectProperties – свойства выпадающего списка с выбором из справочника

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {string} [initialValue] – начальное значение.

@property {function} [onChange] – обработчик события изменения. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {string} displayRequisite – отображаемый реквизит.

@property {string} storeRequisite – хранимый реквизит.

@property {string} systemName – имя целевой системы.

@property {string} referenceName – имя справочника.

@property {boolean} [isUserQuerySupported=false] – признак отображения поля для ввода значения, по которому фильтруется список. По умолчанию поле не отображается.

@property {string} [filter] – строка фильтра.

TDialogSelectProperties – свойства выпадающего списка

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {Array<Object>} options – опции для выбора. Массив объектов с опциями в формате **{ caption: string, value: string }**;

@property {string} [initialValue] – начальное значение;

@property {function} [onChange] – обработчик события изменения. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

TDialogSelectWithInfoBlockProperties – свойства выпадающего списка с информационным блоком

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {string} [initialValue] – начальное значение.

@property {string} valueField – поле, используемое как значение в списке объектов для выбора.

@property {string} captionField – поле, используемое как текст в списке объектов для выбора.

@property {Object} infoKeyValues – объект с элементами для отображения в формате ключ-значение.

@property {Array<Object>} options – опции для выбора. Массив объектов с опциями в формате ключ-значение.

@property {function} [onChange] – обработчик события изменения. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

TDialogSettings – настройки системного диалога

@property {boolean} [small=false] – признак того, что в мобильном представлении диалог отображается с уменьшенной высотой.

@property {string} [height] – значение минимальной высоты диалога в веб представлении. Указывается в пикселях. Для сжатия высоты под содержимое используется значение **"auto"**. Ограничивается максимальной высотой в 740px. По умолчанию диалог растягивается, стремясь к максимальной высоте.

TDialogTextareaProperties – свойства многострочного поля ввода

Наследуется от TDialogBaseItemProperties.

@property {string} [title] – заголовок.

@property {boolean} [isReadOnly=false] – признак того, что элемент доступен только для чтения. По умолчанию элемент доступен для взаимодействия.

@property {boolean} [isHidden=false] – признак того, что элемент скрыт. По умолчанию элемент отображается в диалоге. Если элемент скрыт, его валидация не выполняется.

@property {string} [hint] – подсказка.

@property {string} [initialValue] – начальное значение.

@property {function} [onChange] – обработчик события изменения. Вызывается при вводе каждого символа. Входные параметры:

- **value** – новое значение элемента;

- **{TFormApi} formApi** – API для работы с диалогом.

@property {function} [onBlur] – обработчик события смены фокуса. Входные параметры:

- **value** – новое значение элемента;
- **{TFormApi} formApi** – API для работы с диалогом.

@property {number} [rows=3] – максимальное количество отображаемых строк, по умолчанию равно **3**. Принимает значение в виде целого положительного числа, иначе будет использоваться значение по умолчанию. В мобильном представлении ограничения по строкам нет.

@property {number} [maxLength] – максимальное количество символов. По умолчанию ограничения нет. Принимает значение в виде целого положительного числа, иначе свойство будет игнорироваться.

TDialogTextProperties – свойства текстового поля

@property {string} text – текст для отображения. Поддерживается использование поддержка mustache-шаблона со значениями всех элементов диалога.

TFormApi – API для работы с формой диалога

@property {TFormItemsApi} items – функции для работы с элементами формы диалога.

@property {TFormButtonsApi} buttons – функции для работы с кнопками.

@property {TFormErrorApi} errors – функции для работы с ошибками формы диалога.

@property {TFormDialogApi} dialog – функции для работы с диалогом.

TFormButtonsApi – функции для работы с кнопками

@property {function} get – получение свойств кнопки по ее имени. Входной параметр: **name** – название кнопки.

@property {function} getAll – получение свойств всех кнопок.

@property {function} setDisabled – функция для блокирования кнопки диалога. Входной параметр: **name** – название кнопки. Также принимает значение **ALL** из списка констант для изменения свойства всех кнопок диалога.

@property {function} setEnabled – функция для разблокирования кнопки диалога. Входной параметр: **name** – название кнопки. Также принимает значение **ALL** из списка констант для изменения свойства всех кнопок диалога.

@property {function} setVisible – функция для отображения кнопки диалога. Входной параметр: **name** – название кнопки. Также принимает значение **ALL** из списка констант для изменения свойства всех кнопок диалога.

@property {function} setHidden – функция для скрытия кнопки диалога. Входной параметр: **name** – название кнопки. Также принимает значение **ALL** из списка констант для изменения свойства всех кнопок диалога.

Пример вызова функций по работе с кнопками диалога:

```
dialogApi.buttons.setDisabled(ESS.constants.ALL);
dialogApi.buttons.setEnabled('nameButton');
```

TFormDialogApi – функции для работы с диалогом

@property {function} setDisabled – функция для блокирования всех элементов и кнопок диалога.

@property {function} setEnabled – функция для разблокирования всех элементов и кнопок диалога.

@property {function} isDisabled – функция для получения признака, что диалог заблокирован.

@property {function} close – функция для закрытия диалога. Входной параметр: **{string} [actionName]** – название действия, по которому происходит закрытие.

@property {function} showLoader – функция для отображения индикатора прогресса. Входной параметр: **{string} [loaderId]** – ИД индикатора.

@property {function} hideLoader – функция для скрытия индикатора прогресса. Входной параметр: **{string} [loaderId]** – ИД индикатора.

@property {function} showHint – функция для отображения подсказки к диалогу Подсказка отображается только при заблокированном диалоге или при наличии индикатора прогресса. При этом ошибки, относящиеся ко всему диалогу, скрываются. Входной параметр: **{TDialogHint} [hint]** – подсказка.

@property {function} hideHint – функция для скрытия подсказки. Входной параметр: **{string} [hintId]** – ИД подсказки.

Пример 1. Заблокировать все элементы и кнопки диалога на 1000 мс:

```
dialogApi.dialog.setDisabled();
setTimeout(dialogApi.dialog.setEnabled, 1000);
```

Пример 2. Отобразить подсказку к диалогу на 1000 мс:

```
dialogApi.dialog.showHint({id: 'hint_01', text: 'Идет получение данных. Пожалуйста, подождите.'});
setTimeout(dialogApi.dialog.hideHint('hint_01'), 1000);
```

TFormErrorApi – функции для работы с ошибками формы диалога

@property {function} getAll – получение массива со всеми ошибками.

@property {function} add – добавление новой ошибки. Входные параметры:

- **text** – текст ошибки;
- **type** – необязательное поля с типом ошибки.

Функция возвращает идентификатор созданной ошибки, по которому ее после можно будет удалить.

@property {function} remove – удаление ошибки по ее идентификатору. Входной параметр: **id** – идентификатор ошибки.

@property {function} hasValidationErrors – функция, возвращающая признак наличия ошибок валидации.

Пример вызова функции по работе с ошибками диалога:

```
var errorId = dialogApi.errors.add('Error');
dialogApi.errors.remove(errorId);
```

TFormItemsApi – функции для работы с элементами формы диалога

@property {function} getValues – получение значений всех элементов диалога.

@property {function} getValue – получение значения элемента диалога или ячейки таблицы. Входные параметры:

- **name** – название элемента диалога;
- **[rowIndex]** – индекс строки таблицы;
- **[columnName]** – имя колонки таблицы.

@property {function} setValue – изменение значения элемента диалога. Входные параметры:

- **name** – название элемента диалога;
- **value** – новое значение элемента диалога;
- **[rowIndex]** – индекс строки таблицы;
- **[columnName]** – название колонки таблицы.

@property {function} get – получение свойств элемента диалога. В качестве результата возвращается объект с базовым набором полей **type**, **name**, **isRequired**, а также свойствами **properties**, зависящими от типа элемента. Возможные значения поля **properties** см. в разделе «enums – перечисления», объект DialogItemProperties. Входной параметр: **name** – название элемента диалога.

@property {function} getAll – получение свойств всех элементов диалога. В качестве результата возвращается массив объектов с базовым набором полей **type**, **name**, **isRequired**, а также свойствами **properties**, зависящими от типа элемента. Возможные значения поля **properties** см. в разделе «enums – перечисления», объект DialogItemProperties

@property {function} setDisabled – функция для блокирования элемента диалога. Входные параметры:

- **name** – название элемента диалога. Также принимает значение **ALL** из списка констант для изменения свойства всех элементов диалога;
- **[columnName]** – название колонки таблицы.

@property {function} setEnabled – функция для разблокирования элемента диалога. Входные параметры:

- **name** – название элемента диалога. Также принимает значение **ALL** из списка констант для изменения свойства всех элементов диалога;
- **[columnName]** – название колонки таблицы.

@property {function} setVisible – функция для отображения элемента диалога. Входные параметры:

- **name** – название элемента диалога. Также принимает значение **ALL** из списка констант для изменения свойства всех элементов диалога;
- **[columnName]** – название колонки таблицы.

@property {function} setHidden – функция для скрытия элемента диалога. Входные параметры:

- **name** – название элемента диалога. Также принимает значение ALL из списка констант для изменения свойства всех элементов диалога;
- **[columnName]** – название колонки таблицы.

@property {function} setProperty – функция для изменения свойств элемента диалога. Входным параметром является объект с ключами:

- **name** – название элемента диалога;
- **[columnName]** – название колонки таблицы;
- **propertyName** – название изменяемого свойства. Возможные значения см. в разделе «enums – перечисления», объект DialogItemProperties;
- **value** – новое значение элемента диалога.

@property {function} addRow – асинхронная функция для добавления строки в таблицу. Позволяет дождаться окончания добавления строки перед последующими операциями. Входные параметры:

- **tableName** – название таблицы.

@property {function} removeRow – асинхронная функция для удаления строки из таблицы. Позволяет дождаться окончания удаления строки перед последующими операциями. Входные параметры:

- **tableName** – название таблицы.

Пример вызова функции по работе с элементами диалога:

```
var itemName = 'itemName';
dialogApi.items.setDisabled(ESS.constants.ALL);
dialogApi.items.setEnabled(itemName);
dialogApi.items.setProperty({
 itemName,
 propertyName: ESS.enums.DialogItemProperties[ESS.enums.DialogItemType.INPUT].TITLE,
 value: 'Новый текст заголовка'
});
```

TOverlayOptions – опции оверлея new

@property {boolean} [blockUnload] – признак блокировки события **unload** страницы. Если значение признака **true**, то при попытке закрыть или обновить страницу с отображенным оверлеем появляется браузерное предупреждение о потере данных.

TSetCellDataSourceParams – параметры функции setCellDataSource

@property {string} gridName – имя таблицы.

@property {string} gridRow – индекс строки таблицы.

@property {string} gridColumn – имя столбца.

@property {string} newDataSource – новый источник данных.

@property {string | null} [newValue] – новое значение ячейки.

@property {string} [newDataSourceSingle] – новый источник данных для получения одиночного элемента.

@property {boolean} [shouldSaveValue] – признак необходимости сохранить ранее выбранное значение. Если признак установлен, значение параметра **newValue** игнорируется. Особенности сохранения значения аналогичны особенностям установки нового значения.

Подробнее см. в описании функции setCellDataSource.

TSetParameterDataSourceParams – параметры функции setDataSource

@property {string} parameterName – имя параметра.

@property {string} newDataSource – новый источник данных.

@property {string | null} [newValue] – новое значение параметра.

@property {string} [newDataSourceSingle] – новый источник данных для получения одиночного элемента.

@property {boolean} [shouldSaveValue] – признак необходимости сохранить ранее выбранное значение. Если признак установлен, значение параметра **newValue** игнорируется. Особенности сохранения значения аналогичны особенностям установки нового значения.

Подробнее см. в описании функции setDataSource.

TSystemDialog – параметры системного диалога

@property {string} text – текст диалога.

@property {string} title – заголовок диалога.

@property {ButtonsType} [buttonsType] – набор кнопок. По умолчанию отображается кнопка **OK**.

@property {function} [onCloseCallback] – обработчик события закрытия. Доступен параметр **buttonName** с названием кнопки, по которой произошло закрытие.

@property {TDialogSettings} [dialogSettings] – настройки диалога.

Validator – проверка корректности значения элемента

@property {unknown} value – значение элемента.

@returns {FormError} – возвращается при некорректном значении элемента диалога.

Пример:

```
value => {
  if (!value) return;
  const numberValue = Number(value);
  if (numberValue < 0 || !Number.isInteger(numberValue)) {
    return {
      text: ESS.L('FORM.DATETIMEINTERVAL_DURATION_INVALID'),
      type: ESS.enums.DialogErrorType.FORM_VALIDATION
    }
  }
}
```

Лямбда-функции, доступные в mustache-шаблонах

ВАЖНО. Содержимое функции обязательно должно оформляться в одну строку.

reduceFullName – сократить полное имя

Функция преобразует полное имя в формате <Фамилия Имя Отчество> в короткое в формате <Фамилия И.О.>.

Пример:

```
{{#reduceFullName}}{{performerFullName}}{{/reduceFullName}}
```

simplifyDate – вернуть локализованную дату в виде строки

Локализация соответствует языку приложения. Если время нулевое, строка будет в упрощенном виде (без времени). Результат соответствует результату метода toLocaleDateString, если время нулевое, и методу toLocaleString, если отличное от нуля. Формат данных должен соответствовать поддерживаемым в конструкторе js-объекта Date. Рекомендуется использовать ISO-формат.

Подробнее см. на сайте developer.mozilla.org, статьи Date.prototype.toLocaleDateString(), Date.prototype.toLocaleString(), Date() constructor, Date.prototype.toISOString().

localizeDate – вернуть локализованную дату в виде строки

Локализация соответствует языку приложения. Результат соответствует результату метода toLocaleDateString. Формат данных должен соответствовать поддерживаемым в конструкторе js-объекта Date. Рекомендуется использовать ISO-формат.

localizeDateTime – вернуть локализованную дату со временем в виде строки

Локализация соответствует языку приложения. Результат соответствует результату метода toLocaleString. Формат данных должен соответствовать поддерживаемым в конструкторе js-объекта Date. Рекомендуется использовать ISO-формат.

getColor – вернуть цвет из палитры по указанному токену

ВАЖНО. В функции **getColor** должно содержаться название токена. Список токенов см. разделе «Colors».

Пример:

```
{{#getColor}}Primary50Color{{/getColor}}
```

Прикладные клиентские конвертеры new

Имена лямбда-функций для вызова прикладных клиентских конвертеров формируются по шаблону: <Имя решения>_<Имя конвертера>.

Пример использования конвертера **toLongMonthYear** из решения **EssBase**:

```
{{#EssBase_toLongMonthYear}}{{date}}{{/EssBase_toLongMonthYear}}
```

Mustache-шаблон

Mustache — это шаблонизатор, который позволяет встраивать данные в HTML-разметку. В качестве входных данных выступает контекст родительского компонента. Подробнее см. в документации Mustache на сайте https://mustache.github.io/.

Основные возможности Mustache:

- подстановка переменных, полученных из контекста родительского компонента:

```
<![CDATA[
  <p>Name: {{userName}}</p>
]]>
```

- условные блоки:

```
<![CDATA[
  {{#isActive}}
    <p>User is active.</p>
  {{/isActive}}
]]>
```

- итерация по массивам:

```
<![CDATA[
  <ul>
    {{#.}}
      <li>{{todo}}</li>
    {{/.}}
  </ul>
]]>
```

- встраивание сторонних компонентов:

```
<![CDATA[
  <component
    name="example"
    public-name="Directum_ExampleComponent_1_0"
    module-name="loaders"
    loader-name="for-test"
  />
]]>
```

- использование лямбда-функций:

```
<![CDATA[
  <p>Date: {{#localizeDate}}{{today}}{{/localizeDate}}</p>
]]>
```

Стилизация элементов в Mustache-шаблонах

Раздел содержит описания прикладных CSS-классов. Чтобы применить класс, оберните содержимое шаблона в HTML-элемент (рекомендуется **div**) и укажите для него класс.

ess-template-widget-padded

Класс для добавления отступов контента виджета типа templateWidget.

```
<widget name="MustacheTemplatePadded" type="templateWidget">
  <icon>check-mark</icon>
  <title>Виджет с шаблоном (с отступами без скролла)</title>
  <dataSourceUrl>https://jsonplaceholder.typicode.com/users</dataSourceUrl>
  <properties>
```

```
    <Template>
      <![CDATA[
        <div class="ess-template-widget-padded">
          {{#.}}
            Name: {{name}} (<a href="mailto:{{email}}">{{email}}</a>)<br/>
          {{/.}}
        </div>
      ]]>
    </Template>
  </properties>
</widget>
```

ess-template-widget-scrollable

Класс для добавления вертикального скролла в контенте виджета типа templateWidget.

В примере используются классы ess-template-widget-padded и ess-template-widget-scrollable:

```
<widget name="MustacheTemplatePaddedScrollable" type="templateWidget">
  <icon>check-mark</icon>
  <title>Виджет с шаблоном (с отступами и скроллом)</title>
  <dataSourceUrl>https://jsonplaceholder.typicode.com/users</dataSourceUrl>
  <properties>
    <Template>
      <![CDATA[
        <div class="ess-template-widget-padded ess-template-widget-scrollable">
          {{#.}}
            Name: {{name}} (<a href="mailto:{{email}}">{{email}}</a>)<br/>
          {{/.}}
        </div>
      ]]>
    </Template>
  </properties>
</widget>
```

ess-no-data

Класс для добавления стилей блока с сообщением об отсутствии данных.

```
<widget name="MustacheTemplate" type="templateWidget">
  <icon>check-mark</icon>
  <title>Виджет с шаблоном</title>
  <dataSourceUrl>https://jsonplaceholder.typicode.com/users</dataSourceUrl>
  <properties>
    <Template>
      <![CDATA[
        {{^data}}
          <div class="ess-no-data">Нет данных</div>
        {{/data}}
      ]]>
    </Template>
  </properties>
</widget>>
```

ess-badge

Класс для добавления стилей блока исполнителя.

```
<widget name="MustacheTemplate" type="templateWidget">
  <icon>check-mark</icon>
```

```
  <title>Виджет с шаблоном</title>
  <dataSourceUrl>https://jsonplaceholder.typicode.com/user</dataSourceUrl>
  <properties>
    <Template>
      <![CDATA[
        <div class="ess-badge">
          {{#FullName}}
            <span>{{FullName}}</span>
          {{/FullName}}
          {{#Organization}}
            <span>{{Organization}}</span>
          {{/Organization}}
        </div>
      ]]>
    </Template>
  </properties>
</widget>
```

Прикладные CSS-классы для типографики

ess-text-h1 – для первого уровня.

ess-text-h2 – для второго уровня.

ess-text-h3 – для третьего уровня.

ess-text-h4 – для четвертого уровня.

ess-text-h4-strong – для четвертого уровня с полужирным начертанием.

ess-text-h5 – для пятого уровня.

ess-text-h5-strong – для пятого уровня с полужирным начертанием.

ess-text-h6 – для шестого уровня.

ess-text-primary – для стилей параграфа.

ess-text-primary-strong – для стилей параграфа с полужирным начертанием.

ess-text-secondary – для стилей вспомогательного текста.

Описание HTML-элементов для оформления текста на сайте личного кабинета см. в разделе«Рекомендации по оформлению текста».

Рекомендации по оформлению текста

Для оформления текста на сайте личного кабинета используйте HTML-элементы:

- **h1** – заголовок первого уровня, по внешнему виду эквивалентен прикладному классу ess-text-h1;
- **h2** – заголовок второго уровня, по внешнему виду эквивалентен прикладному классу ess-text-h2;
- **h3** – заголовок третьего уровня, по внешнему виду эквивалентен прикладному классу ess-text-h3;
- **h4** – заголовок четвертого уровня, по внешнему виду эквивалентен прикладному классу ess-text-h4;

- **h5** – заголовок пятого уровня, по внешнему виду эквивалентен прикладному классу ess-text-h5;
- **h6** – заголовок шестого уровня, по внешнему виду эквивалентен прикладному классу ess-text-h6;
- **p** – параграф, по внешнему виду эквивалентен прикладному классу ess-text-primary;
- **ul** – маркированный список;
- **ol** – нумерованный список;
- **a** – ссылка;
- **b** – полужирное начертание текста;
- **i** – наклонное начертание текста;
- **div** – элемент без стилей. Для корректного отображения текста, который находится внутри этого блока, необходимо использовать прикладные классы, которые начинаются с **ess-text-**.

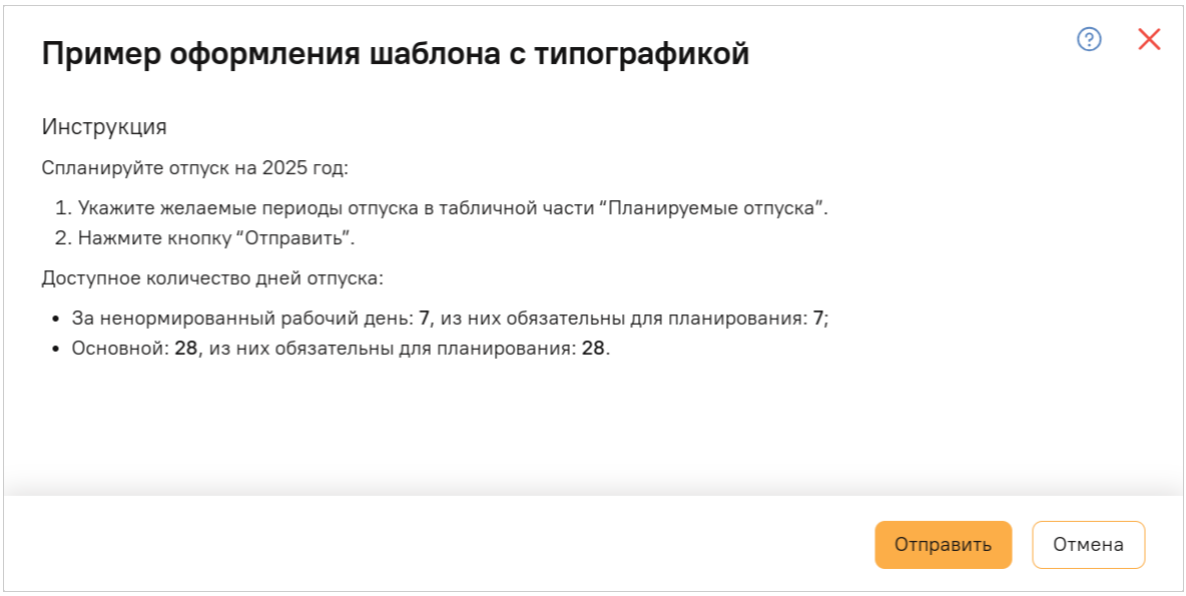
  ПРИМЕЧАНИЕ. Элементы **a**, **b** и **i** не отображаются на сайте отдельными текстовыми блоками. Для корректного отображения их необходимо использовать только внутри других HTML-элементов. Рекомендуется **p** – тег объединение текста в параграф. В этом случае идущие рядом теги **p** разделяются переносом строки.

Для сохранения размерности текста на сайте, в mustache-шаблонах рекомендуется использовать заголовки, начиная с третьего уровня. Использование стилей первого и второго уровней рекомендуется только для названий прикладных страниц и основных разделов.

Подробнее о классах для добавления стилей заголовков см. в разделе «Описание прикладных CSS-классов».

Пример оформления инструкции для услуги:

```
<facility name="Typography">
  <title>Пример оформления шаблона с типографикой</title>
  <facilityFormItems>
    <facilityFormItem type="FacilityFormTextblock">
      <content>
        <![CDATA[
          <h3>Инструкция</h3>
          <p>Спланируйте отпуск на 2025 год:</p>
          <ol>
            <li>Укажите желаемые периоды отпуска в табличной части “Планируемые отпуска”.</li>
            <li>Нажмите кнопку “Отправить”.</li>
          </ol>
          <p>Доступное количество дней отпуска:</p>
          <ul>
            <li>За ненормированный рабочий день: <b>7</b>, из них обязательны для
планирования: <b>7</b>;</li>
            <li>Основной: <b>28</b>, из них обязательны для планирования: <b>28</b>.</li>
          </ul>
        ]]>
      </content>
    </facilityFormItem>
  </facilityFormItems>
</facility>
```


Пример оформления шаблона с типографикой
Инструкция
Спланируйте отпуск на 2025 год:
1. Укажите желаемые периоды отпуска в табличной части “Планируемые отпуска”.
2. Нажмите кнопку “Отправить”.
Доступное количество дней отпуска:
• За ненормированный рабочий день: 7, из них обязательны для планирования: 7;
• Основной: 28, из них обязательны для планирования: 28.
Отправить
Отмена

Физическая структура базы данных

Вся прикладная конфигурация размещается в базе данных в следующих таблицах:

TargetSystems	Содержит настройки точек подключения к целевым системам
LocalizedStrings	Строки локализации
UsageAgreements	Пользовательские соглашения
FacilityGroups	Группы услуг
Tiles	Плитки быстрых действий
Facilities	Услуги
NotificationTemplates	Шаблоны оповещений
ObjectCards	Карточки объектов
Projections	Проекции прикладных объектов
Pages	Прикладные страницы
Components	Сторонние компоненты
ComponentContents	Дополнительные файлы стороннего компонента
Solutions	Прикладные решения
Converters	Прикладные конвертеры
Scripts	Прикладные сценарии
Widgets	Виджеты
Functions	Прикладные функции